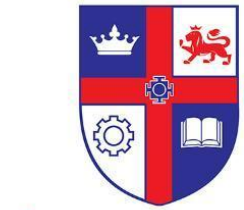
LONDON METROPOLITAN UNIVERSITY



Islington college
(इस्लिङ्टन कलेज)

CS4001NI Programming

60% Individual Coursework

2025 Spring

Student Name: Kritika Basel

London Met ID: 24046774

College ID: NP01CP4A240157

Group: L1C6

Assignment Due Date: Friday, May 16, 2025

Assignment Submission Date: Friday, May 16, 2025

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded

24016774_kritikabasel.docx

Islington College,Nepal

Document Details

Submission ID
trn:oid:::3618:96180414

Submission Date
May 16, 2025, 11:06 AM GMT+5:45

Download Date
May 16, 2025, 11:08 AM GMT+5:45

File Name
24016774_kritikabasel.docx

File Size
82.7 KB

103 Pages

9,756 Words

55,734 Characters

turnitin Page 1 of 107 - Cover Page Submission ID trn:oid:::3618:96180414

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **55** Not Cited or Quoted 8%
  Matches with neither in-text citation nor quotation marks
- **7** Missing Quotations 1%
  Matches that are still very similar to source material
- **7** Missing Citation 1%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

3% Internet sources
0% Publications
8% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 55 Not Cited or Quoted 8%
  Matches with neither in-text citation nor quotation marks
- 7 Missing Quotations 1%
  Matches that are still very similar to source material
- 7 Missing Citation 1%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

3% Internet sources
0% Publications
8% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	islingtoncollege on 2025-05-16 — 3%
2	Internet	www.geeksforgeeks.org — <1%
3	Submitted works	iacademy on 2025-05-14 — <1%
4	Internet	balsamiq.com — <1%
5	Internet	slidetodoc.com — <1%
6	Submitted works	HAEF on 2024-03-20 — <1%
7	Submitted works	The Robert Gordon University on 2022-08-04 — <1%
8	Internet	www.java2s.com — <1%
9	Submitted works	CSU, Long Beach on 2016-02-07 — <1%
10	Submitted works	University of Technology, Sydney on 2006-09-21 — <1%

11 Internet
coderanch.com <1%

12 Submitted works
University of Sussex on 2024-04-08 <1%

13 Submitted works
Staffordshire University on 2016-01-15 <1%

14 Internet
www.coursehero.com <1%

15 Submitted works
University of Wollongong on 2021-04-02 <1%

16 Internet
online.une.edu <1%

17 Submitted works
Ace Acumen Academy on 2024-03-19 <1%

18 Submitted works
Michigan Virtual on 2024-02-14 <1%

19 Submitted works
Edith Cowan University on 2017-05-10 <1%

20 Internet
mu.ac.in <1%

21 Submitted works
Colorado Technical University Online on 2014-05-13 <1%

Table of Contents

..... **Error! Bookmark not defined.**
1. Introduction 1
   1.1. Aims and objective 2
   1.2. Verdict 2
   1.3. Tools Used 3
2. GUI Wireframe 6
   2.1. GUI 9
3. Class Diagram 13
4. Pseudocode 20
   4.1. Pseudocode of Gym Member 20
   4.2. Pseudocode of premium member 23
   4.3. Pseudocode of Regular Member 27
   4.4. Pseudocode for gym GUI 31
5. Method Description 81
6. Testing 86
   Test 1: Compile and run using command prompt 86
   Test 2: Add regular and premium member 88
   Test 3: Mark Attendance and upgrade plan 95
   Test 4: Calculate discount, pay due and revert member 101
   Test 5: Save and read from file 107
7. Error detection and correction 109
8. Conclusion 113
9. References 115
10. Appendix 116
   10.1. CODE FOR GYM MEMBER 116
   10.2. CODE FOR PREMIUM MEMBER 120
   10.3. CODE FOR REGULAR MEMBER 124
   10.4. CODE FOR GYM GUI 129

Table of Tables

Table 1:Class Diagram of Gym Member 13
Table 2:Class Diagram of Premium Member 14
Table 3:Class Diagram of Regular Member 15
Table 4:Class diagram of Gym GUI 16
Table 5:GYM Member method 81
Table 6:RegularMember method 82
Table 7:Premium method 83
Table 8:GUI Methods 84
Table 9:Compile and run using command prompt 86
Table 10: Add regular and premium member 88
Table 11:Mark Attendance and upgrade plan 95
Table 12:pay due amt ,discount revert member 101
Table 13:save and read from the file 107

Table of Figures

Figure 1:Wireframe of GUI 7
Figure 2:Wireframe of Premium Member 8
Figure 3:Wireframe of RegularMember 9
Figure 4:GUI of Main 10
Figure 5:GUI of premium Member 11
Figure 6:GUI of Regular member 12
Figure 7:Class diadram(1) 19
Figure 8:Class Diagram(2) 19
Figure 9:Command Prompt 86
Figure 10: Command Prompt(2) 87
Figure 11:(Test 2)Invalid Display 89
Figure 12:(Test 2)phone number required 90
Figure 13:(Test 2)Email required 91
Figure 14:(test 2)Regular Member added 92
Figure 15:(Test 2) Premium member added 93
Figure 16:(test2) regular display 94
Figure 17:(Test 2)Premium Display 94
Figure 18:atendence not being marked 96
Figure 19:attendance marked 97
Figure 20:new plan input 98
Figure 21:updated plan 98
Figure 22:updating into same 99
Figure 23:wrong value in plan 100
Figure 24:(Test 4):pay due balance 102
Figure 25:(Test 4)Not paid 103
Figure 26:Payment successful 103
Figure 27:full payment done 104
Figure 28:adding more payment 105
Figure 29:discount after full payment 106
Figure 30 :Revert Memership 106
Figure 31:(Test 5)save to file 107
Figure 32:(Test 5)Read fro file 108
Figure 33:symtax error 109
Figure 34:syntax error solved 109
Figure 35:attendance marked error 110
Figure 36:Logical error 110
Figure 37:logical error solved 111
Figure 38:deactivate not functioning 111

Figure 39:runtime error..... 112
Figure 40:runtime error solved 112

1. Introduction

In today's growing era technology of the basic things have been modernized with the help of few software many people's Lives have been much easier. The connection between humans and electronic devices, software has been progressively better and has made human life far better than we can ever imagine. The assigned coursework of Programming module which is to develop a system for implementing the different OOP, attributes, methods to turn it into a functional real-life program to take user input and to perform specific tasks using java programming language. The development of such program requires a certain level of research, understanding, algorithm, and problem-solving skill which the assigned course work has helped to established.the designed program was assigned for the imp,lamentation of the program, development of logic building and many more.the development of the technology has made everyone very dependent on many devices in where software,programs also takes place so for the real life implementation of the software and handling of the the data,working with the system and being familiar with it is very important and essential.

This individual course was assigned for the purpose of improving the necessary skills which required making an entire user friendly program which include to make the three Java classes gym member, premium member and regular member and to optimize those classes and make it functional the Gym GUI is also developed where user interaction I actually performed.in this program we were assigned to make a gym membership's program where people can easily register and perform different functionality with the help of different methods and variables. In the process of the development of the programs stud3nts can enhance their learning and fulfil the objective of the module to gain the knowledge on JAVA. With the help of this we can learn the OOP ,implementation of different methods, and work with the functioning program.it helps to overcome the possible difficulties with ease with proper practice and knowledge the task can be performed very smoothly. The knowledge on the different methods, classes and the use of inheritance, polymorphism can be performed easily. The data handling is also implement using the i/o operations and has gave access to enable member records to

read,displayed and save for future use. The core understanding of Java features and OOP concepts and practical problem solving for managing gym operations is the aim of the project and it offers hand on learning experience by building a real worls application.

1.1. Aims and objective

The major goal of the program is to create a project consisting the terminal for the gym to take the required information from their user to make them the part of the member of the gym. The aim covering a large area of the implementation of the Java programming language for the calculation and the data can be easily taken and store without any fear of complication, missing the required information. The program is developed with the aim of being used in a very large-scale networking area as it is developed to make tasks easier and efficient. The objective of the project is to develop a certain level of understanding of the programming language Java. The project helps to check the knowledge while developing much more skills for the further projects and even in the work field. the basic objective of the projects is:

- Implementation of inheritance, Polymorphism and other OOPs,
- Enhancing the logic and problem-solving skill
- Encapsulating using the getter method while protecting the data,
- Work with JDK and Java control statement.

1.2. Verdict

With the plan and the aim of the project the development of the classes with the methods, attributes the system tends to be developed smoothly and it will be user friendly for the input of the information and the calculation ang the optimization of the data in the required fields. The program is to be develop with a strategy of development in the data survey and other big gathering of the data while processing and passing the data more easily. With the use of the tools the implementation of the strategies will be as planned to the final result. The use of blueJ helps to implement the OOP language and passing and initializing the values and methods. At the end of the project, it is expected to gain a fruitful outcome with the planning and the outcome.

1.3. Tools Used

1.1.1. Word



Microsoft word is a documenting the data which helps to save the information .Microsoft Word is a word processing program which helps to edit text. In this project this software helped to write the documentation part. This is the most important task to write a report on the stuffs that you have made project on. So,the Word helps to make that thing possible.

1.1.2. BlueJ



BlueJ is a windows-based platform for Java Development Kit (JDK). It was started by Michael Kollin and john Rosenberg at Monash University, Austrailia in 1999. It can be freely downloaded from its official website of BlueJ. It was developed to support learning and teaching of OOPs (object-oriented programming) i.e. Encapsulation, polymorphism, inheritance, class, object. The objects can be interactively created and tested. It is a simple user interface. BlueJ has a simpler interface than most professional IDEs. It offers many tools that are specific to its educational goals. There are also standard development tools available, such as an editor, compiler and runtime environment (GeeksforGeeks, 2023).

In this coursework, every java programming is done in this application. So, it helps for the program to perform giving it a space to do its task. helped to implement the OOP and other factors of the Java.

1.1.3. Balsamiq



Balsamiq is a user interface design tool for creating wire frames (sometimes called mockups or low-fidelity prototypes).

You can use it to generate digital sketches of your idea or concept for an application or website, and to facilitate discussion and understanding before any code is written. The completed wireframes can be used for user testing, clarifying your vision, getting feedback from stakeholders, or getting approval to start development. (Balsamiq, 2025)

I used balsamiq in this project in order to create the wireframe for the Gym GUI. The Gym Gui is an interface between the user and the system. For the system to have and process the data the GUI has to take the data from the user. So, it is important for the wireframe to help guide for the final product to be just as good as the wire frame.

1.1.4. Draw.io



Draw.io is a free, open source diagramming application which is available as a web application or desktop app.it allows users to create edit and collaborate on various type of diagrams, flowchart and many more. It can be used for various purpose such as designing, creating, and visualizing networks as well. I have used this in the course work to make the class diagram which displays the class, variables ,

methods in the class inside a table which helps to know the access modifier, relations and many more.

2. GUI Wireframe

The wireframe of the Gym GUI is the structure and connective pathway between the user and the software which also reflects the actual results. It helps to structure the final outcome as expected with meaningful process and logic. It turns the imagination to the real life, functionable software. At first, in the GUI part must include the user input part where the necessary text fields as id, name, location, phone, email, DOB, membership Start Date, referral source, paid amount, removal reason, trainers name, plan, plan price and some additional buttons. These all text fields should be aligned vertically in a significant straight order. All the text fields should be functional where it should take correct input from the user and guide the once who are finding the required information difficult. This is the design idea for the Graphical user interface to make the software smoother and efficient. The wireframe design should be fare and understandable as it is the plan of the final outcome and the more it is clear the more easier it will be for the user for the data input and the main goal is to provide efficiency, ease to the user.

In this project, there are three GUI frames which is separated by inserting the data and display according to the type of membership choose. In the first frame of the user can input the data and can add it, save it, read it in a separate frame and in the nest two frame it displays members information according to the type of plan and has all these button like mark attendance, activate, deactivate to make the page dynamic.

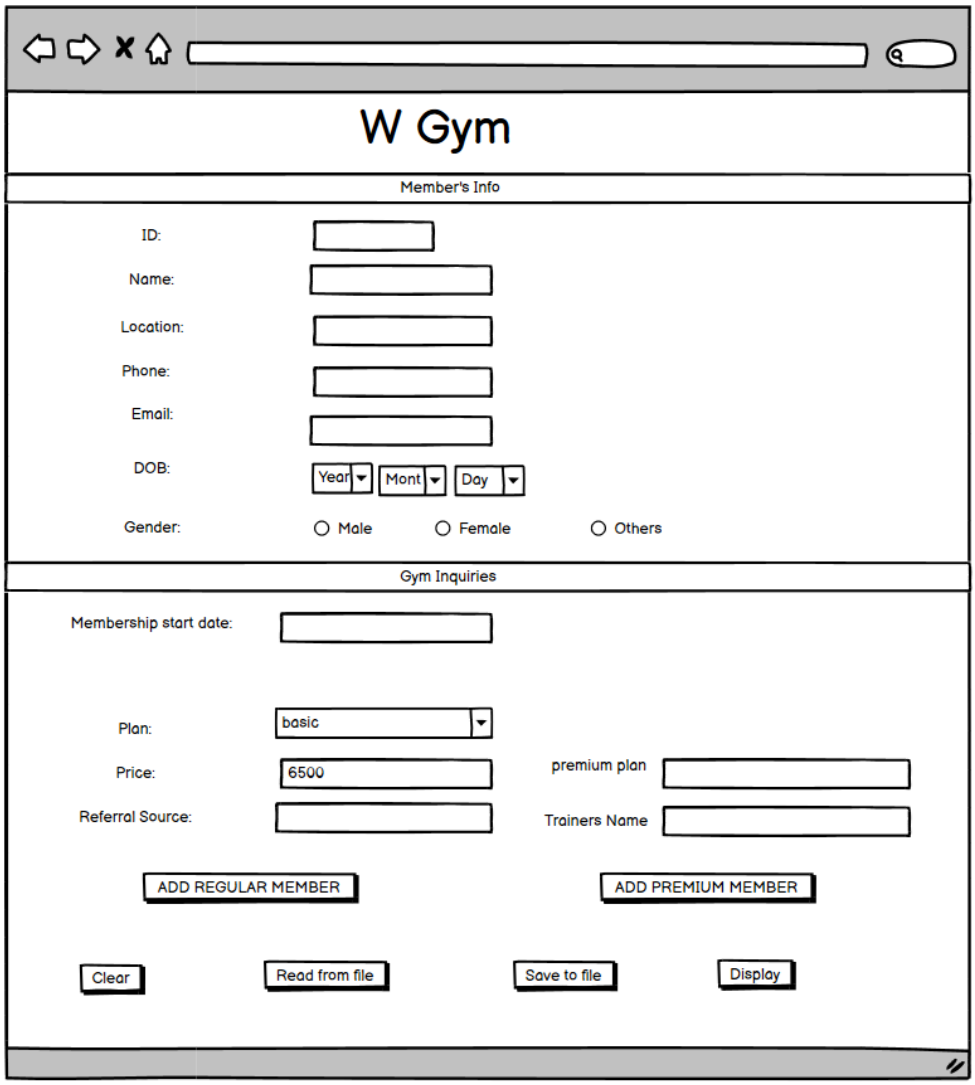


Figure 1:Wireframe of GUI

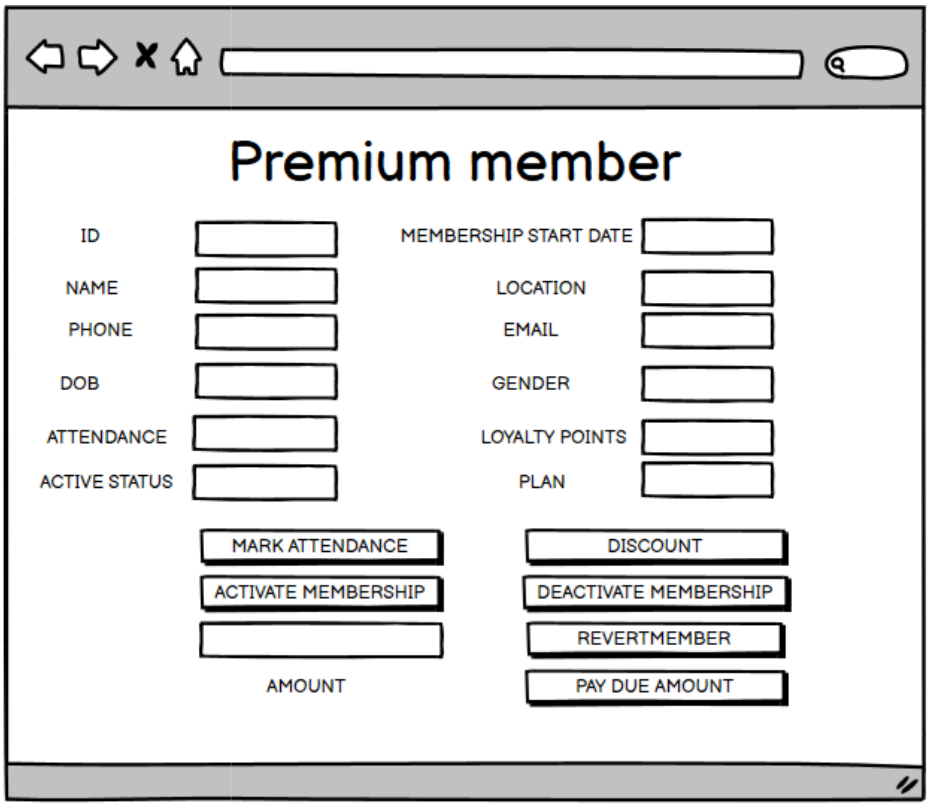


Figure 2:Wireframe of Premium Member

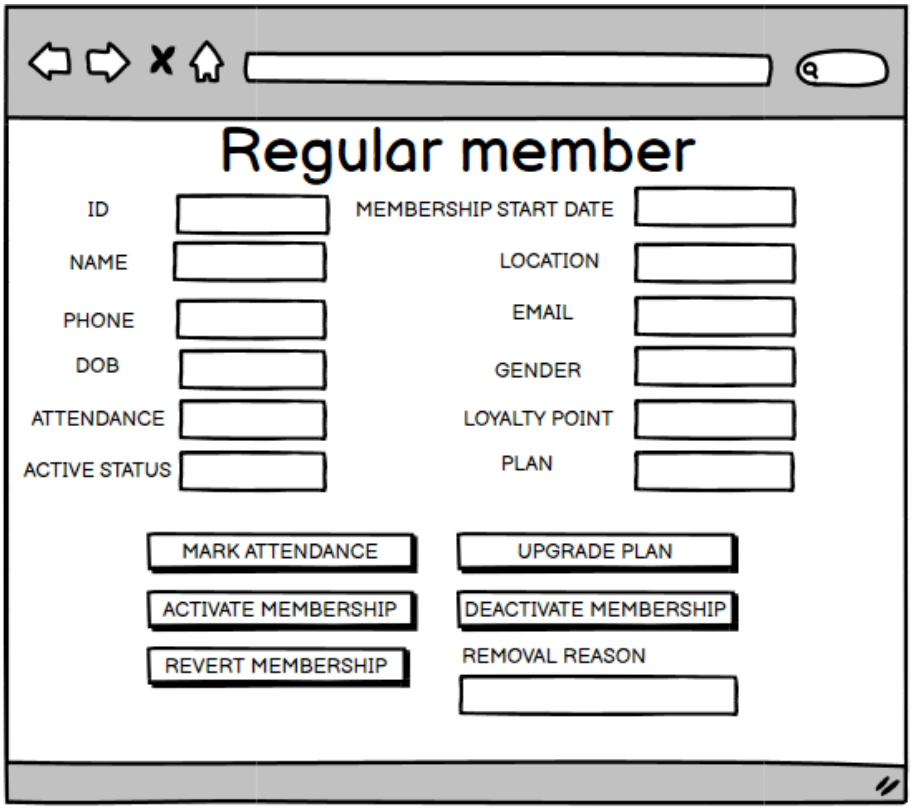


Figure 3:Wireframe of RegularMember

2.1. GUI

GUI is an abbreviated for graphical user interface, the interface by which user can interact with electronic devices such as computers , laptops, mobile phones and tablets using graphical symbols. It's a handy aspect of soft ware application programming for human computer interaction in line of text command and creating easy to use action. It exists to provide the user with simple s=decision points that are easy to find, understand and utilize. (Indeed Editorial Team, 2025)

GUI is implemented in this coursework to display the user-friendly page. The GUI parts help input the details of the user so it could be stored and the user can be the member of the Gym. GUI shows the wanted outcome and be more smother and cleaner to be visibly simple and friendly.

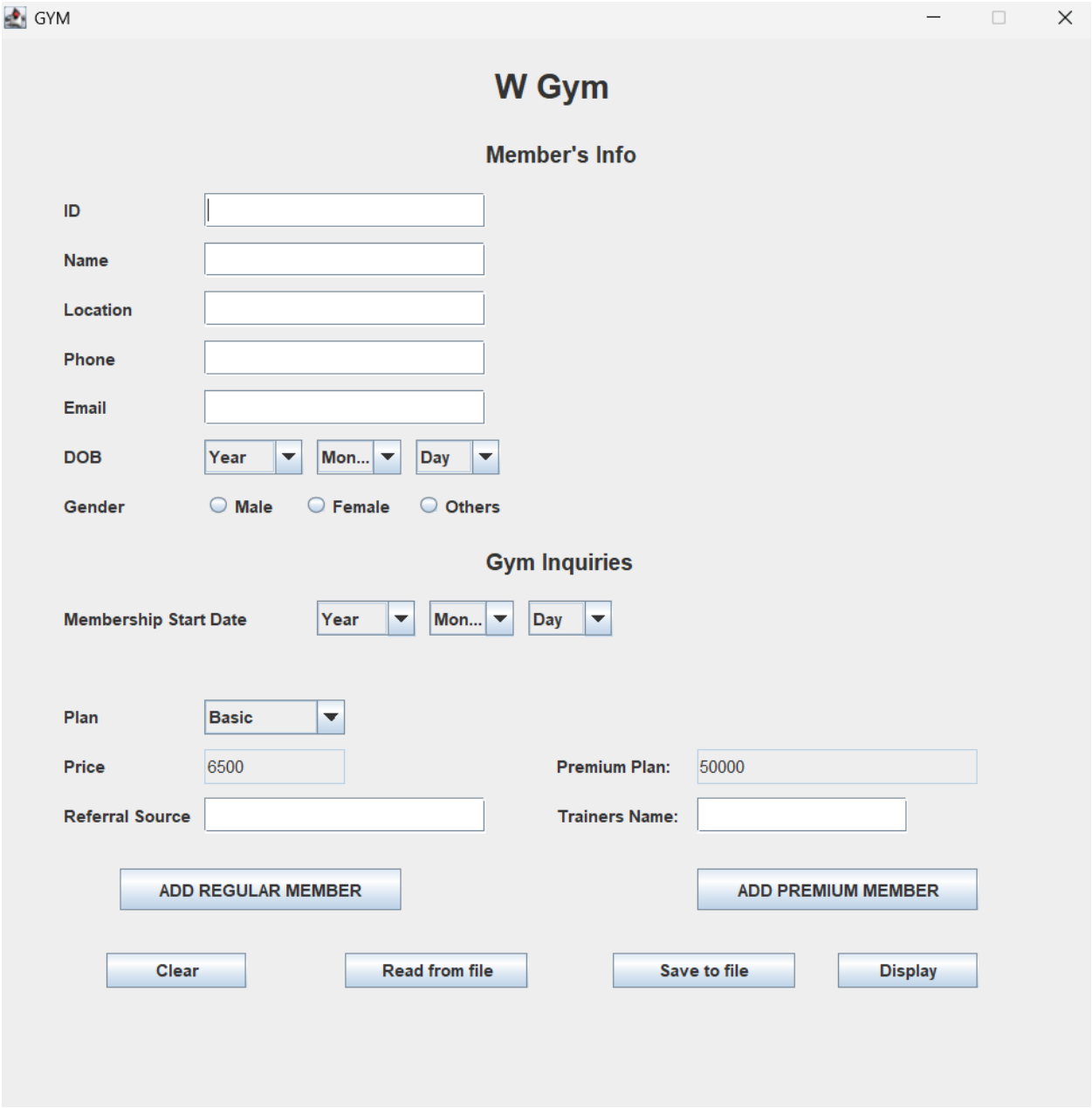


Figure 4:GUI of Main

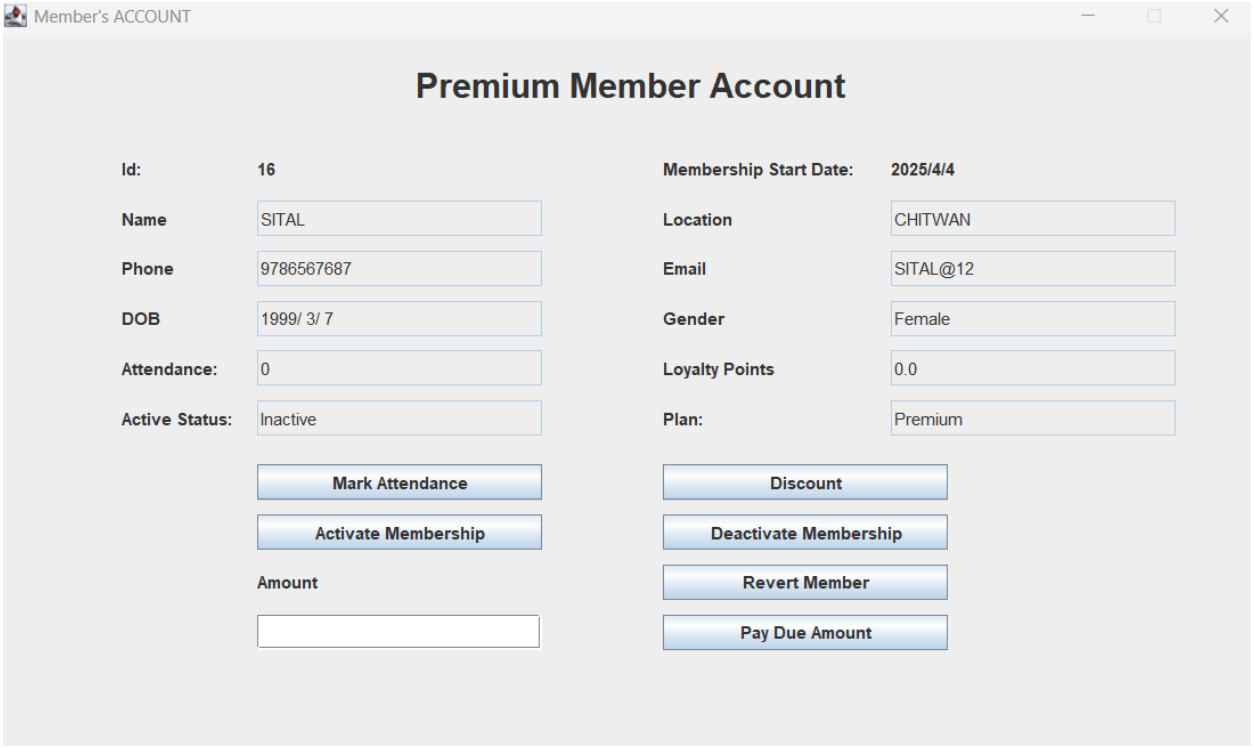


Figure 5:GUI of premium Member

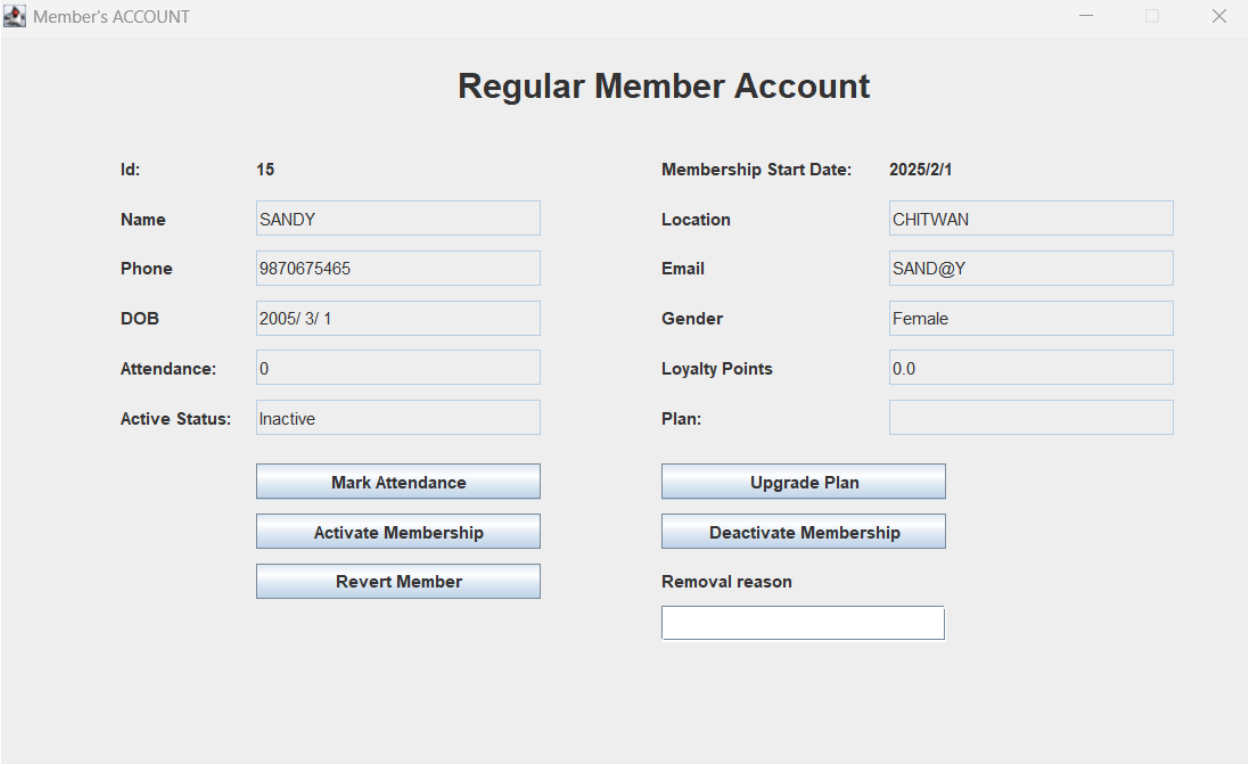


Figure 6:GUI of Regular member

3. Class Diagram

Table 1:Class Diagram of Gym Member

<< abstract >> GymMember
- id: int - name: String - location: String - phone: String - email: String - gender: String - DOB: String - membershipStartDate: String - attendance: int - loyaltyPoints: double
+<<constructor>> GymMember (id: int, name: String, location: String, phone: String, email: String, gender: String, DOB: String, membershipStartDate: String) + getId(): int + getName(): String + getLocation: String + getPhone(): String + getEmail(): String + getGender(): String + getDOB(): String + getMembershipStartDate(): String + getAttendance(): int + getLoyaltyPoints(): double + getActiveStatus(): Boolean + activateMembership(): void + deactiveMembership(): void + resetValue(): void + markAttendance(): void {abstract}

Table 2:Class Diagram of Premium Member

PremiumMember (extends GymMember)
- premiumCharge: double{ final } - personalTrainer: String - isFullPayment: Boolean - paidAmount: double - discountAmount: double
+ <<constructor>> PremiumMember(id: int, name: String, location: String, phone: String, email: String, gender: String, DOB: String, membershipStartDate: String, personalTrainer: String) + getPremiumCharge(): double + getPersonalTrainer(): String + getIsFullPayment(): Boolean + getPaidAmount(): double + getDiscountAmount(): double + markAttendance(): void + payDueAmount(amount: double): String + calculateDiscount(): String + revertPremiumMember(): void + display(): void

Table 3:Class Diagram of Regular Member

RegularMember (extends GymMember)
- attendanceLimit: int {final} - isEligibleForUpgrade: Boolean - removalReason: String - referralSource: String - plan: String - price: double
+ RegularMember(id: int, name: String, location: String, phone: String, email: String, gender: String, DOB: String, membershipStartDate: String, referralSource: String) + getReferralSource(): String + getIsEligibleForUpgrade(): boolean + getRemovalReason(): String + getAttendanceLimit(): int + getPlan(): String + getPrice(): double + getPlanPrice(plan: String): double + markAttendance(): void + upgradePlan(nPlan: String): String + revertRegularMember(removalReason: String): void + display(): void

Table 4:Class diagram of Gym GUI

GymGUI
- gymMember: JLabel - memberInfo: JLabel - gymInfo: JLabel - id, name, location, phone, trainer, startC: JLabel - email, dob, gender, plan, price, paidAmt: JLabel - premiumPlan, discount, start, referral: JLabel - idText, nameText, locationText, premiumPlanText: JTextField - phoneText, emailText, trainerText: JTextField - priceText, paidText, discountText, startText: JTextField - referralText: JTextField - yearDob, monthDob, dayDob: JComboBox<String> - planYear, planMonth, planDay: JComboBox<String> - planC: JComboBox<String> - maleButton, femaleButton, otherButton: JRadioButton - button: ButtonGroup - clear, readFile, saveFile, display: JButton - addRegular, addPremium: JButton -rframe: JFrame - rpanel: JPanel - rId: JLabel - rIdText: JLabel - rStart, rStartText: JLabel - rName, rNameText: JTextField

- rLocation, rLocationText: JTextField
- rPhone, rPhoneText: JTextField
- rEmail, rEmailText: JTextField
- rDob, rDobText: JTextField
- rGender, rGenderText: JTextField
- attendance, attendText: JTextField
- loyal, loyalText: JTextField
- active, activeText: JTextField
- rPlan, rPlanText: JTextField
- markAttendance: JButton
- upgradePlan: JButton
- activateMembership: JButton
- deactivateMembership: JButton
- revertRegularMember: JButton
- removalReason: JLabel
- removalReasonText: JTextField

- pframe: JFrame
- ppanel: JPanel
- pId, pIdText: JLabel
- pStart, pStartText: JLabel
- pName, pNameText: JTextField
- pLocation, pLocationText: JTextField
- pPhone, pPhoneText: JTextField
- pEmail, pEmailText: JTextField
- pDob, pDobText: JTextField
- pGender, pGenderText: JTextField
- pAttendance, pAttendText: JTextField
- pLoyal, pLoyalText: JTextField
- pActive, pActiveText: JTextField
- pPlan, pPlanText: JTextField

- pMarkAttendance: JButton - discountBtn: JButton - pActivateMembership: JButton - pDeactivateMembership: JButton - revertPremiumMember: JButton - due: JButton - pAmt, pAmtText: JTextField - reading: FileReader - writing: FileWriter - file, Doc: File - textArea: JTextArea - gym: ArrayList<GymMember>
+ GymGUI() + actionPerformed(ActionEvent e): void + clearEverything(): void + saveToFile(): void + detailsFromFile(): void + RegularMemberAc(RegularMember): void + PremiumMemberAc(PremiumMember): void + searchMember(id: int): GymMember

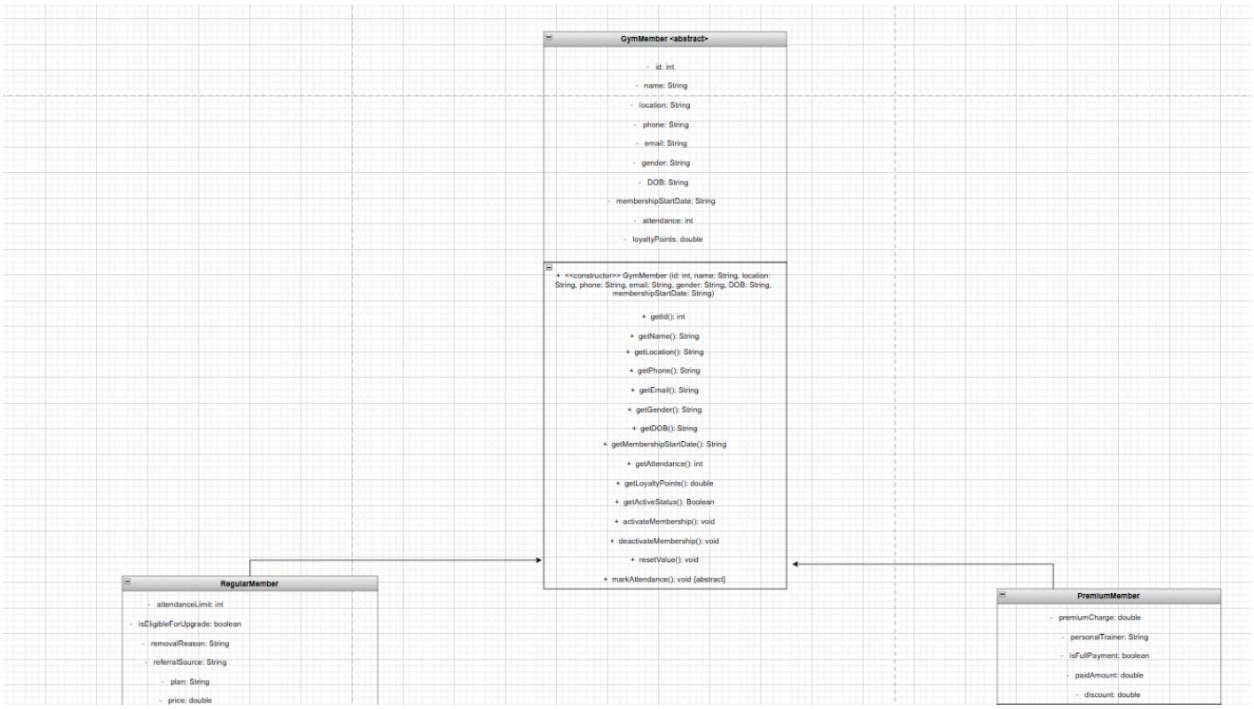


Figure 7:Class diadram(1)

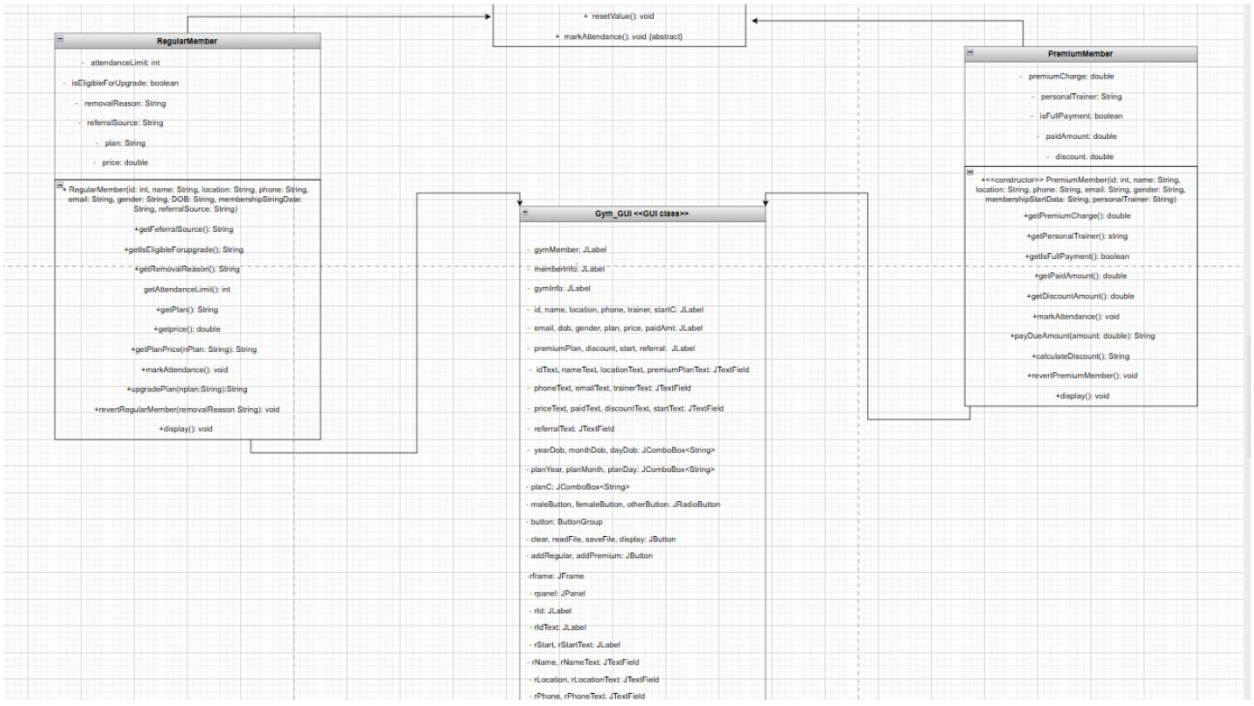


Figure 8:Class Diagram(2)

4. Pseudocode

4.1. Pseudocode of Gym Member

CLASS GymMember

// Instance Variables

PROTECTED INTEGER id

PROTECTED STRING name

PROTECTED STRING location

PROTECTED STRING phone

PROTECTED STRING email

PROTECTED STRING gender

PROTECTED STRING DOB

PROTECTED STRING membershipStartDate

PROTECTED INTEGER attendance

PROTECTED DOUBLE loyaltyPoints

PROTECTED BOOLEAN activeStatus

// Constructor

METHOD GymMember(INTEGER id, STRING name, STRING location, STRING phone, STRING email, STRING gender, STRING DOB, STRING membershipStartDate)

SET this.id = id

SET this.name = name

SET this.location = location

SET this.phone = phone

```
    SET this.email = email
    SET this.gender = gender
    SET this.DOB = DOB
    SET this.membershipStartDate = membershipStartDate
    SET this.attendance = 0
    SET this.loyaltyPoints = 0.0
    SET this.activeStatus = FALSE
END METHOD



// Abstract Method
METHOD markAttendance() ABSTRACT

// Activate Membership
METHOD activateMembership()
    IF NOT activeStatus THEN
        SET this.activeStatus = TRUE
        PRINT "The membership is activated"
    ELSE
        PRINT "The membership is already renewed"
    END IF
END METHOD
```

```
// Deactivate Membership
METHOD deactivateMembership()
    IF activeStatus THEN
        SET this.activeStatus = FALSE
        PRINT "The membership is deactivated"
    ELSE
        PRINT "The membership has already been deactivated"
    END IF
END METHOD

// Reset Values
METHOD resetValue()
    SET loyaltyPoints = 0.0
    SET attendance = 0
    SET activeStatus = FALSE
END METHOD

// Display Member Information
METHOD display()
    PRINT "The id is: " + getId()
    PRINT "The name is: " + getName()
    PRINT "The location is: " + getLocation()
```

PRINT "The phone number is: " + getPhone()

PRINT "The email is: " + getEmail()

PRINT "The gender is: " + getGender()

PRINT "The date of birth is: " + getDOB()

PRINT "The membership start date is: " + getMembershipStartDate()

PRINT "The attendance is: " + getAttendance()

PRINT "The loyalty points are: " + getLoyaltyPoints()

PRINT "The active status is: " + getActiveStatus()

END METHOD

END CLASS

4.2. Pseudocode of premium member

CLASS PremiumMember **EXTENDS** GymMember

// Instance Variables

PRIVATE FINAL DOUBLE premiumCharge

PRIVATE STRING personalTrainer

PRIVATE BOOLEAN isFullPayment

PRIVATE DOUBLE paidAmount

PRIVATE DOUBLE discountAmount

// Constructor

METHOD PremiumMember(INTEGER id, STRING name, STRING location, STRING phone, STRING email, STRING gender, STRING DOB, STRING membershipStartDate, STRING personalTrainer)

```
        CALL super(id, name, location, phone, email, gender, DOB,
membershipStartDate)
        SET this.personalTrainer = personalTrainer
        SET premiumCharge = 50000
        SET paidAmount = 0
        SET isFullPayment = FALSE
        SET discountAmount = 0
    END METHOD


    // Attendance & Loyalty Points Calculation
    OVERRIDE METHOD markAttendance()
        INCREMENT attendance BY 1
        INCREMENT loyaltyPoints BY 10
        PRINT "The attendance is: " + attendance
        PRINT "The loyalty points are: " + loyaltyPoints
    END METHOD


    // Payment of the Due Amount
    METHOD payDueAmount(DOUBLE amount) RETURNS STRING
        IF isFullPayment THEN
            RETURN "Payment already done!!"
        END IF
```

```
        // New paid amount after inserting the remaining amount
        SET paidAmount = amount + paidAmount

        IF paidAmount > premiumCharge THEN
            SET paidAmount = paidAmount - amount
            RETURN "EXCEEDS THE LIMIT!!!.\nRequired: " + premiumCharge + "\nAlready Paid Balance: " + paidAmount + "\nPaid Balance: " + amount + "\nRemaining: " + (premiumCharge - paidAmount)
        END IF

        // Update isFullPayment if both are the same
        SET isFullPayment = (paidAmount == premiumCharge)
        IF isFullPayment THEN
            RETURN "Payment Successful: " + paidAmount + "\nNo Remaining Balance."
        ELSE
            DOUBLE remaining = premiumCharge - paidAmount // Remaining due balance to pay
            RETURN "Payment Successful!\nRemaining balance: " + remaining
        END IF
    END METHOD

    // Calculate Discount if Full Payment is Made
```

```
METHOD calculateDiscount() RETURNS STRING
    IF isFullPayment THEN
        SET discountAmount = 0.10 * premiumCharge
        RETURN "The given discount is: " + discountAmount
    ELSE
        RETURN "Discount cannot be applied.\nPayment is not complete"
    END IF
END METHOD

// Reset Values to Default
METHOD revertPremiumMember()
    CALL super.resetValue()

    SET personalTrainer = ""
    SET isFullPayment = FALSE
    SET paidAmount = 0
    SET discountAmount = 0
    PRINT "This account has been reverted to default premium membership"
END METHOD

// Display Parent and Child Class Output
OVERRIDE METHOD display()
    CALL super.display()
```

PRINT "The personal trainer is: " + getPersonalTrainer()

PRINT "The paid amount is: " + getPaidAmount()

PRINT "Is full payment done? " + getIsFullPayment()

DOUBLE remaining = premiumCharge - paidAmount

PRINT "The remaining amount is: " + remaining

END METHOD

END CLASS

4.3. Pseudocode of Regular Member

CLASS RegularMember EXTENDS GymMember

// Instance Variables

PRIVATE FINAL INTEGER attendanceLimit // constant

PRIVATE BOOLEAN isEligibleForUpgrade

PRIVATE STRING removalReason

PRIVATE STRING referralSource

PRIVATE STRING plan

PRIVATE DOUBLE price

// Constructor

METHOD RegularMember(INTEGER id, STRING name, STRING location, STRING phone, STRING email, STRING gender, STRING DOB, STRING membershipStartDate, STRING referralSource)

```
        CALL super(id, name, location, phone, email, gender, DOB,
membershipStartDate)
        SET this.referralSource = referralSource
        SET this.isEligibleForUpgrade = FALSE
        SET this.attendanceLimit = 30
        SET this.plan = "Basic"
        SET this.price = 6500
        SET this.removalReason = ""
    END METHOD


    // Attendance & Loyalty Points
    OVERRIDE METHOD markAttendance()
        INCREMENT attendance BY 1
        INCREMENT loyaltyPoints BY 5
        PRINT "The attendance is: " + attendance
        PRINT "The loyalty points are: " + loyaltyPoints
    END METHOD


    // Get Plan Price
    METHOD getPlanPrice(STRING plan) RETURNS DOUBLE
        SWITCH plan.toLowerCase()
            CASE "basic":
```

```
            RETURN 6500
        CASE "standard":
            RETURN 8500
        CASE "deluxe":
            RETURN 10500
        DEFAULT:
            PRINT "Invalid plan: " + plan
            RETURN -1
    END SWITCH
END METHOD

// Upgrade Plan
METHOD upgradePlan(STRING nPlan) RETURNS STRING
    IF attendance < this.attendanceLimit THEN
        RETURN "Cannot upgrade the current plan because of low attendance"
    END IF

    IF nPlan.equalsIgnoreCase(plan) THEN
        RETURN "Cannot upgrade because you are subscribed to the same
membership"
    END IF

    DOUBLE nPrice = CALL getPlanPrice(nPlan)
```

```
    IF nPrice == -1 THEN
        RETURN "The entered plan not found"
    END IF

    IF nPrice <= this.price THEN
        RETURN "Cannot upgrade to the same or lower plans."
    END IF

    // For the upgrade
    SET this.plan = nPlan
    SET this.price = nPrice
    RETURN "Successfully updated the membership!!"
END METHOD

// Remove Membership
METHOD revertRegularMember(STRING removalReason)
    // Calling parent class method
    CALL super.resetValue()

    SET this.isEligibleForUpgrade = FALSE
    SET this.plan = "Basic"
    SET this.price = 6500
```

```
        SET this.removalReason = removalReason

        // Printing the reason

        PRINT "This account has been reverted because: " + removalReason

    END METHOD


    // Display All Data

    OVERRIDE METHOD display()

        CALL super.display() // Call parent class method using super keyword

        PRINT "The referral source is: " + getReferralSource()

        PRINT "The plan's type is: " + getPlan()

        PRINT "The price is: " + getPrice()


        IF NOT removalReason.isEmpty() THEN

            PRINT "The removal reason is: " + getRemovalReason()

        END IF

    END METHOD

END CLASS
```

4.4. Pseudocode for gym GUI

```
CLASS GymGUI IMPLEMENTS ActionListener


BEGIN

    // Declare labels for Gym Member UI
```

DECLARE gymMember, memberInfo, gymInfo, id, name, location, phone, trainer, startC AS JLabel

DECLARE email, dob, gender, plan, price, paidAmt, premiumPlan, discount, start, referral AS JLabel

// Declare text fields for input

DECLARE idText, nameText, locationText, premiumPlanText, phoneText, emailText, trainerText AS JTextField

DECLARE priceText, paidText, discountText, startText, referralText AS JTextField

// Declare combo boxes for date selections

DECLARE yearDob, monthDob, dayDob, planYear, planMonth, planDay AS JComboBox<String>

DECLARE planC AS JComboBox<String>

// Declare radio buttons and button group

DECLARE otherButton, maleButton, femaleButton AS JRadioButton

DECLARE button AS ButtonGroup

// Declare main operation buttons

DECLARE clear, readFile, saveFile, display, addRegular, addPremium AS JButton

// Declare frames and panels

DECLARE frame, rframe, pframe, readFrame AS JFrame

DECLARE panel, rpanel, ppanel AS JPanel

// Declare Premium Member display labels and text fields

DECLARE ptitle, pId, pIdText, pName, pAmt, pLocation, pPhone, pEmail, pStart, pDob, pGender, pStartText, pActive, pPlan, pAttendance, pLoyal AS JLabel

DECLARE pNameText, pLocationText, pPhoneText, pEmailText, pDobText, pGenderText, pAttendText, pActiveText, pPlanText, pLoyalText AS JTextField

// Declare Regular Member display labels and text fields

DECLARE title, rId, rIdText, rName, removal, rLocation, rPhone, rEmail, rStart, rDob, rGender, rStartText, active, rPlan, attendance, removalReason, loyal AS JLabel

DECLARE rNameText, pAmtText, rLocationText, removalText, rPhoneText, rEmailText, rDobText, rGenderText, attendText, activeText, rPlanText, removalReasonText, loyalText AS JTextField

// Declare action buttons for regular and premium members

DECLARE markAttendance, activateMembership, deactivateMembership, revertMember, revertRegularMember, revertPremiumMember AS JButton

DECLARE pMarkAttendance, pActivateMembership, pDeactivateMembership AS JButton

DECLARE upgradePlan, due, discountBtn AS JButton

// Declare file and reader/writer related objects

DECLARE reading AS FileReader

DECLARE file AS File

```
    DECLARE writing AS FileWriter
    DECLARE textArea AS JTextArea
    DECLARE Doc AS File

    // Declare array list to store GymMember objects
    DECLARE gym AS ArrayList<GymMember>
CONSTRUCTOR GymGUI
    CREATE JFrame frame WITH TITLE "GYM"
    CREATE JPanel panel
    SET panel layout TO null

    CREATE JLabel gymMember WITH TEXT " W Gym ", FONT Arial BOLD 24
    SET BOUNDS (350, 20, 300, 30)
    ADD gymMember TO panel

    CREATE JLabel memberInfo WITH TEXT "Member's Info", FONT Arial BOLD 16
    SET BOUNDS (350, 70, 200, 25)
    ADD memberInfo TO panel

    CREATE JLabel id WITH TEXT "ID"
    SET BOUNDS (50, 110, 100, 25)
    ADD id TO panel
```

```
CREATE JTextField idText
SET BOUNDS (150, 110, 200, 25)
ADD idText TO panel

CREATE JLabel name WITH TEXT "Name"
SET BOUNDS (50, 145, 100, 25)
ADD name TO panel

CREATE JTextField nameText
SET BOUNDS (150, 145, 200, 25)
ADD nameText TO panel

CREATE JLabel location WITH TEXT "Location"
SET BOUNDS (50, 180, 100, 25)
ADD location TO panel

CREATE JTextField locationText
SET BOUNDS (150, 180, 200, 25)
ADD locationText TO panel

CREATE JLabel phone WITH TEXT "Phone"
SET BOUNDS (50, 215, 100, 25)
ADD phone TO panel
```

```
CREATE JTextField phoneText
SET BOUNDS (150, 215, 200, 25)
ADD phoneText TO panel

CREATE JLabel email WITH TEXT "Email"
SET BOUNDS (50, 250, 100, 25)
ADD email TO panel

CREATE JTextField emailText
SET BOUNDS (150, 250, 200, 25)
ADD emailText TO panel

CREATE JLabel dob WITH TEXT "DOB"
SET BOUNDS (50, 285, 100, 25)
ADD dob TO panel

CREATE year ARRAY OF STRING SIZE 100
FOR i FROM 0 TO 99 DO
   SET year[i] TO STRING OF (1980 + i)
END FOR
CREATE JComboBox yearDob WITH year
INSERT "Year" AT INDEX 0
```

```
SET SELECTED INDEX TO 0
SET BOUNDS (150, 285, 70, 25)
ADD yearDob TO panel

CREATE month ARRAY WITH {"Month", "1", ..., "12"}
CREATE JComboBox monthDob WITH month
SET SELECTED INDEX TO 0
SET BOUNDS (230, 285, 60, 25)
ADD monthDob TO panel

CREATE day ARRAY OF STRING SIZE 31
FOR i FROM 0 TO 30 DO
    SET day[i] TO STRING OF (i + 1)
END FOR
CREATE JComboBox dayDob WITH day
INSERT "Day" AT INDEX 0
SET SELECTED INDEX TO 0
SET BOUNDS (300, 285, 60, 25)
ADD dayDob TO panel

CREATE JLabel gender WITH TEXT "Gender"
SET BOUNDS (50, 320, 100, 25)
ADD gender TO panel
```

```
CREATE ButtonGroup button

CREATE JRadioButton maleButton WITH TEXT "Male"

SET BOUNDS (150, 320, 60, 25)

ADD maleButton TO button AND panel


CREATE JRadioButton femaleButton WITH TEXT "Female"

SET BOUNDS (220, 320, 70, 25)

ADD femaleButton TO button AND panel


CREATE JRadioButton otherButton WITH TEXT "Others"

SET BOUNDS (300, 320, 80, 25)

ADD otherButton TO button AND panel


CREATE JLabel gymInfo WITH TEXT "Gym Inquiries", FONT Arial BOLD 16

SET BOUNDS (350, 360, 200, 25)

ADD gymInfo TO panel


CREATE JLabel start WITH TEXT "Membership Start Date"

SET BOUNDS (50, 400, 200, 25)

ADD start TO panel


CREATE planYearArray ARRAY OF STRING SIZE 50
```

```
FOR i FROM 0 TO 49 DO
    SET planYearArray[i] TO STRING OF (2015 + i)
END FOR
CREATE JComboBox planYear WITH planYearArray
INSERT "Year" AT INDEX 0
SET SELECTED INDEX TO 0
SET BOUNDS (230, 400, 70, 25)
ADD planYear TO panel

CREATE planMonthArray ARRAY WITH {"Month", "1", ..., "12"}
CREATE JComboBox planMonth WITH planMonthArray
SET SELECTED INDEX TO 0
SET BOUNDS (310, 400, 60, 25)
ADD planMonth TO panel

CREATE planDayArray ARRAY OF STRING SIZE 31
FOR i FROM 0 TO 30 DO
    SET planDayArray[i] TO STRING OF (i + 1)
END FOR
CREATE JComboBox planDay WITH planDayArray
INSERT "Day" AT INDEX 0
SET SELECTED INDEX TO 0
SET BOUNDS (380, 400, 60, 25)
```

```
ADD planDay TO panel

CREATE JLabel plan WITH TEXT "Plan"
SET BOUNDS (50, 470, 100, 25)
ADD plan TO panel

CREATE planCombo ARRAY WITH {"Basic", "Standard", "Deluxe"}
CREATE JComboBox planC WITH planCombo
SET BOUNDS (150, 470, 100, 25)
ADD ActionListener TO planC
ADD planC TO panel

CREATE JLabel price WITH TEXT "Price"
SET BOUNDS (50, 505, 100, 25)
ADD price TO panel

CREATE JTextField priceText
SET BOUNDS (150, 505, 100, 25)
SET priceText NOT EDITABLE
ADD priceText TO panel

CREATE JLabel premiumPlan WITH TEXT "Premium Plan:"
SET BOUNDS (400, 505, 100, 25)
```

ADD premiumPlan TO panel

CREATE JTextField premiumPlanText WITH TEXT "50000"

SET BOUNDS (500, 505, 200, 25)

SET premiumPlanText NOT EDITABLE

ADD premiumPlanText TO panel

CREATE JLabel referral WITH TEXT "Referral Source"

SET BOUNDS (50, 540, 100, 25)

ADD referral TO panel

CREATE JTextField referralText

SET BOUNDS (150, 540, 200, 25)

ADD referralText TO panel

CREATE JLabel trainer WITH TEXT "Trainers Name:"

SET BOUNDS (400, 540, 200, 25)

ADD trainer TO panel

CREATE JTextField trainerText

SET BOUNDS (500, 540, 150, 25)

ADD trainerText TO panel

```
CREATE JButton addRegular WITH TEXT "ADD REGULAR MEMBER"

SET BOUNDS (90, 590, 200, 30)

ADD ActionListener TO addRegular

ADD addRegular TO panel


CREATE JButton addPremium WITH TEXT "ADD PREMIUM MEMBER"

SET BOUNDS (500, 590, 200, 30)

ADD ActionListener TO addPremium

ADD addPremium TO panel


CREATE JButton clear WITH TEXT "Clear"

SET BOUNDS (80, 650, 100, 25)

ADD ActionListener TO clear

ADD clear TO panel


CREATE JButton readFile WITH TEXT "Read from file"

SET BOUNDS (250, 650, 130, 25)

ADD ActionListener TO readFile

ADD readFile TO panel


CREATE JButton saveFile WITH TEXT "Save to file"

SET BOUNDS (440, 650, 130, 25)

ADD ActionListener TO saveFile
```

```
    ADD saveFile TO panel

    CREATE JButton display WITH TEXT "Display"
    SET BOUNDS (600, 650, 100, 25)
    ADD ActionListener TO display
    ADD display TO panel

    ADD panel TO frame
    SET frame SIZE TO (800, 800)
    SET DEFAULT CLOSE OPERATION TO EXIT
    SET RESIZABLE TO false
    SET LOCATION TO CENTER
    SET VISIBLE TO true
END CONSTRUCTOR
METHOD RegularMemberAc

INPUT: gym (an object of RegularMember)

BEGIN
    CREATE JFrame rframe titled "Member's ACCOUNT"
    CREATE JPanel rpanel with null layout

    // Title
```

CREATE JLabel title with text "Regular Member Account"

SET font of title to Arial, Bold, 24

SET bounds of title to (320, 20, 300, 30)

ADD title to rpanel

// Member ID

CREATE JLabel rId with text "Id:"

SET bounds of rId to (90, 80, 180, 25)

ADD rId to rpanel

CREATE JLabel rIdText with text from gym.getId()

SET bounds of rIdText to (185, 80, 200, 25)

ADD rIdText to rpanel

// Membership Start Date

CREATE JLabel rStart with text "Membership Start Date:"

SET bounds to (470, 80, 180, 25)

ADD rStart to rpanel

CREATE JLabel rStartText with text from gym.getMembershipStartDate()

SET bounds to (630, 80, 180, 25)

ADD rStartText to rpanel

// Name

CREATE JLabel rName with text "Name"

SET bounds to (90, 115, 100, 25)

ADD rName to rpanel

CREATE JTextField rNameText with text from gym.getName()

SET bounds to (185, 115, 200, 25)

SET editable to FALSE

ADD rNameText to rpanel

// Location

CREATE JLabel rLocation with text "Location"

SET bounds to (470, 115, 100, 25)

ADD rLocation to rpanel

CREATE JTextField rLocationText with text from gym.getLocation()

SET bounds to (630, 115, 200, 25)

SET editable to FALSE

ADD rLocationText to rpanel

// Phone

CREATE JLabel rPhone with text "Phone"

SET bounds to (90, 150, 100, 25)

ADD rPhone to rpanel

CREATE JTextField rPhoneText with text from gym.getPhone()

SET bounds to (185, 150, 200, 25)

SET editable to FALSE

ADD rPhoneText to rpanel

// Email

CREATE JLabel rEmail with text "Email"

SET bounds to (470, 150, 100, 25)

ADD rEmail to rpanel

CREATE JTextField rEmailText with text from gym.getEmail()

SET bounds to (630, 150, 200, 25)

SET editable to FALSE

ADD rEmailText to rpanel

// Date of Birth

CREATE JLabel rDob with text "DOB"

SET bounds to (90, 185, 100, 25)

ADD rDob to rpanel

CREATE JTextField rDobText with text from gym.getDOB()

```
SET bounds to (185, 185, 200, 25)

SET editable to FALSE

ADD rDobText to rpanel


// Gender

CREATE JLabel rGender with text "Gender"

SET bounds to (470, 185, 100, 25)

ADD rGender to rpanel


CREATE JTextField rGenderText with text from gym.getGender()

SET bounds to (630, 185, 200, 25)

SET editable to FALSE

ADD rGenderText to rpanel


// Attendance

CREATE JLabel attendance with text "Attendance:"

SET bounds to (90, 220, 100, 25)

ADD attendance to rpanel


CREATE JTextField attendText with text "0"

SET bounds to (185, 220, 200, 25)

SET editable to FALSE

ADD attendText to rpanel
```

// Loyalty Points

CREATE JLabel loyal with text "Loyalty Points"

SET bounds to (470, 220, 100, 25)

ADD loyal to rpanel

CREATE JTextField loyalText with text "0.0"

SET bounds to (630, 220, 200, 25)

SET editable to FALSE

ADD loyalText to rpanel

// Active Status

CREATE JLabel active with text "Active Status:"

SET bounds to (90, 255, 100, 25)

ADD active to rpanel

CREATE JTextField activeText with text "Inactive"

SET bounds to (185, 255, 200, 25)

SET editable to FALSE

ADD activeText to rpanel

// Plan

CREATE JLabel rPlan with text "Plan:"

SET bounds to (470, 255, 100, 25)

ADD rPlan to rpanel

CREATE JTextField rPlanText with text "Regular"

SET bounds to (630, 255, 200, 25)

SET editable to FALSE

ADD rPlanText to rpanel

// Buttons

CREATE JButton markAttendance with text "Mark Attendance"

SET bounds to (185, 300, 200, 25)

ADD ActionListener

ADD to rpanel

CREATE JButton upgradePlan with text "Upgrade Plan"

SET bounds to (470, 300, 200, 25)

ADD ActionListener

ADD to rpanel

CREATE JButton activateMembership with text "Activate Membership"

SET bounds to (185, 335, 200, 25)

ADD ActionListener

ADD to rpanel

```
CREATE JButton deactivateMembership with text "Deactivate Membership"

SET bounds to (470, 335, 200, 25)

ADD ActionListener

ADD to rpanel


CREATE JButton revertRegularMember with text "Revert Member"

SET bounds to (185, 370, 200, 25)

ADD ActionListener

ADD to rpanel


// Removal Reason

CREATE JLabel removalReason with text "Removal reason"

SET bounds to (470, 370, 200, 25)

ADD to rpanel


CREATE JTextField removalReasonText

SET bounds to (470, 400, 200, 25)

ADD to rpanel


// Final Frame Settings

ADD rpanel to rframe

SET rframe size to (900, 550)
```

SET default close operation to EXIT_ON_CLOSE

SET resizable to FALSE

SET location relative to NULL

SET visible to TRUE

END METHOD

METHOD PremiumMemberAc

INPUT: gym (an object of PremiumMember)

BEGIN

CREATE JFrame pframe titled "Member's ACCOUNT"

CREATE JPanel ppanel with null layout

// Title

CREATE JLabel ptitle with text "Premium Member Account"

SET font to Arial, Bold, 24

SET bounds to (290, 20, 400, 30)

ADD to ppanel

// ID

CREATE JLabel pId with text "Id:"

SET bounds to (90, 80, 180, 25)

ADD to ppanel

CREATE JLabel pIdText with gym.getId()

SET bounds to (185, 80, 200, 25)

ADD to ppanel

// Membership Start Date

CREATE JLabel pStart with text "Membership Start Date:"

SET bounds to (470, 80, 180, 25)

ADD to ppanel

CREATE JLabel pStartText with gym.getMembershipStartDate()

SET bounds to (630, 80, 180, 25)

ADD to ppanel

// Name

CREATE JLabel pName with text "Name"

SET bounds to (90, 115, 100, 25)

ADD to ppanel

CREATE JTextField pNameText with gym.getName()

SET bounds to (185, 115, 200, 25)

SET editable to FALSE

ADD to ppanel

// Location

CREATE JLabel pLocation with text "Location"

SET bounds to (470, 115, 100, 25)

ADD to ppanel

CREATE JTextField pLocationText with gym.getLocation()

SET bounds to (630, 115, 200, 25)

SET editable to FALSE

ADD to ppanel

// Phone

CREATE JLabel pPhone with text "Phone"

SET bounds to (90, 150, 100, 25)

ADD to ppanel

CREATE JTextField pPhoneText with gym.getPhone()

SET bounds to (185, 150, 200, 25)

SET editable to FALSE

ADD to ppanel

// Email

CREATE JLabel pEmail with text "Email"

SET bounds to (470, 150, 100, 25)

ADD to ppanel

CREATE JTextField pEmailText with gym.getEmail()

SET bounds to (630, 150, 200, 25)

SET editable to FALSE

ADD to ppanel

// Date of Birth

CREATE JLabel pDob with text "DOB"

SET bounds to (90, 185, 100, 25)

ADD to ppanel

CREATE JTextField pDobText with gym.getDOB()

SET bounds to (185, 185, 200, 25)

SET editable to FALSE

ADD to ppanel

// Gender

CREATE JLabel pGender with text "Gender"

SET bounds to (470, 185, 100, 25)

ADD to ppanel

```
CREATE JTextField pGenderText with gym.getGender()
SET bounds to (630, 185, 200, 25)
SET editable to FALSE
ADD to ppanel

// Attendance
CREATE JLabel pAttendance with text "Attendance:"
SET bounds to (90, 220, 100, 25)
ADD to ppanel

CREATE JTextField pAttendText with text "0"
SET bounds to (185, 220, 200, 25)
SET editable to FALSE
ADD to ppanel

// Loyalty Points
CREATE JLabel pLoyal with text "Loyalty Points"
SET bounds to (470, 220, 100, 25)
ADD to ppanel

CREATE JTextField pLoyalText with text "0.0"
SET bounds to (630, 220, 200, 25)
```

```
SET editable to FALSE

ADD to ppanel


// Active Status

CREATE JLabel pActive with text "Active Status:"

SET bounds to (90, 255, 100, 25)

ADD to ppanel


CREATE JTextField pActiveText with text "Inactive"

SET bounds to (185, 255, 200, 25)

SET editable to FALSE

ADD to ppanel


// Plan

CREATE JLabel pPlan with text "Plan:"

SET bounds to (470, 255, 100, 25)

ADD to ppanel


CREATE JTextField pPlanText with text "Premium"

SET bounds to (630, 255, 200, 25)

SET editable to FALSE

ADD to ppanel
```

// Buttons

CREATE JButton pMarkAttendance with text "Mark Attendance"

SET bounds to (185, 300, 200, 25)

ADD ActionListener

ADD to ppanel

CREATE JButton discountBtn with text "Discount"

SET bounds to (470, 300, 200, 25)

ADD ActionListener

ADD to ppanel

CREATE JButton pActivateMembership with text "Activate Membership"

SET bounds to (185, 335, 200, 25)

ADD ActionListener

ADD to ppanel

CREATE JButton pDeactivateMembership with text "Deactivate Membership"

SET bounds to (470, 335, 200, 25)

ADD ActionListener

ADD to ppanel

// Amount Field

CREATE JLabel pAmt with text "Amount"

SET bounds to (185, 370, 200, 25)

ADD to ppanel

CREATE JTextField pAmtText

SET bounds to (185, 405, 200, 25)

ADD to ppanel

// Revert and Pay Due Buttons

CREATE JButton revertPremiumMember with text "Revert Member"

SET bounds to (470, 370, 200, 25)

ADD ActionListener

ADD to ppanel

CREATE JButton due with text "Pay Due Amount"

SET bounds to (470, 405, 200, 25)

ADD ActionListener

ADD to ppanel

// Final Frame Setup

ADD ppanel to pframe

SET size of pframe to (900, 550)

SET default close operation to EXIT_ON_CLOSE

SET resizable to FALSE

```
    SET location relative to NULL
    SET visible to TRUE

END METHOD
METHOD clearEverything

BEGIN
    SET idText to ""
    SET nameText to ""
    SET locationText to ""
    SET phoneText to ""
    SET emailText to ""
    SET priceText to ""
    SET referralText to ""
    SET trainerText to ""

    CALL button.clearSelection()

    CALL yearDob.setSelectedIndex(0)
    CALL monthDob.setSelectedIndex(0)
    CALL dayDob.setSelectedIndex(0)

    CALL planYear.setSelectedIndex(0)
```

```
    CALL planMonth.setSelectedIndex(0)
    CALL planDay.setSelectedIndex(0)

    CALL planC.setSelectedIndex(0)
END METHOD
METHOD searchMember

INPUT: id (an integer)
OUTPUT: A GymMember object or NULL

BEGIN
    FOR EACH exist IN gym DO
        IF exist.getId() EQUALS id THEN
            RETURN exist
        END IF
    END FOR

    RETURN NULL
END METHOD
METHOD detailsFromFile

BEGIN
    CREATE JFrame readFrame WITH title "Saved Member Details"
```

```
SET size of readFrame to 1000x500
CENTER readFrame on screen
SET layout of readFrame to null

CREATE textArea as non-editable JTextArea
SET font to Monospaced, PLAIN, size 12
SET bounds of textArea to (10, 10, 960, 440)

TRY
    SET file to new File("MemberDetails.txt")

    IF file does not exist OR file length is 0 THEN
        SET textArea text to "No saved member details found."
    ELSE
        CREATE FileReader object called reading for the file
        SET chart to 0
        SET content to empty string

        WHILE reading.read() NOT EQUAL TO -1 DO
            APPEND (char) chart to content
        END WHILE

        SET textArea text to content
```

```
        CLOSE reading
    END IF
  CATCH IOException ex
    DISPLAY message dialog with "Error reading file: " + ex.getMessage()
  END TRY

  ADD textArea to readFrame
  SET readFrame visible
END METHOD
METHOD saveToFile

BEGIN
  TRY
    CREATE file object with path "MemberDetails.txt"
    CREATE FileWriter object writing for file (overwrite mode)

    IF file is empty THEN
      WRITE formatted header row to file
    END IF

    IF gym list is empty THEN
      DISPLAY message "No members to save."
      CLOSE writing
```

```
        RETURN
    END IF


    FOR EACH check IN gym DO
        IF check IS INSTANCEOF RegularMember THEN
            CONVERT check to RegularMember r
            WRITE formatted line with:
                r.getId(), r.getName(), r.getLocation(), r.getPhone(), r.getEmail(),
                r.getMembershipStartDate(), r.getPlan(), r.getPrice(),
                r.getAttendance(), r.getLoyaltyPoints(), r.getActiveStatus(),
                "N/A", "N/A", "N/A"
            CALL clearEverything()
        ELSE IF check IS INSTANCEOF PremiumMember THEN
            CONVERT check to PremiumMember pre
            WRITE formatted line with:
                pre.getId(), pre.getName(), pre.getLocation(), pre.getPhone(),
pre.getEmail(),
                pre.getMembershipStartDate(), "Premium", pre.getPremiumCharge(),
                pre.getAttendance(), pre.getLoyaltyPoints(), pre.getActiveStatus(),
                pre.getIsFullPayment(), pre.getDiscountAmount(), pre.getPaidAmount()
            CALL clearEverything()
        END IF
    END FOR
```

```
        PRINT "Number of members to save: " + gym.size()
        CALL writing.flush()
        CALL writing.close()
        DISPLAY message "Data saved successfully!"
    CATCH IOException ex
        DISPLAY message dialog with "Error saving data: " + ex.getMessage()
    END TRY
END METHOD
METHOD actionPerformed(ActionEvent event)
    // Selecting the price according to the plan
    STRING selectedPlan = (STRING) planC.getSelectedItem()
    SWITCH selectedPlan
        CASE "Basic":
            priceText.setText("6500")
            BREAK
        CASE "Standard":
            priceText.setText("8500")
            BREAK
        CASE "Deluxe":
            priceText.setText("10500")
            BREAK
        DEFAULT:
```

```
        priceText.setText(" ")
END SWITCH


// To Display
IF event.getSource() == display THEN
    STRING abc = idText.getText()
    IF abc.isEmpty() THEN
        SHOW message dialog "Empty id field"
        RETURN
    END IF
    TRY
        INTEGER id = Integer.parseInt(abc) // Parse the ID from input
        BOOLEAN found = FALSE // Default value false
        FOR GymMember exist IN gym DO
            IF exist.getId() == id THEN
                IF exist instanceof RegularMember THEN
                    RegularMember regM = (RegularMember) exist
                    CALL RegularMemberAc(regM)
                END IF
                IF exist instanceof PremiumMember THEN
                    PremiumMember preM = (PremiumMember) exist
                    CALL PremiumMemberAc(preM)
                END IF
```

```
            found = TRUE // Becomes true
            BREAK
        END IF
    END FOR
    IF NOT found THEN
        SHOW message dialog "This ID does not exist, please enter the existing ID"
    END IF
  CATCH NumberFormatException e
    SHOW message dialog "Please enter a valid numeric ID."
  END TRY
END IF

// Clear button to remove data temporarily
ELSE IF event.getSource() == clear THEN
  CALL clearEverything()

// Adding regular member
ELSE IF event.getSource() == addRegular THEN
  TRY
    INTEGER id = Integer.parseInt(idText.getText())
    IF idText.getText().isEmpty() OR
      nameText.getText().isEmpty() OR
      locationText.getText().isEmpty() OR
```

```
        phoneText.getText().isEmpty() OR
        emailText.getText().isEmpty() OR
        referralText.getText().isEmpty() OR
        planC.getSelectedItem() == NULL OR
        yearDob.getSelectedItem() == NULL OR
        monthDob.getSelectedItem() == NULL OR
        dayDob.getSelectedItem() == NULL OR
        planYear.getSelectedItem() == NULL OR
        planMonth.getSelectedItem() == NULL OR
        planDay.getSelectedItem() == NULL OR
        NOT (maleButton.isSelected() OR femaleButton.isSelected() OR
otherButton.isSelected()) THEN
            SHOW message dialog "Please fill in all required fields."
            RETURN
        END IF

        // Validate phone number
        STRING phone = phoneText.getText()
        IF NOT phone.matches("^[0-9]{10}$") THEN
            SHOW message dialog "Please enter a valid phone number."
            RETURN
        END IF
```

```
// Validate email
STRING email = emailText.getText()
IF NOT email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$") THEN
    SHOW message dialog "Please enter a valid email address."
    RETURN
END IF

STRING gender = IF maleButton.isSelected() THEN "Male" ELSE IF femaleButton.isSelected() THEN "Female" ELSE IF otherButton.isSelected() THEN "Others" ELSE "Unspecified"
STRING dob = yearDob.getSelectedItem() + "/ " + monthDob.getSelectedItem() + "/ " + dayDob.getSelectedItem()
STRING start = planYear.getSelectedItem() + "/" + planMonth.getSelectedItem() + "/" + planDay.getSelectedItem()
// All the information is stored in an object of child class
RegularMember fault = new RegularMember(id, nameText.getText(), locationText.getText(), phoneText.getText(), emailText.getText(), gender, dob, start, referralText.getText())
FOR GymMember exit IN gym DO
    IF exit.getId() == id THEN
        SHOW message dialog "ID already exists."
        RETURN // Don't add if ID exists
    END IF
END FOR
```

```
        gym.add(fault) // Adding fault in gym arraylist
        SHOW message dialog "Regular member added."
    CATCH Exception ex
        SHOW message dialog "Please fill all the details."
    END TRY
END IF
// Adding premium member
ELSE IF event.getSource() == addPremium THEN
    TRY
        INTEGER id = Integer.parseInt(idText.getText())
        IF idText.getText().isEmpty() OR
            nameText.getText().isEmpty() OR
            locationText.getText().isEmpty() OR
            phoneText.getText().isEmpty() OR
            emailText.getText().isEmpty() OR
            trainerText.getText().isEmpty() OR
            planC.getSelectedItem() == NULL OR
            yearDob.getSelectedItem() == NULL OR
            monthDob.getSelectedItem() == NULL OR
            dayDob.getSelectedItem() == NULL OR
            planYear.getSelectedItem() == NULL OR
            planMonth.getSelectedItem() == NULL OR
            planDay.getSelectedItem() == NULL OR
```

```
        NOT (maleButton.isSelected() OR femaleButton.isSelected() OR
otherButton.isSelected()) THEN
            SHOW message dialog "Please fill in all required fields."
            RETURN
        END IF

        // Validate phone number
        STRING phone = phoneText.getText()
        IF NOT phone.matches("^[0-9]{10}$") THEN
            SHOW message dialog "Please enter a valid phone number (10 digits)."
            RETURN
        END IF

        // Validate email
        STRING email = emailText.getText()
        IF NOT email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$") THEN
            SHOW message dialog "Please enter a valid email address."
            RETURN
        END IF

        STRING gender = IF maleButton.isSelected() THEN "Male" ELSE IF
femaleButton.isSelected() THEN "Female" ELSE IF otherButton.isSelected() THEN
"Others" ELSE "Unspecified"
```

```
        STRING dob = yearDob.getSelectedItem() + "/ " + monthDob.getSelectedItem()
+ "/ " + dayDob.getSelectedItem()

        STRING start = planYear.getSelectedItem() + "/" + planMonth.getSelectedItem()
+ "/" + planDay.getSelectedItem()

        // Storing the value of every component in an object

        PremiumMember fault = new PremiumMember(id, nameText.getText(),
locationText.getText(), phoneText.getText(), emailText.getText(), gender, dob, start,
trainerText.getText())


        FOR GymMember exit IN gym DO
            IF exit.getId() == id THEN
                SHOW message dialog "ID already exists."
                RETURN // Don't add if ID exists
            END IF
        END FOR
        gym.add(fault)
        SHOW message dialog "Premium member added."
    CATCH Exception ex
        SHOW message dialog "Invalid"
    END TRY


// Reading from file
ELSE IF event.getSource() == readFile THEN
    CALL detailsFromFile()
```

```
    // Saving to file
    ELSE IF event.getSource() == saveFile THEN
        CALL saveToFile()

    // Mark attendance for Regular Member
    ELSE IF event.getSource() == markAttendance THEN
        GymMember attend = searchMember(Integer.parseInt(rIdText.getText()))
        IF attend instanceof RegularMember THEN
            RegularMember reg = (RegularMember) attend
            IF reg.activeStatus THEN
                CALL reg.markAttendance()
                attendText.setText(reg.getAttendance() as STRING)
                loyalText.setText(reg.getLoyaltyPoints() as STRING)
                SHOW message dialog "Attendance marked for " + attend.getId()
                CALL saveToFile()
            ELSE
                SHOW message dialog "Attendance cannot be marked.\nAccount is not
activated"
            END IF
        END IF

    // Mark attendance for Premium Member
```

```
    ELSE IF event.getSource() == pMarkAttendance THEN
        GymMember attend = searchMember(Integer.parseInt(pIdText.getText()))
        IF attend instanceof PremiumMember THEN
            PremiumMember pre = (PremiumMember) attend
            IF pre.activeStatus THEN
                CALL pre.markAttendance()
                pAttendText.setText(pre.getAttendance() as STRING)
                pLoyalText.setText(pre.getLoyaltyPoints() as STRING)
                SHOW message dialog "Attendance marked for " + attend.getId()
                CALL saveToFile()
            ELSE
                SHOW message dialog "Attendance cannot be marked.\nAccount is not
activated"
            END IF
        END IF

    // Revert Premium Member
    ELSE IF event.getSource() == revertPremiumMember THEN
        GymMember revert = searchMember(Integer.parseInt(pIdText.getText()))
        IF revert instanceof PremiumMember THEN
            PremiumMember pre = (PremiumMember) revert
            IF pre.activeStatus THEN
                CALL pre.revertPremiumMember()
```

```
            pAttendText.setText(pre.getAttendance() as STRING)
            pLoyalText.setText(pre.getLoyaltyPoints() as STRING)
            pActiveText.setText("Inactive")
            SHOW message dialog "Membership reverted"
            CALL saveToFile()
        ELSE
            SHOW message dialog "Membership not activated"
        END IF
    END IF

// Revert Regular Member
ELSE IF event.getSource() == revertRegularMember THEN
    GymMember revert = searchMember(Integer.parseInt(rIdText.getText()))
    STRING fReason = removalReasonText.getText()
    IF revert instanceof RegularMember THEN
        IF fReason IS NULL OR fReason.isEmpty() THEN
            SHOW message dialog "Please fill the reason"
        ELSE
            TRY
                INTEGER.parseInt(fReason)  // Checking if reason is numeric
                SHOW message dialog "The reason must be valid"
            CATCH NumberFormatException e
                RegularMember reg = (RegularMember) revert
```

```
            IF reg.activeStatus THEN
                CALL reg.revertRegularMember(fReason)
                rPlanText.setText(reg.getPlan())
                attendText.setText(reg.getAttendance() as STRING)
                loyalText.setText(reg.getLoyaltyPoints() as STRING)
                activeText.setText("Inactive")
                removalReasonText.setText("")
                SHOW message dialog "Membership reverted. \nReason: " + fReason
                CALL saveToFile()
            ELSE
                SHOW message dialog "Membership not activated"
            END IF
        END TRY
    END IF
END IF

// Activate membership for Regular Member
ELSE IF event.getSource() == activateMembership THEN
    GymMember active = searchMember(Integer.parseInt(rIdText.getText()))
    IF NOT active.getActiveStatus() THEN
        CALL active.activateMembership()
        SHOW message dialog "Membership Activated."
        activeText.setText("Active")
```

```
        CALL saveToFile()
    ELSE
        SHOW message dialog "Membership already Activated."
    END IF

// Activate membership for Premium Member
ELSE IF event.getSource() == pActivateMembership THEN
    GymMember active = searchMember(Integer.parseInt(pIdText.getText()))
    IF NOT active.getActiveStatus() THEN
        CALL active.activateMembership()
        SHOW message dialog "Membership Activated."
        pActiveText.setText("Active")
        CALL saveToFile()
    ELSE
        SHOW message dialog "Membership already Activated."
    END IF

// Deactivate membership for Premium Member
ELSE IF event.getSource() == pDeactivateMembership THEN
    GymMember deactive = searchMember(Integer.parseInt(pIdText.getText()))
    IF deactive.getActiveStatus() THEN
        CALL deactive.deactivateMembership()
        SHOW message dialog "Membership Deactivated."
```

```
        pActiveText.setText("Inactive")
        CALL saveToFile()
    ELSE
        SHOW message dialog "Membership already Deactivated."
    END IF

// Deactivate membership for Regular Member
ELSE IF event.getSource() == deactivateMembership THEN
    GymMember deactive = searchMember(Integer.parseInt(rIdText.getText()))
    IF deactive.getActiveStatus() THEN
        CALL deactive.deactivateMembership()
        SHOW message dialog "Membership Deactivated."
        activeText.setText("Inactive")
        CALL saveToFile()
    ELSE
        SHOW message dialog "Membership already Deactivated."
    END IF

// Upgrade Plan for Regular Member
ELSE IF event.getSource() == upgradePlan THEN
    GymMember upgrade = searchMember(Integer.parseInt(rIdText.getText()))
    IF upgrade instanceof RegularMember THEN
        RegularMember reg = (RegularMember) upgrade
```

```
        IF reg.activeStatus THEN
            STRING newPlan = SHOW input dialog "Enter New
Plan(Basic,Standard,Deluxe):"
            IF newPlan IS NOT NULL THEN
                STRING finall = reg.upgradePlan(newPlan)
                IF finall == "Sucessfully updated the membership!!" THEN
                    SHOW message dialog finall + "\nNew plan: " + reg.getPlan() + "\nNew
price: " + reg.getPrice()
                    rPlanText.setText(reg.getPlan())
                ELSE
                    SHOW message dialog finall
                    CALL saveToFile()
                END IF
            ELSE
                SHOW message dialog "Empty input"
            END IF
        ELSE
            SHOW message dialog "Membership not activated."
        END IF
    END IF

  // Pay due amount for Premium Member
  ELSE IF event.getSource() == due THEN
```

```
GymMember due = searchMember(Integer.parseInt(pIdText.getText()))
STRING amt = pAmtText.getText()
IF due.activeStatus THEN
   IF amt.isEmpty() THEN
      SHOW message dialog "Please Enter the balance."
      RETURN
   END IF
   TRY
      DOUBLE left = Double.parseDouble(amt)
   CATCH NumberFormatException e
      SHOW message dialog "Please Enter A Valid Number."
      RETURN
   END TRY
   IF due instanceof PremiumMember THEN
      PremiumMember pre = (PremiumMember) due
      STRING total = pre.payDueAmount(left)
      SHOW message dialog total
      pAmtText.setText("")
      CALL saveToFile()
   END IF
ELSE
   SHOW message dialog "Membership not activated."
END IF
```

```
    // Calculate discount for Premium Member
    ELSE IF event.getSource() == discountBtn THEN
        GymMember discount = searchMember(Integer.parseInt(pIdText.getText()))
        IF discount instanceof PremiumMember THEN
            PremiumMember dis = (PremiumMember) discount
            STRING display = dis.calculateDiscount()
            SHOW message dialog display
            CALL saveToFile()
        END IF
    END IF
END METHOD

METHOD main(STRING[] args)
    CALL new GymGUI()
END METHOD
END CLASS
```

5. Method Description

i. Gym Member Methods

Table 5:GYM Member method

Methods	Descriptions
getId()	this is a getter method for the id so it can be accessed everywhere in the child classes while also encrypting the data.it majorly Returns the Member's Id.
getName()	It is a getter method which returns name of the person.
getLocation()	It is a getter method which returns location of the person while also encrypting the data.
getPhone()	It is a getter method which returns phone number.
getEmail()	It is a getter method which returns email of the person.
getGender()	It is a getter method which returns gender of the person.
getDOB()	It is a getter method which Returns DOB of the person
getMembershipStartDate()	It is a getter method which Return membership Start Date.
getAttendance()	It is a getter method which returns attendance of the person. .
getLoyaltyPoints()	It is a getter method which returns loyalty points.
getActiveStatus()	It is a getter method which return active status whether if it is true or not.
abstract markAttendance()	It is a abstract method which needs to be override in sub classes.

activateMembership()	It helps to set the membership status to true if the member is deactivated.
deactivateMembership()	It helps to sets the membership status to false if the member is activated.
resetValue()	It reset the value of loyalty points, attendance, active status to default.
display ()	It prints all the member's details to the frame while required.

ii. Regular Member Methods

Table 6:RegularMember method

Methods	**Descriptions**
RegularMember	It is a constructor which passes the value to the instance variables.
markAttendance()	This method in regular member increases attendance and loyalty points based on the attendance.
upgradePlan(String nPlan)	The upgrade plan takes nPlan as input and if the upgrade id available id the attendance is sufficient.
revertRegularMember(String reason)	The revert regular member resets the value of the regular member to default after taking the reason as an input.
getPlan()	It is a getter method which returns the plan user is currently involved .
getPrice()	It is a getter method which return the price of the plan currently involved.
getRemovalReason()	It is a getter method which is necessary while reverting the regular member and it returns removal reasons.
getPlanPrice (String Plan)	It is a getter method which returns thebprice of the provided plan name.

getReferralSource ()	It is a getter method which returns the referral source.
getIsEligibleForUpgrade ()	It is a getter method which check if the plan can be upgraded or not and it returns is eligible for upgrade/
getAttendanceLimit()	It gets the attendance limit if it will be required for ither method to perform calculation and it returns attendance limit.

iii. Premium Member method

Table 7:Premium method

Method	**Description**
Premium Member(constructor)	It passes the value to the instance variable.
markAttendance()	It is an abstract method in the main class so has to be overridden where increment of attendance and loyalty point gradually happens with the premium logic.
calculateDiscount()	It is a method which helps to give discount to the member who has paid full amount.
payDue(double amount)	It is a method where you can pay the due amount and the amount should be written.
revertPremiumMember()	It is methods which revert the premium member to the default value.
display()	This method override all the display function of the main class and add the requirement of the premium class.
discountAmount()	It is a getter method which returns discount amount.
getPaidAmount()	It is a getter method which returns paid Amount.
getPremiumCharge()	It is a getter method in the premium member class and it returns premium charges.
getPersonalTrainer()	Personal trainer is one of the must requirement in the premium

	member so this getter method returns personal trainer.
getIsFullPayment()	This is the getter method which helps to return if the full payment is paid or not.

iv. Gym GUI method

Table 8:GUI Methods

Method	Description
actionPerformed(ActiveEvent event)	The main method that all button actions in the GUI.
addRegular()	It creates a regular member from the input given by the user and add to the list.
addPremium()	It creates a premium member from The input given to their own respective fields and adds it to the list.
markAttendance()	It marks attendance for regular member and premium member with the help of instance of.
activateMembership()	It is a method that helps to activate the membership if it is deactivated.
deactivateMembership()	It is a method that helps to deactivate the membership if it is activated.
upgradePlan()	This method helps to upgrade Plan for Regular Member with selected plan.
saveToFile()	It helps to save all the member details to the txt file.
detailsFromFile()	It reads and display data from the saved file.
display()	It displays the information after checking the id and if it is saved in the array list it displays it all.
clear()	It helps to set the value of all the fields to default.
searchMember(int id)	Its is a helper method which helps to search member by their id from the array list.
due()	This the method which helps to pay the amount of the regular member.

discountBtn	It helps to know the user if they have got any discount after paying the full due amount.

6. Testing

Test 1: Compile and run using command prompt

Table 9:Compile and run using command prompt

Objective	To open the GUI from command prompt
Action	• Open the command prompt, • Give the pathway of the GUI along with cd"". • Then search for javac java file • At last write java ani give GUI's name.
Expected Output	When the command "java and the GUI" file name is given it should show the GUI developed
Actual Output	After writing the correct command the GUI displayed.
Result	The test was successful.

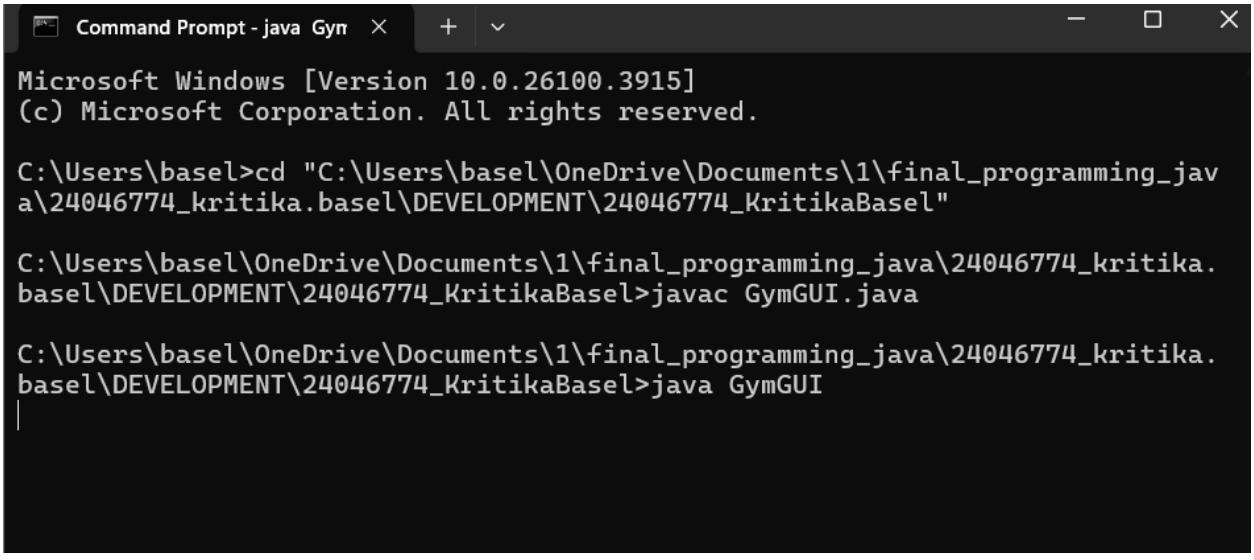


Figure 9:Command Prompt(1)

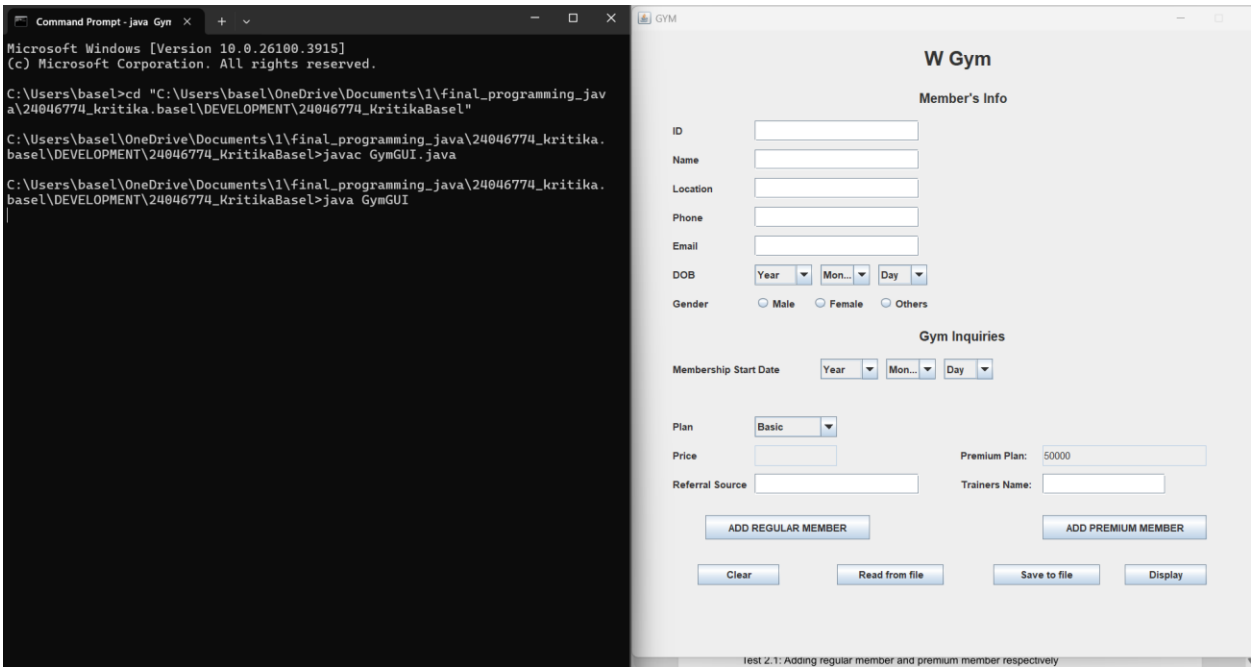


Figure 10: Command Prompt(2)

Test 2: Add regular and premium member

Table 10: Add regular and premium member

Objective	To add regular and premium member.
Action	• Fill all the required information for each member, • Then click on their respective “ADD REGULAR MEMBER” or “ADD PREMIUM MEMBER”, • If there are any empty fields or error is detected then respective message will be shown, • If there is no error found, then the message should be shown off member being added. • Then enter existing Id and click on display.
Expected Output	The required message should popup of member been added.
Actual Output	The required message popped up according to the situation.
Result	The test was successful.

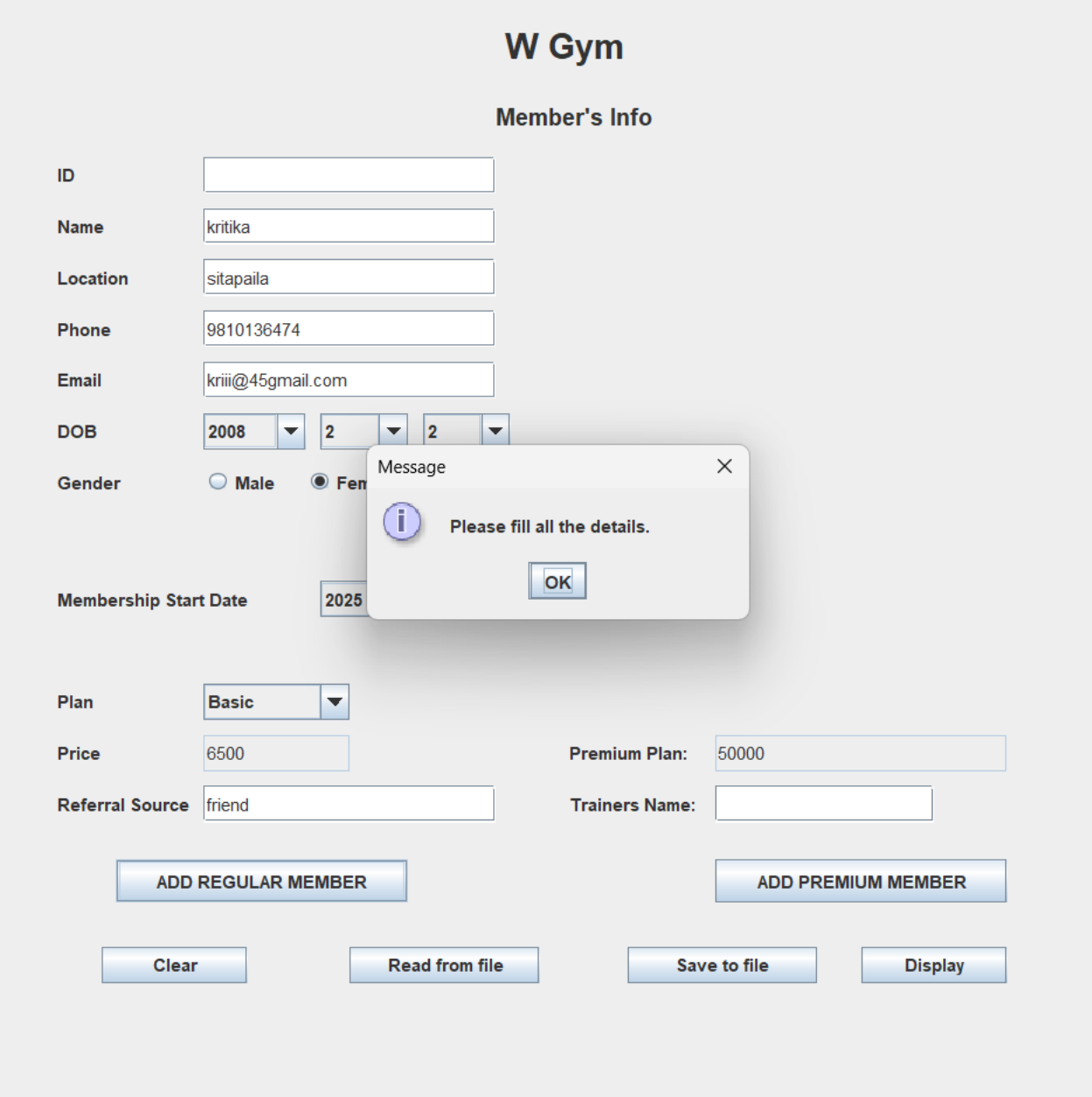


Figure 11:(Test 2)Invalid Display

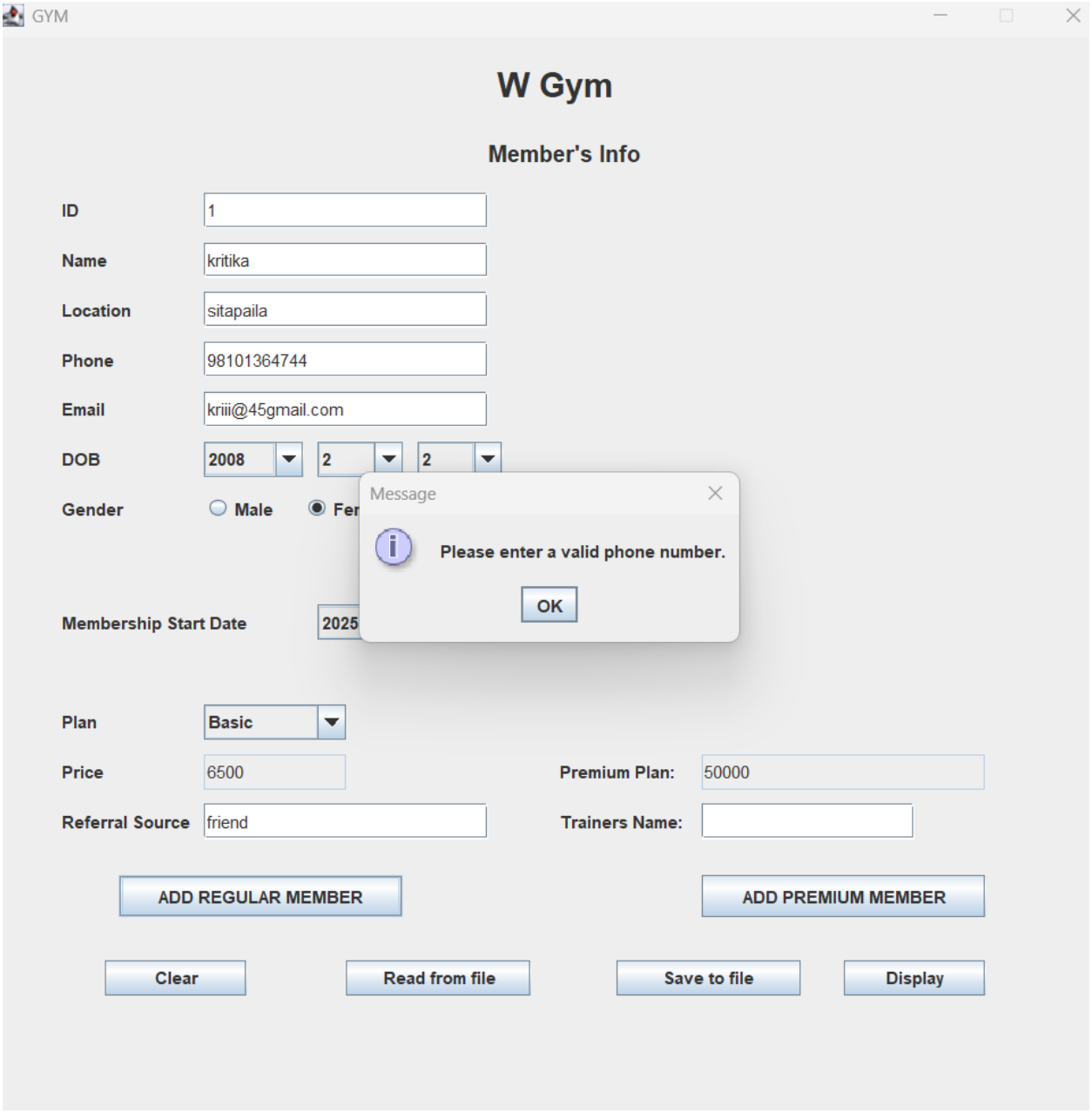


Figure 12:(Test 2)phone number required

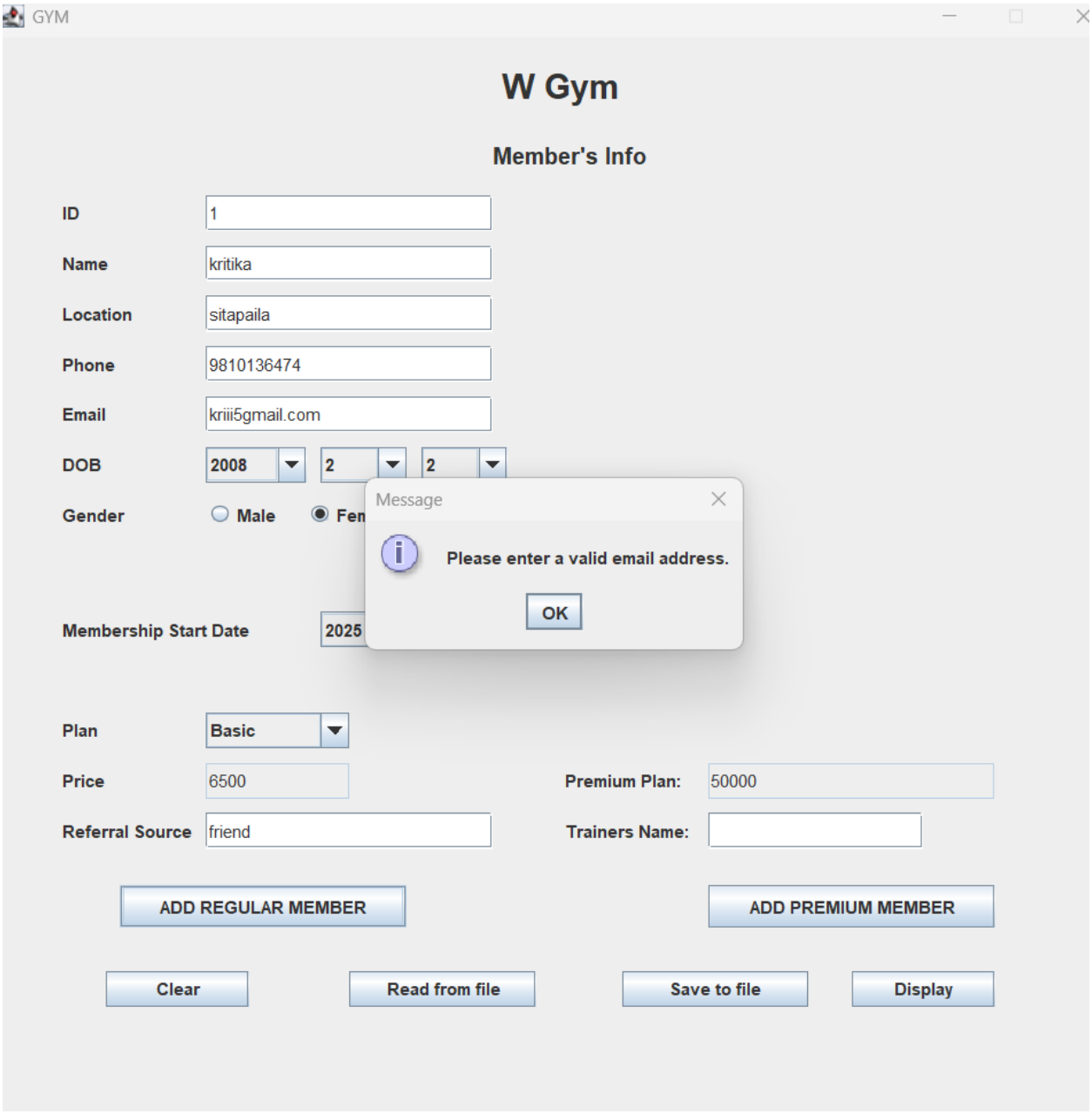


Figure 13:(Test 2)Email required

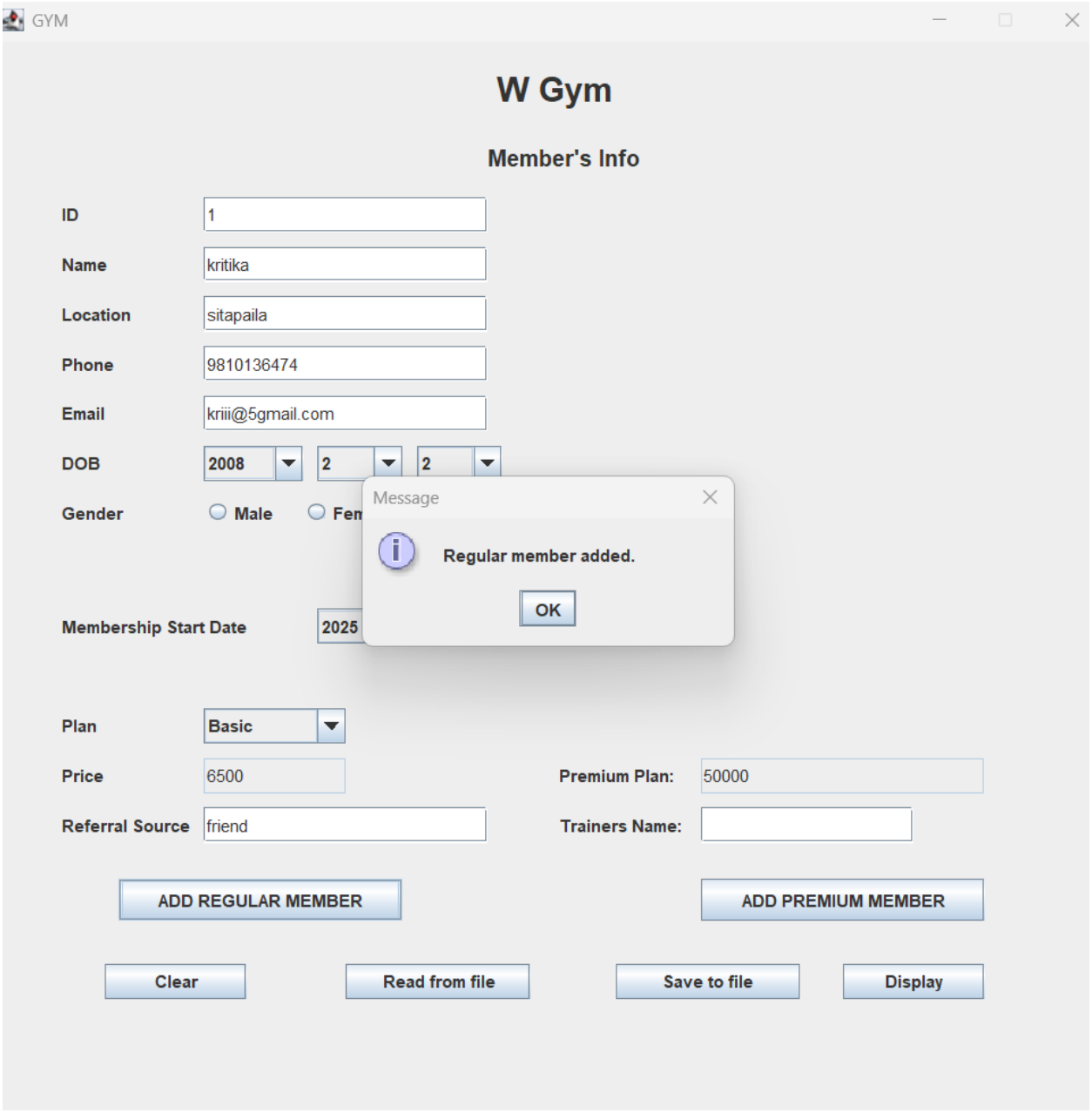


Figure 14:(test 2)Regular Member added

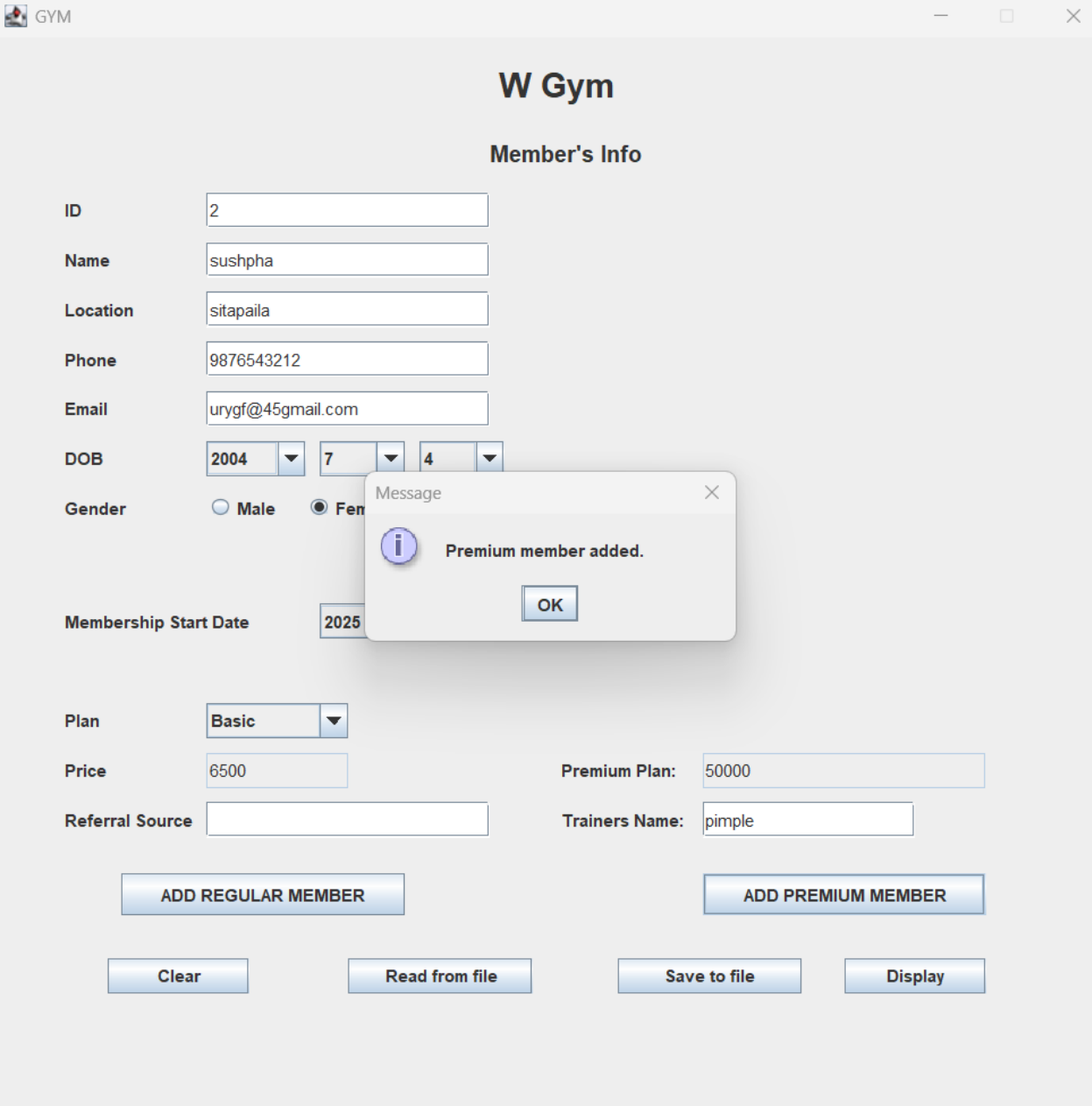


Figure 15:(Test 2) Premium member added

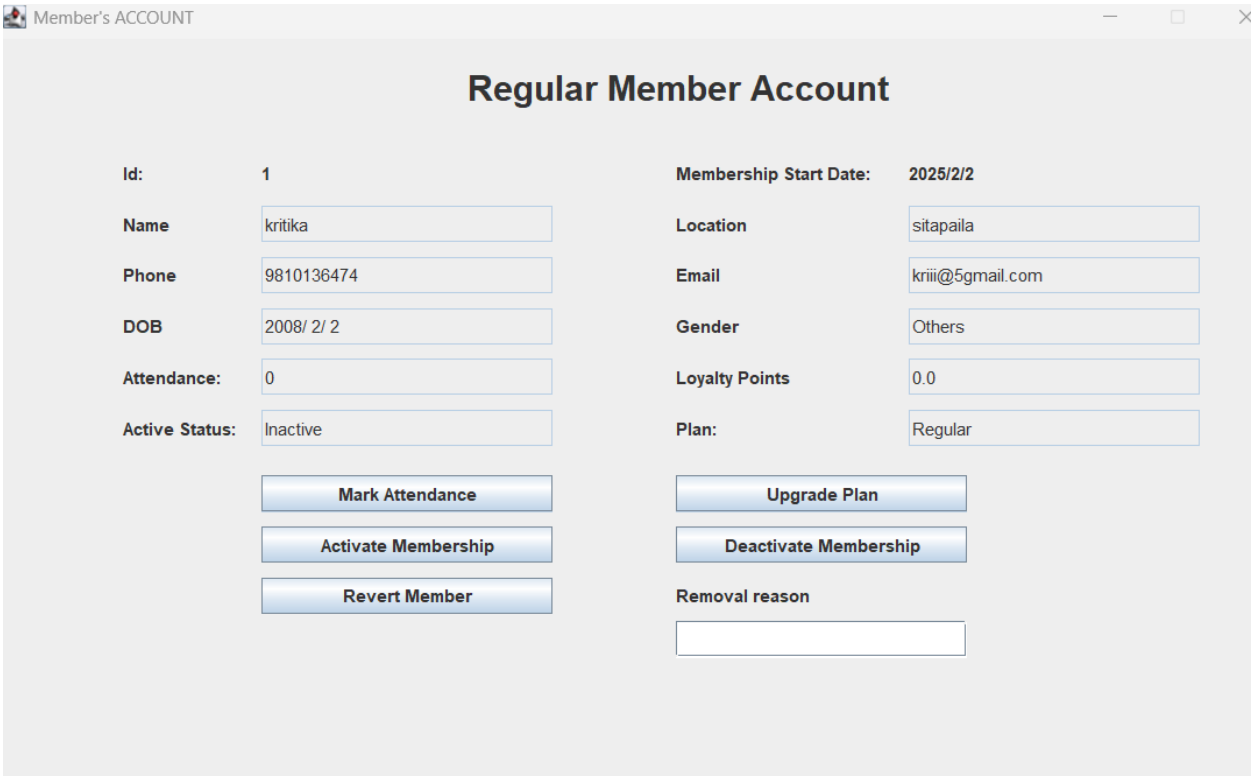


Figure 16:(test2) regular display

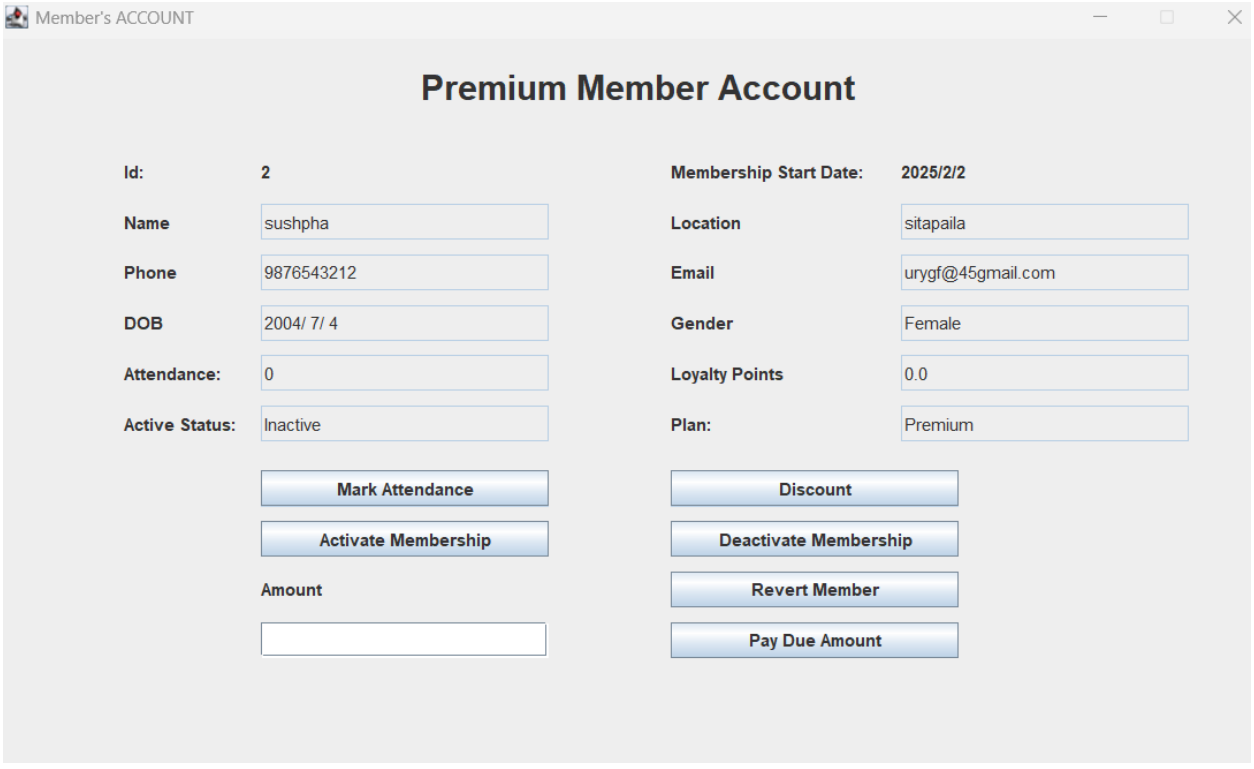


Figure 17:(Test 2)Premium Display

Test 3: Mark Attendance and upgrade plan

Table 11:Mark Attendance and upgrade plan

Objective	To mark attendance of an active member and to upgrade plan if the attendance limit is crossed.
• Action	• For mark attendance, • Enter the id of existing member and click display • Then see id the active status is off then turn that on, • Click on the mark attendance button. • For upgrade plan, • First the attendance limit should be 30 • If it is third or over, then click on the upgrade plan button • Enter the plan user wants to upgrade to • Then the plan should be upgraded.
Expected Output	When clicked on the mark attendance the attendance should be increase by one, When clicked on upgrade plan then the plan should be upgraded, and message should be shown.
Actual Output	• Attendance when up by 1 when mark attendance button was

	pressed and membership is activated. • And the membership upgraded when attendance is eligible for upgrade the plan upgrade.
Result	The test was successful.

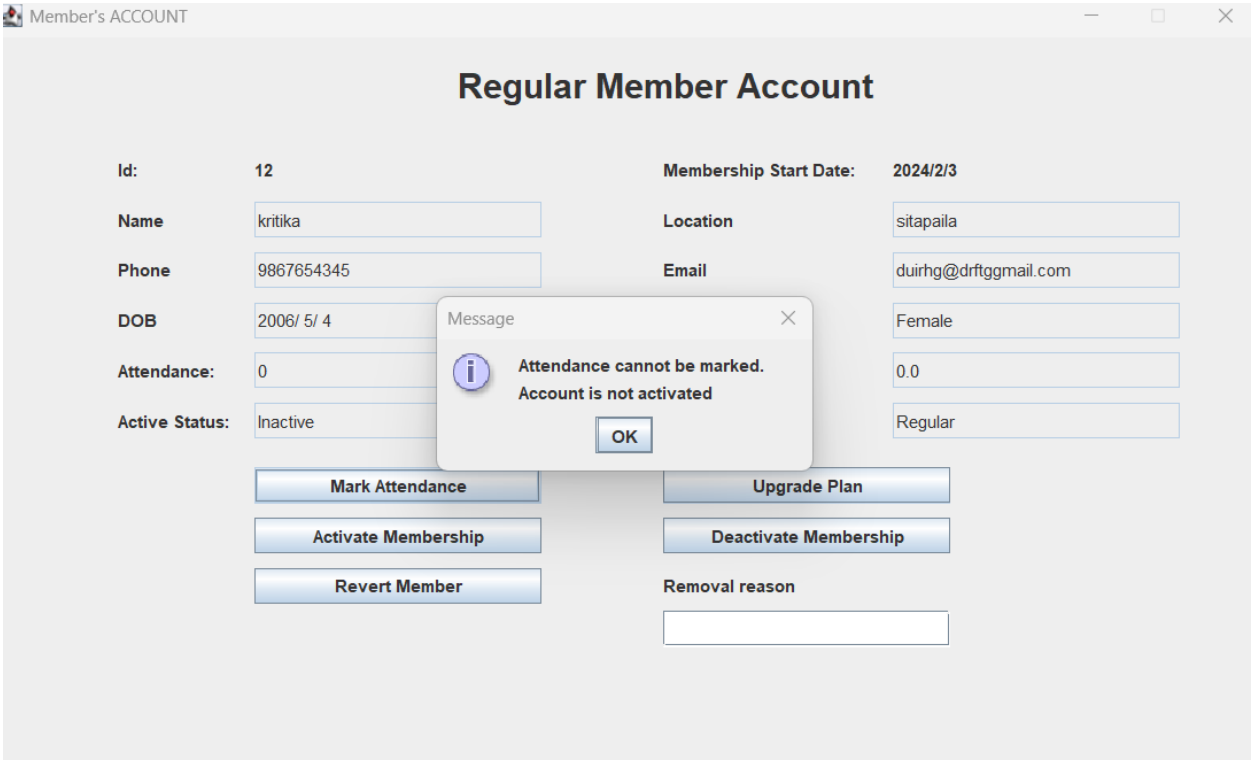


Figure 18: (Test 4)attendance not being marked

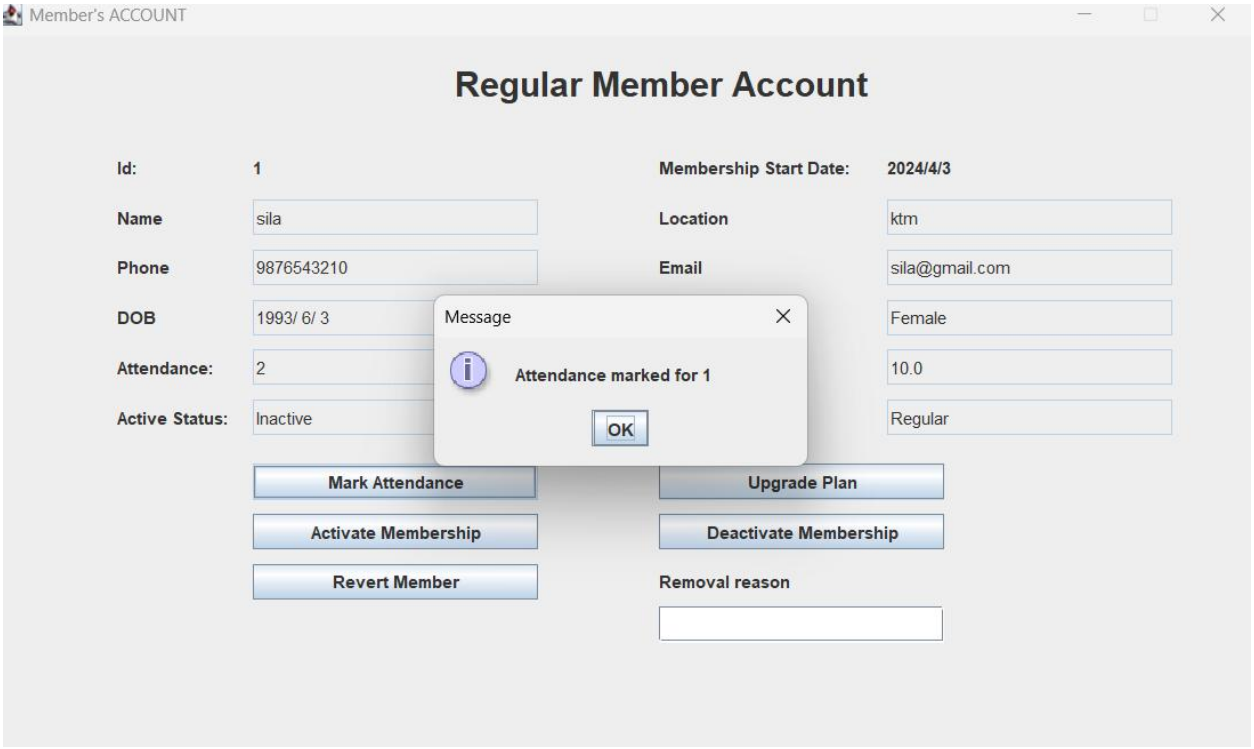


Figure 19: (Test 3)attendance marked

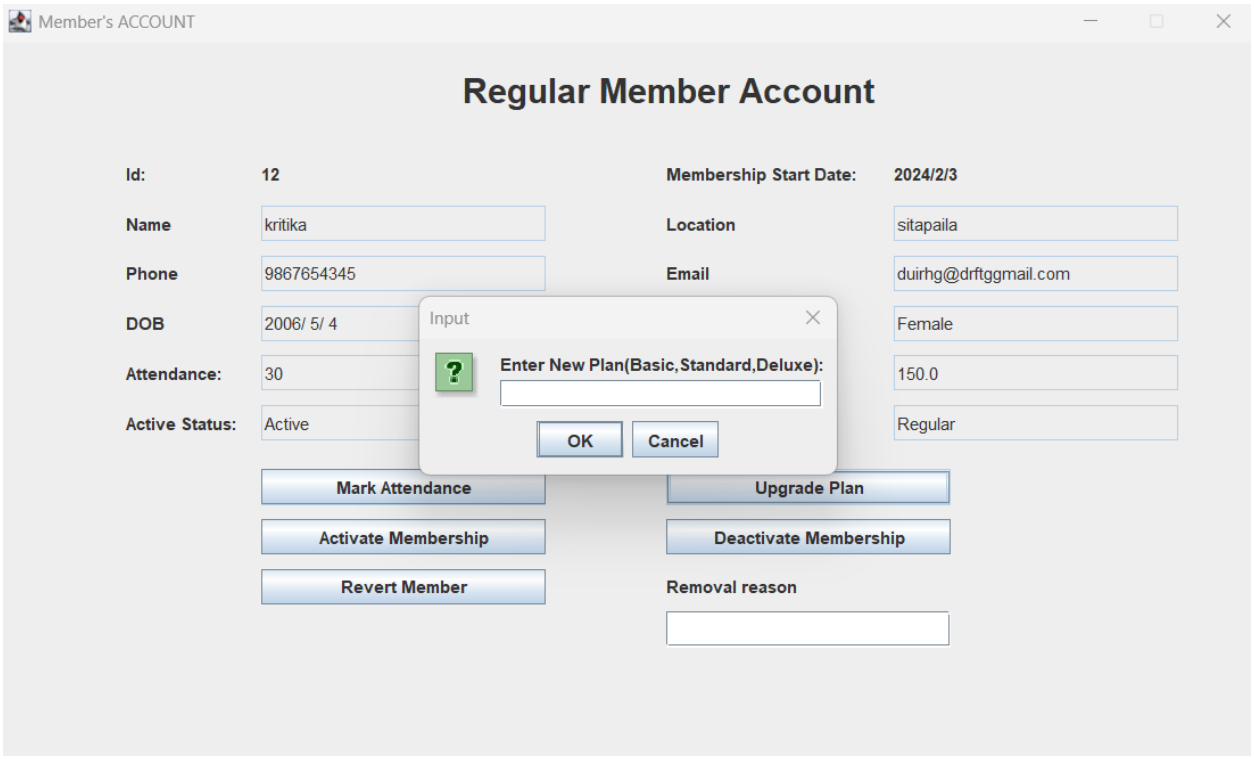


Figure 20: (Test 3)new plan input

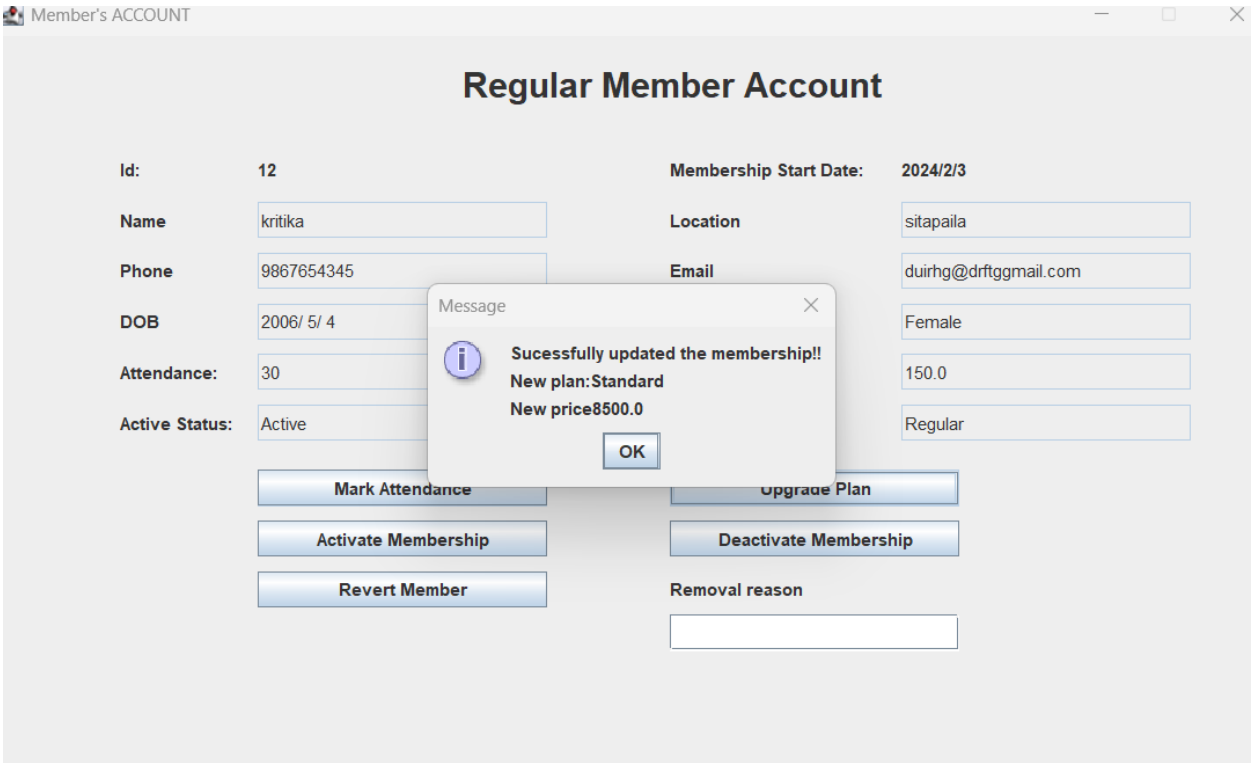


Figure 21: (Test 3)updated plan

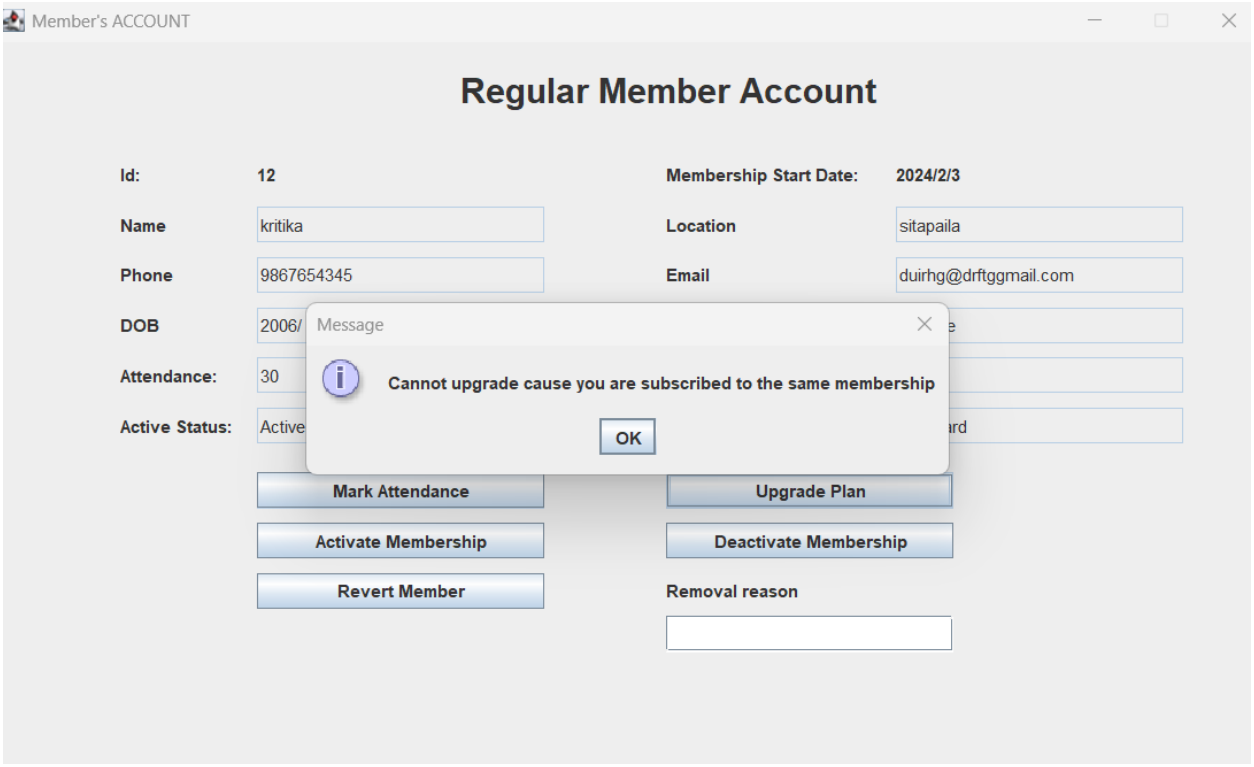


Figure 22: (Test 3)updating into same

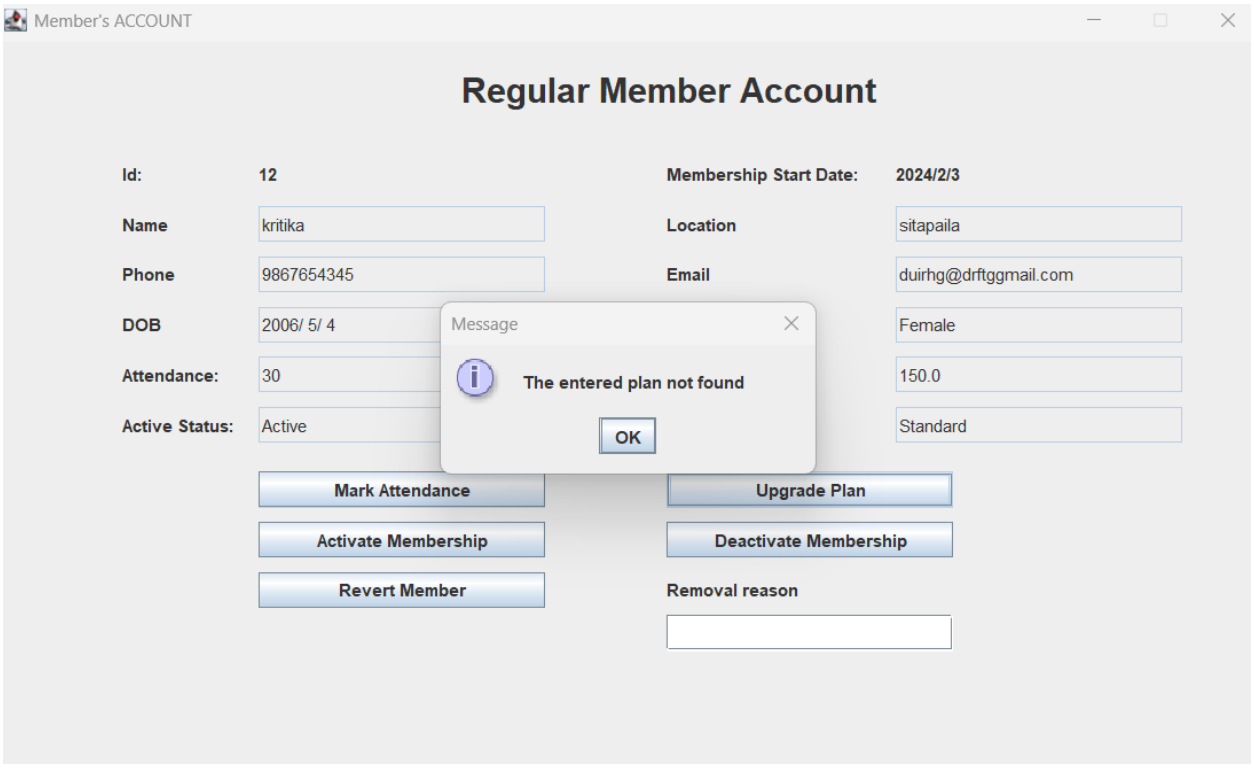


Figure 23: (Test 3)wrong value in plan

Test 4: Calculate discount, pay due and revert member

Table 12:pay due amt ,discount revert member

Objective	To calculate discount, due amount left to pay and revert member buttons.
Action	• Give the balance left to pay in amount and click on pay due amount • Then the message should be displayed • If the payed amount is equal to the plan's charge, then the discount can be applied. • Click on the discount button. • Click on the revert member button in premium but in regular also give reason before reverting.
Expected Output	For pay due amount, • When the amount is entered and the button is pressed then respective message should be shown. For discount, • the discount amount should be displayed in the message box. For revert member, • all the value of the member should be set to default.
Actual Output	When the amount is entered and pay due amount is clicked the remaining balance is shown or if the full payment is done then

	message will pop up and after full payment discount was applied. Then the membership was reverted.
Result	The test was successful.

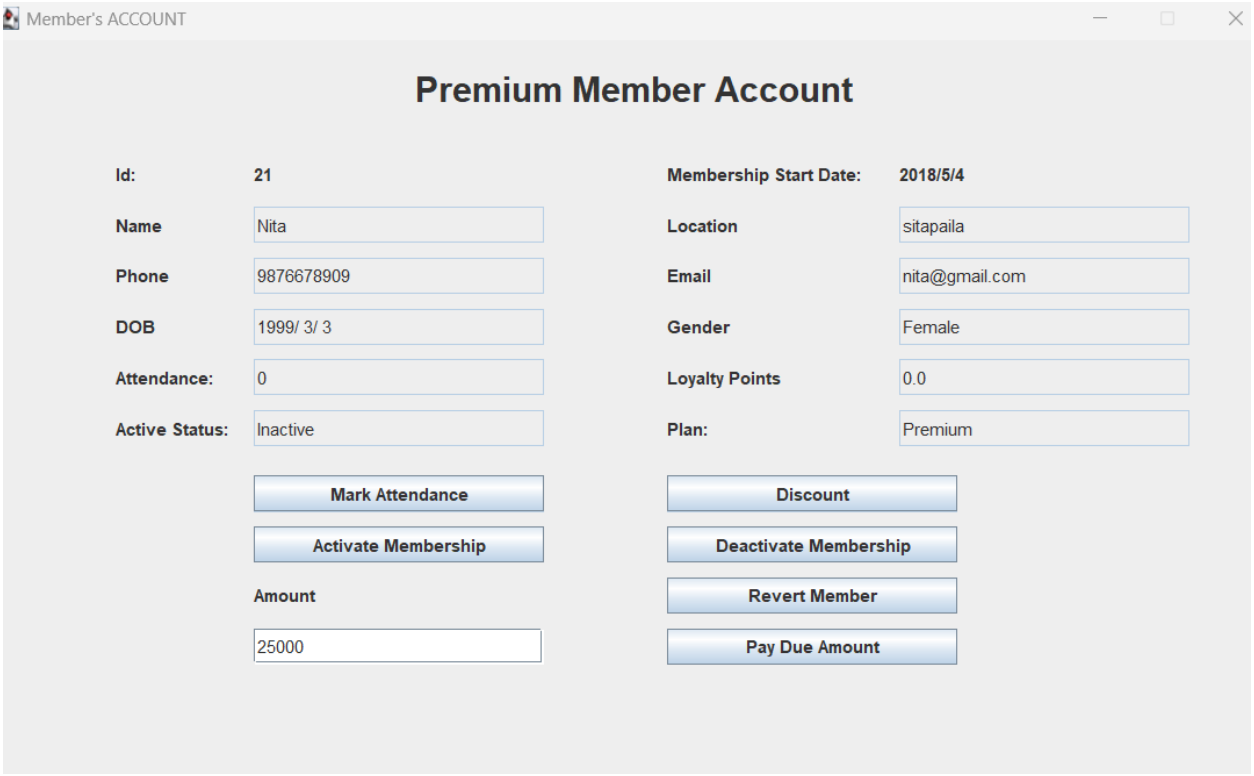


Figure 24:(Test 4)pay due balance

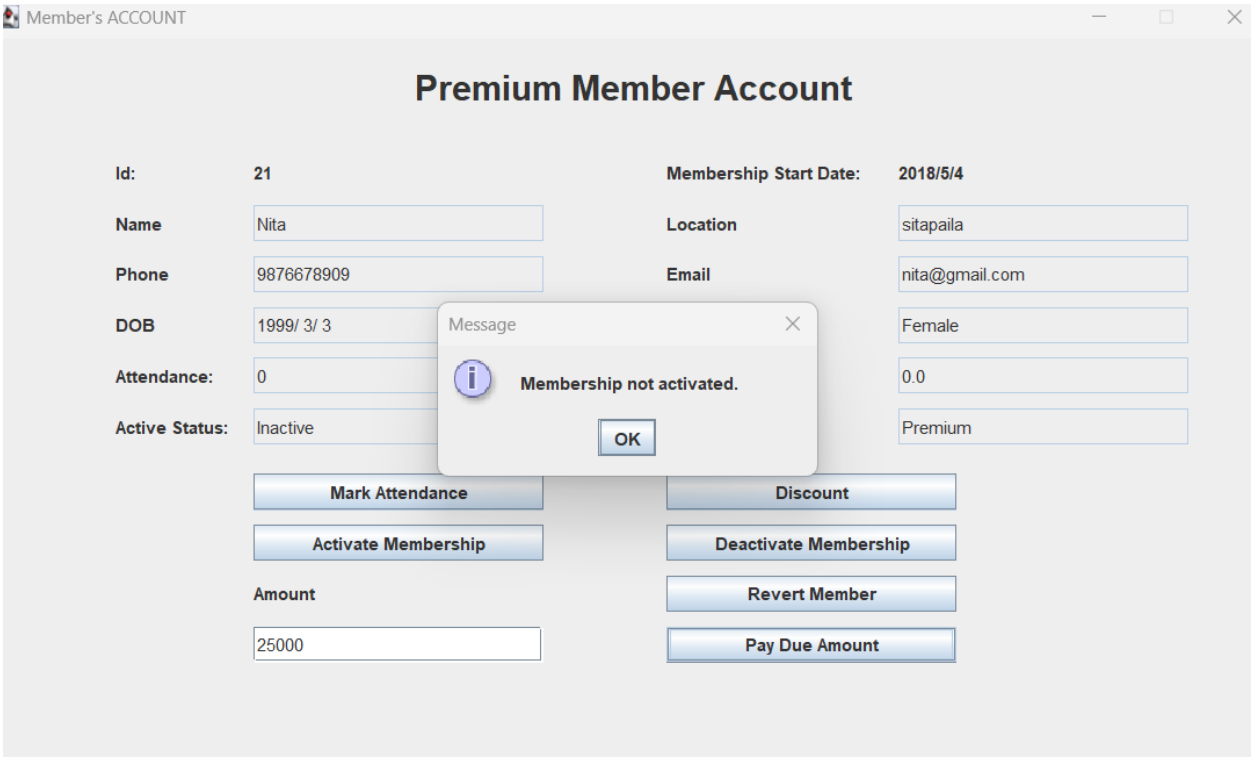


Figure 25:(Test 4)Not paid

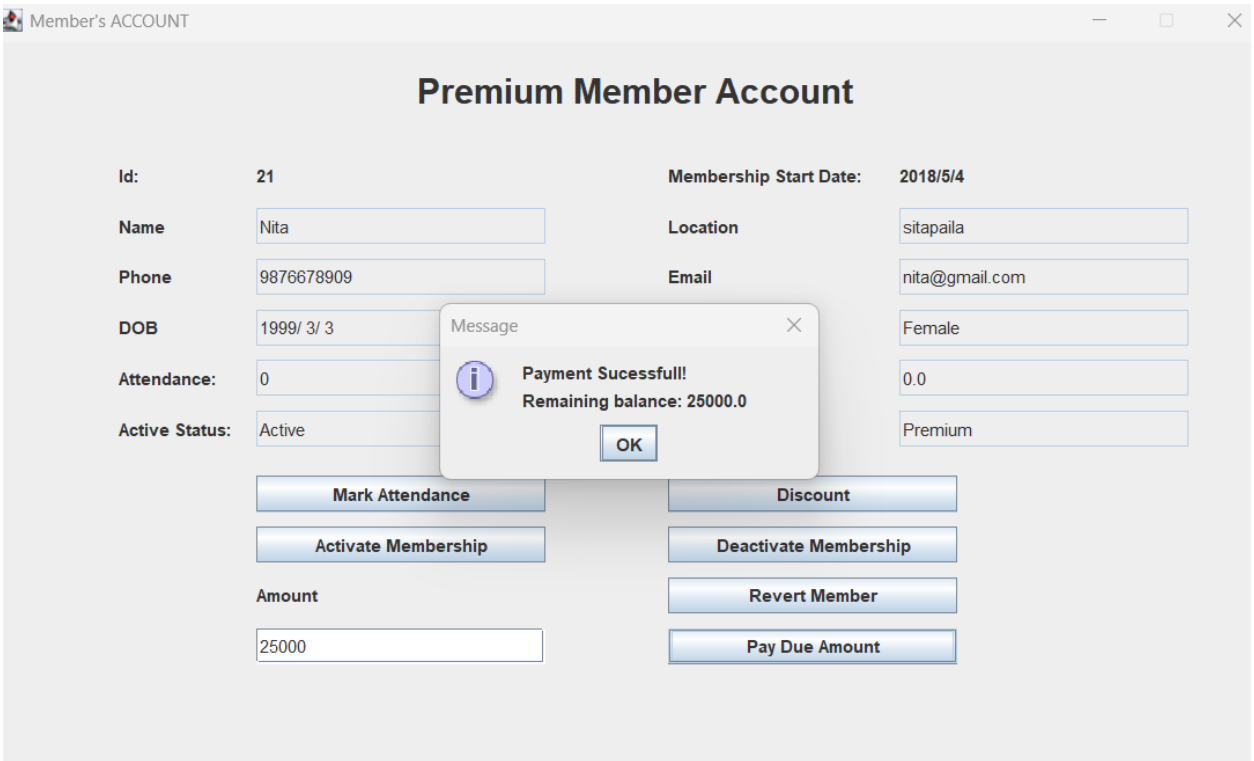


Figure 26: (Test 4))Payment successful

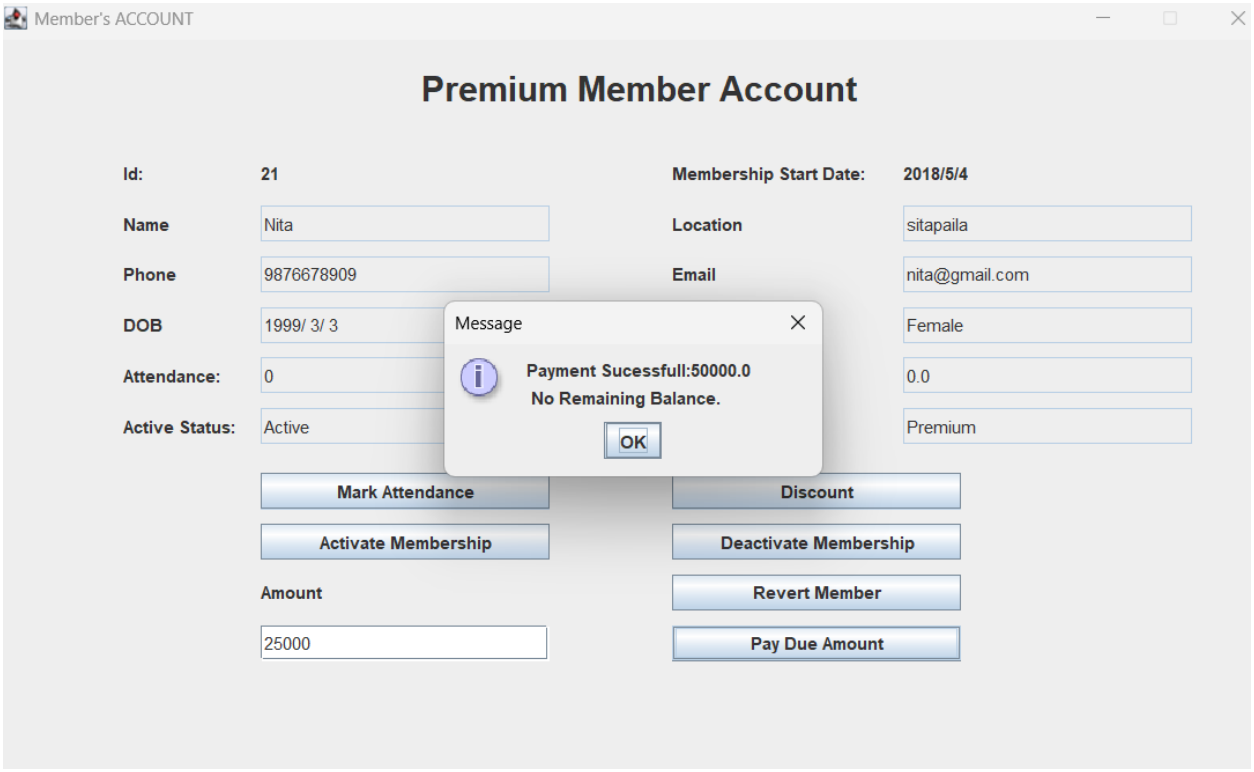


Figure 27 (Test 4))full payment done

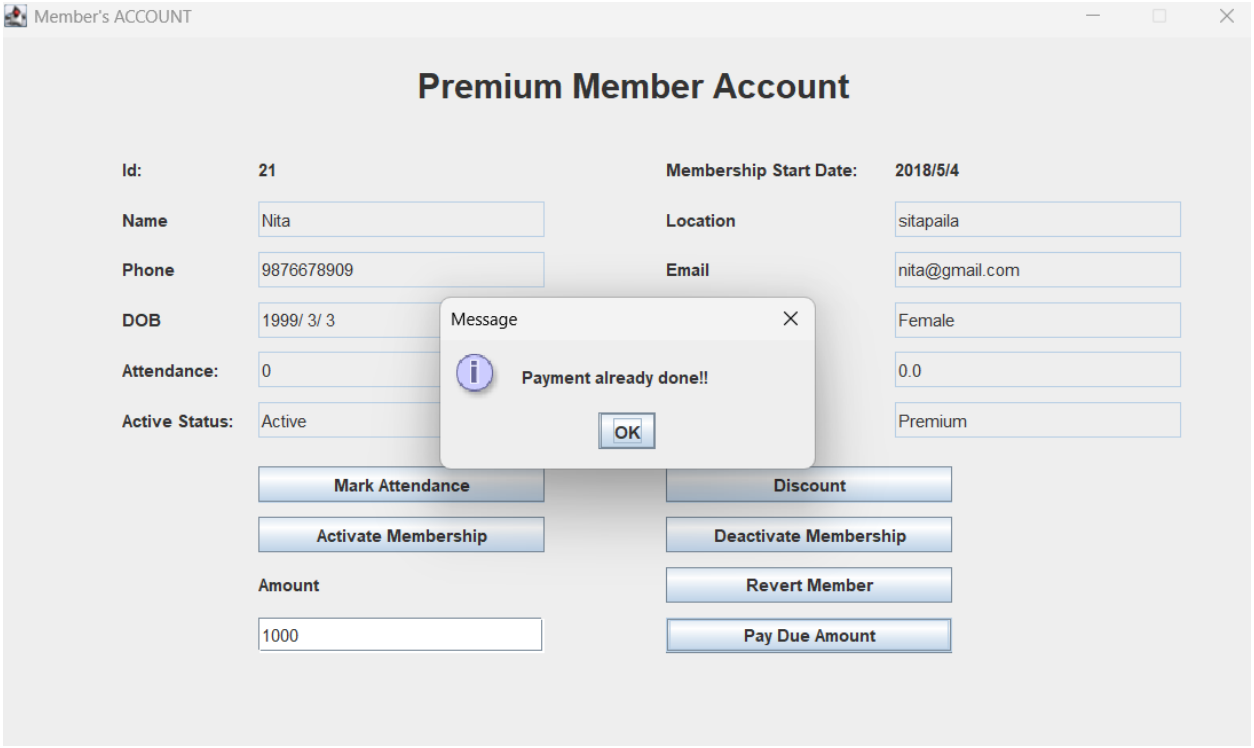


Figure 28: (Test 4))adding more payment

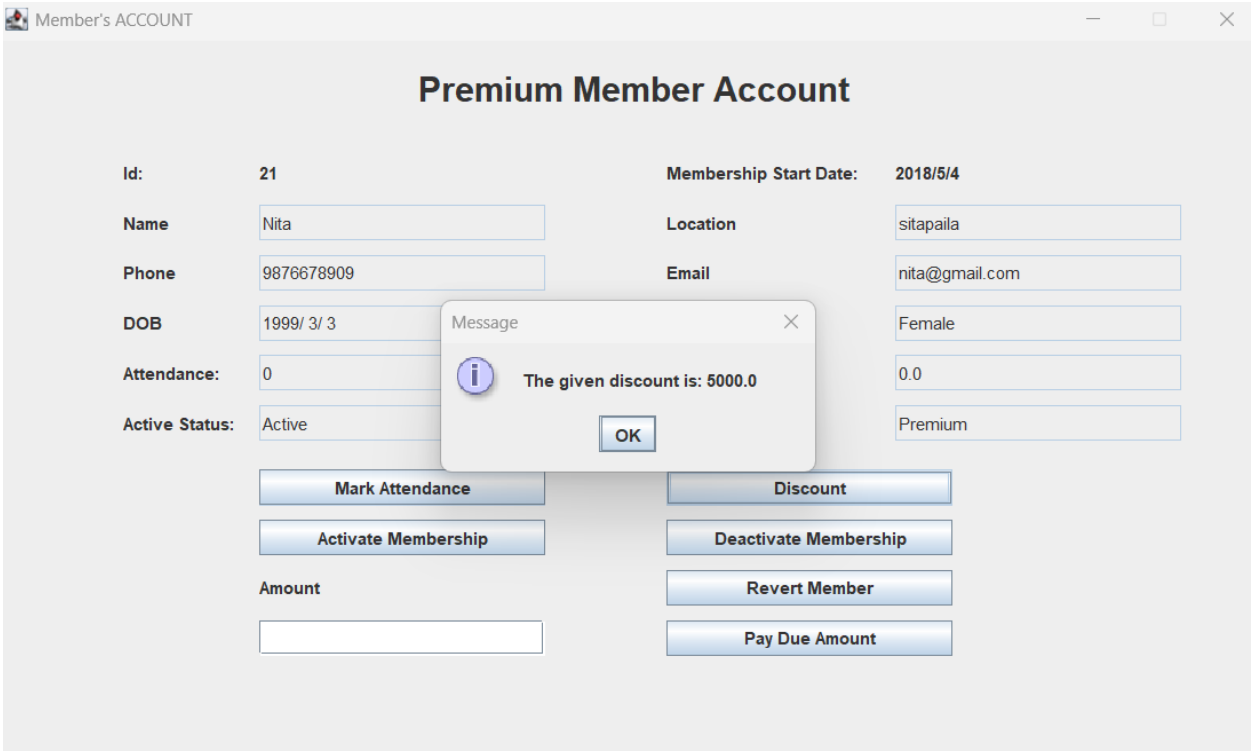


Figure 29: (Test 4))discount after full payment

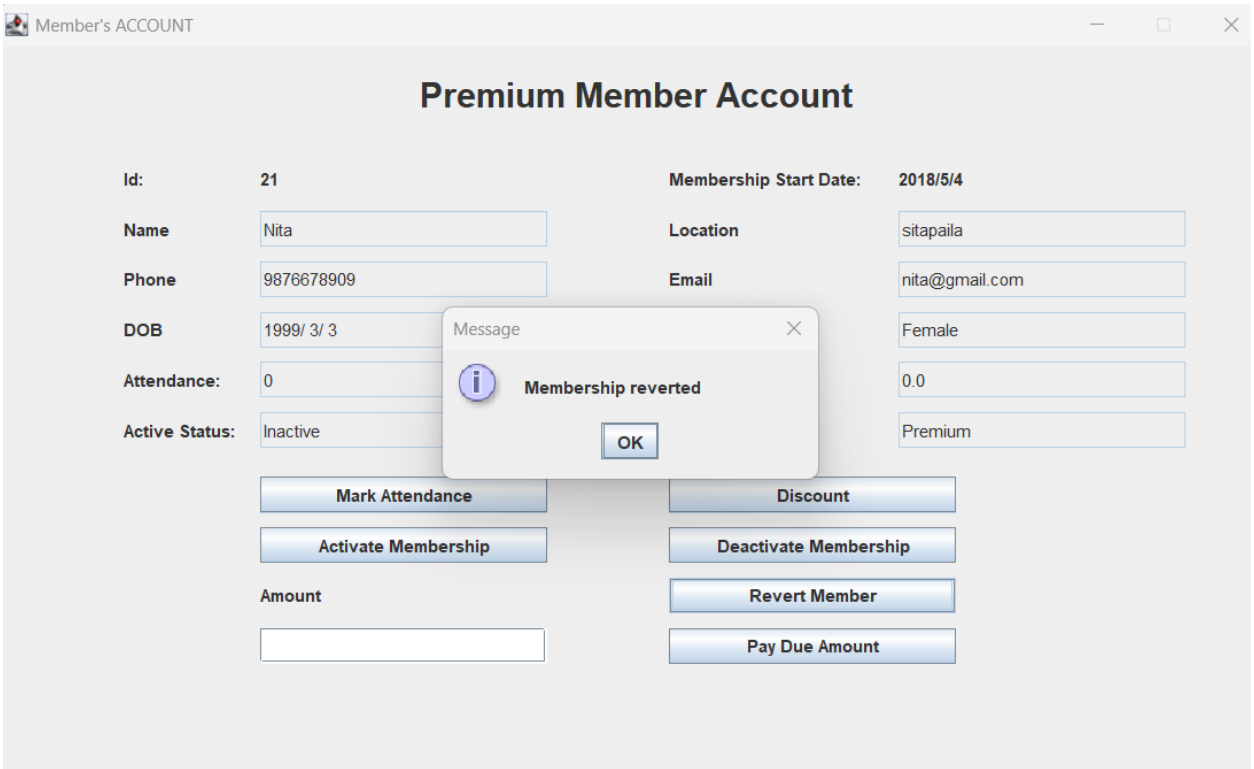


Figure 30 : (Test 4))Revert Membership

Test 5: Save and read from file

Table 13:save and read from the file

Objective	To save the member information in text file and display it.
Action	• Fill all the required information while adding a member, • Click on add to respective member button, • Then click to save to file • Then the information should be store in the file • To read from the file simply press on the read from file button.
Expected Output	• When clicked on save to file the information should be stored in an txt file • When clicked on read from file the information stored should be displayed.
Actual Output	The file was saved in txt file when clicked on save to file and when clicked on read from file the saved file's information was read.
Result	The test was successful.

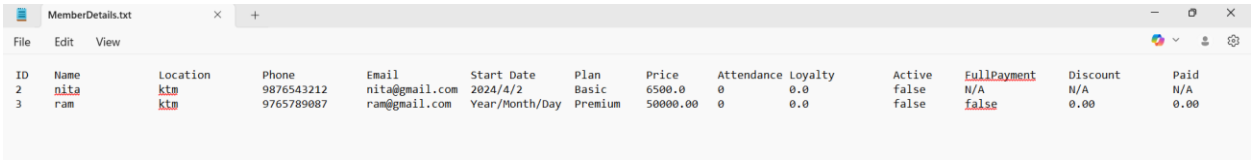


Figure 31:(Test 5)save to file

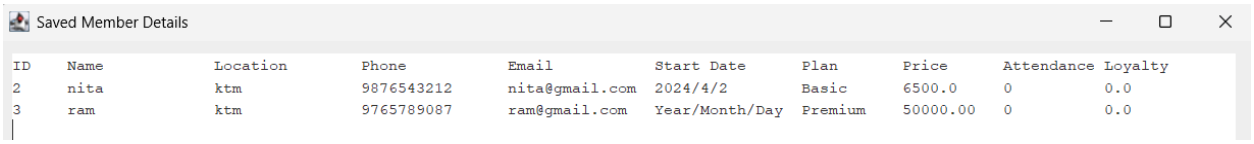


Figure 32:(Test 5)Read fro file

7. Error detection and correction

i. Syntax error detection

The error that violates the rules of the programming language, grammar of the code is syntax error. It entirely stops the code from compiling or running and error would be detected. This error can be detected before program execution.

```
//deactivate membership
public void deactivateMembership(){
    if(activeStatus==true)
    {
        this.activeStatus=false;
        Systm.out.println("The membership is disactivated");
    }
    else
    {
        System.out.println("the membership has already been disactivated");
    }
}
```

Figure 33:symtax error

```
//deactivate membership
public void deactivateMembership(){
    if(activeStatus==true)
    {
        this.activeStatus=false;
        System.out.println("The membership is disactivated");
    }
    else
    {
        System.out.println("the membership has already been disactivated");
    }
}
```

Figure 34:syntax error solved

In this code a syntax error was detected and in the code spelling of system as wrong which is a printing statement and it shows and error we fixed it by rechecking the error and then making it correct.

ii. Logical error detection

the type of error that occurs and that an compiler cannot detect and the code will run normally entirely but the logic of the code can be different than what

the user had expected. Since the compiler cannot detect these errors can be difficult to find and fix.

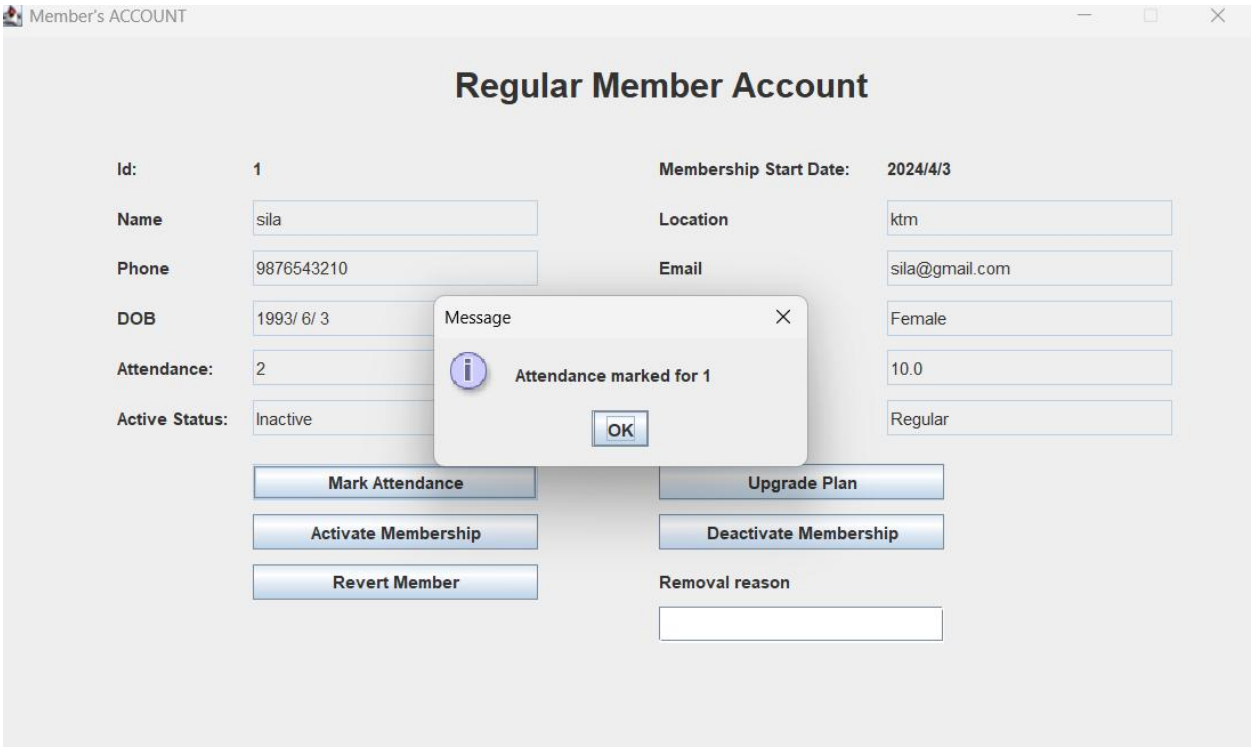


Figure 35:attendance marked error

```
}
else if(event.getSource()== markAttendance){
    GymMember attend=searchMember(Integer.parseInt(rIdText.getText()));
    if (attend instanceof RegularMember) {
        RegularMember reg = (RegularMember) attend;
        if(!reg.activeStatus){
            reg.markAttendance();
            attendText.setText(String.valueOf(reg.getAttendance()));
            loyalText.setText(String.valueOf(reg.getLoyaltyPoints()));
            JOptionPane.showMessageDialog(rframe, "Attendance marked for " +attend.getId() );
            saveToFile();
        }
```

Figure 36:Logical error

```
else if(event.getSource()== markAttendance){
    GymMember attend=searchMember(Integer.parseInt(rIdText.getText()));
    if (attend instanceof RegularMember) {
        RegularMember reg = (RegularMember) attend;
        if(reg.activeStatus){
            reg.markAttendance();
            attendText.setText(String.valueOf(reg.getAttendance()));
            loyalText.setText(String.valueOf(reg.getLoyaltyPoints()));
            JOptionPane.showMessageDialog(rframe, "Attendance marked for " +attend.getId() );
            saveToFile();
        }
```

Figure 37:logical error solved

In this part of error, the code was running fine but when the mark attendance button was clicked it was marking attendance for the member ehose id were deactivated.so to fic the code I simply removed the negative sign which was indicating if the membership was deactivated then the attendance would count.

iii. Runtime error detection

A run time error is an error that happens even if the program is running and has been successfully compiled.

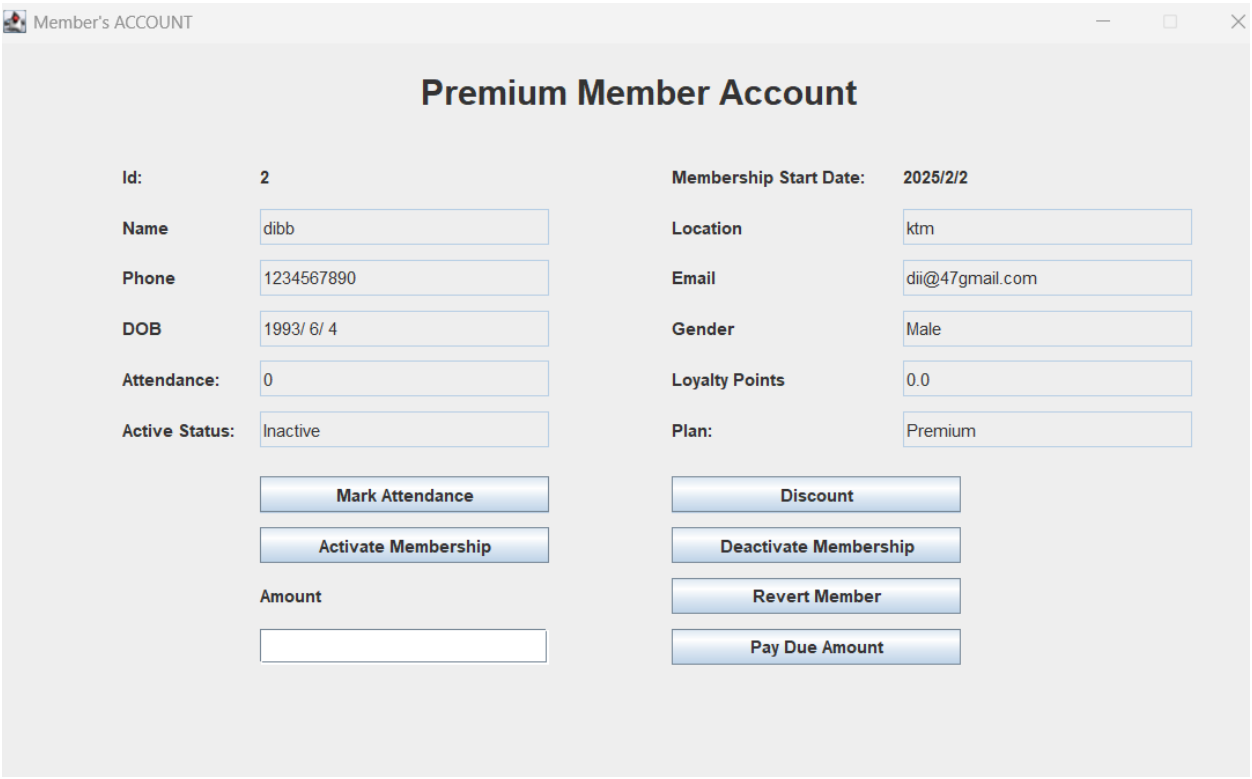


Figure 38:deactivate not functioning

```
else if(event.getSource()==pDeactivateMembership){
    GymMember deactive=searchMember(Integer.parseInt(rIdText.getText()));
    if(deactive.getActiveStatus()){
        deactive.deactivateMembership();
        JOptionPane.showMessageDialog(pframe,"Membership Deactivated.");
        pActiveText.setText("Inactive");
        saveToFile();
    }
    else{
        JOptionPane.showMessageDialog(pframe,"Membership already Deactivated.");
    }
}
```

Figure 39:runtime error

```
else if(event.getSource()==pDeactivateMembership){
    GymMember deactive=searchMember(Integer.parseInt(pIdText.getText()));
    if(deactive.getActiveStatus()){
        deactive.deactivateMembership();
        JOptionPane.showMessageDialog(pframe,"Membership Deactivated.");
        pActiveText.setText("Inactive");
        saveToFile();
    }
    else{
        JOptionPane.showMessageDialog(pframe,"Membership already Deactivated.");
    }
}
```

Figure 40:runtime error solved

In this part of code there was a run time error, where disactivate was being compared to the regular members id which let to a run time error and the deactivate button was not working which was simply fixed by putting the premium member's id to compare.

8. Conclusion

The given coursework was a very inspirational task that helped in skill development. The developed program was achieved successfully with the guidance of the teachers. In course work, we learned about the features of java, and some of the functionalities of the program. In the program, it gives access to the user to add regular member and premium members which has a lot of logic and problem solving task it also helped to gain the knowledge of overall Java programming.it made the concept of OOP much more clearer and differentiate usage of the different principle of OOP in different situations. The principles such as encapsulation, inheritance and polymorphism were implemented in this project to show its advantage and implementation and reusability of the code.

The major reason of the assignment was to help was learn a lot more about the java program so we could learn things which we can easily apply in real life and for the future benefits.it included the features like user interaction, OOP, I/O operations and many more. This also gave us the perspective of the user and make the things easier for them as we were experiencing it as well. Implementing all these feature and logic made us more proficient and made us more keen to learn.

During the process of building an implementing java in a program there were many obstacles that came I between .the development should be done to make it easier for the user use case. .As about the program development the premium and regular had to be separated as it is users free choice to select what they want the management of the data and managing the interactions between Gui components and the little backend logic, handling user input validity all these tasks we challenging but not impossible. Another challenge was to maintain a very neat and readable code avoiding complex terms and functionalities.

Enthought task was pretty tough with the help of proper guidance and breaking the task into smaller portion it was a lot more efficient to deal with the obstacles. After completion of the task the testing of the certain part that were challenging a lot more easier and

manageable Regular debugging and feedback helped a lot in the resolving process and managing errors efficiently.

9. References

Balsamiq (2025) *What is Balsamiq?* Available at: https://balsamiq.com/support/docs/getting-started/what-is-balsamiq/ (Accessed: 16 May 2025).

Indeed Editorial Team (2025) *What Is a GUI (Graphical User Interface)? Definition, Elements and Benefits*. Available at: https://www.indeed.com/career-advice/career-development/gui-meaning (Accessed: 16 May 2025).

10. Appendix

10.1. CODE FOR GYM MEMBER

```
/**
 * Write a description of class GymMember here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public abstract class GymMember
{// initializing the instance variable
    protected int id;
    protected String name;
    protected String location;
    protected String phone;
    protected String email;
    protected String gender;
    protected String DOB;
    protected String membershipStartDate;

    //initializing the values
    protected int attendance;
    protected double loyaltyPoints;
    protected boolean activeStatus;

    //constructor
    public GymMember(int id,String name,String location,String phone,String email,String gender,String DOB,String membershipStartDate)
    {
```

```
    this.id=id;//passing the value
    this.name=name;
    this.location=location;
    this.phone=phone;
    this.email=email;
    this.gender=gender;
    this.DOB=DOB;
    this.membershipStartDate=membershipStartDate;
    this.attendance=0;
    this.loyaltyPoints=0d;
    this.activeStatus=false;

}
//Encapsulation
//getter method
public int getId(){
    return this.id;
}

public String getName(){
    return this.name;
}

public String getLocation(){
    return this.location;
}

public String getPhone(){
    return this.phone;
}
```

```
public String getEmail(){
    return this.email;
}

public String getGender(){
    return this.gender;
}

public String getDOB(){
    return this.DOB;
}

public String getMembershipStartDate(){
    return this.membershipStartDate;
}

public int getAttendance(){
    return this.attendance;
}

public double getLoyaltyPoints(){
    return this.loyaltyPoints;
}

public boolean getActiveStatus(){
    return this.activeStatus;
}

//abstract method

public abstract  void markAttendance();
```

```
//active membership
public void activateMembership( ){
    if (!activeStatus)//deactivate
    {
        this.activeStatus=true;
        System.out.println("The membership is activated");
    }
    else
    {
        System.out.println("The membership is already renewed");
    }
}

//deactivate membership
public void deactivateMembership(){
    if(activeStatus==true)
    {
        this.activeStatus=false;
        System.out.println("The membership is disactivated");
    }
    else
    {
        System.out.println("the membership has already been disactivated");
    }
}

//reset values
public void resetValue(){
    loyaltyPoints=0.0;
    attendance=0;
```

```
        activeStatus=false;
    }

    //for all to print
    public void display(){
        System.out.println("The id is:"+getId());
        System.out.println("The name is:"+getName());
        System.out.println("The location is:"+getLocation());
        System.out.println("The phonenumber is:"+getPhone());
        System.out.println("The email is:"+getEmail());
        System.out.println("The gender is:"+getGender());
        System.out.println("The date of birth is:"+getDOB());
        System.out.println("The                                    membershipStartDate
is:"+getMembershipStartDate());
        System.out.println("The attendance is:"+getAttendance());
        System.out.println("The loyalty points is:"+getLoyaltyPoints());
        System.out.println("The active status is:"+getActiveStatus());
    }

}
```

10.2. CODE FOR PREMIUM MEMBER

```
/**
 * Write a description of class PremiumMember here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class PremiumMember extends GymMember

{   //instance variable
    private final double premiumCharge;
```

```
    private String personalTrainer;
    private boolean isFullPayment;
    private double paidAmount;
    private double discountAmount;

    //constructor creation
    public PremiumMember(int id,String name,String location,String phone,String email,String gender,String DOB,String membershipStartDate, String personalTrainer)
    {
        super(id,name,location,phone,email,gender,DOB,membershipStartDate);
        this.personalTrainer=personalTrainer;
        premiumCharge=50000;
        paidAmount=0;
        isFullPayment=false;
        discountAmount=0;
    }

    //Getter method
    public double getPremiumCharge(){
        return this.premiumCharge;
    }

    public String getPersonalTrainer(){
        return this.personalTrainer;
    }

    public boolean getIsFullPayment(){
        return this.isFullPayment;
    }
```

```
    public double getPaidAmount(){
        return this.paidAmount;
    }

    public double getDiscountAmount(){
        return this.discountAmount;
    }

    //Attendance & loyalty points calculation
    @Override
    public void markAttendance(){
        attendance=attendance+1;
        loyaltyPoints=loyaltyPoints+10;
        System.out.println("the attendance is: "+attendance);
        System.out.println("the loyaltyPoints is: "+loyaltyPoints);
    }

    //Payment of the due amount
    public String payDueAmount( double amount){
        if(isFullPayment==true){
            return "Payment already done!!";
        }
        //new paid amount after inserting the remaining amount
        paidAmount=amount+paidAmount;

        if(paidAmount>premiumCharge){
            paidAmount=paidAmount-amount;
            return "EXCEEDS THE LIMIT!!!.\nRequired:"+premiumCharge+"\nAlready
Paid                                                    Balance:"+paidAmount+"\nPaid
Balance"+amount+"\nRemaining:"+(premiumCharge-paidAmount);
```

```
        }
        //new isfullpayment if both are same
        isFullPayment=(paidAmount==premiumCharge);
        if(isFullPayment){
            return "Payment Sucessfull:"+paidAmount+"\n No Remaining Balance.";
        }
        else{
            double remaining=premiumCharge-paidAmount;//for the remaining due balance to pay
            return"Payment Sucessfull!\nRemaining balance: "+remaining;
        }
    }

    //to calculate discount if the fullpayment is made
    public String calculateDiscount(){
        if(isFullPayment==true){
            discountAmount= 0.10* premiumCharge;
            return "The given discount is: "+ discountAmount;
        }
        else{
            return "Discount cannot be applied.\n Payment is not complete";
        }
    }

    //to reset the values to default
    public void revertPremiumMember(){
        super.resetValue();

        personalTrainer="";
        isFullPayment=false;
        paidAmount=0;
```

```
        discountAmount=0;
        System.out.println("this account has been reverted to default premium membership");
    }

    //Display the parent and chid class output
    public void display(){
        super.display();

        System.out.println("The personal trainer is: "+getPersonalTrainer());
        System.out.println("The paid amount is "+getPaidAmount());
        System.out.println("The full payment is done?? "+getIsFullPayment());

        double remaining=premiumCharge-paidAmount;
        System.out.println("the remaining Amount is: "+remaining);
    }
}
```

10.3. CODE FOR REGULAR MEMBER

```
/**
 * Write a description of class RegularMember here.
 *
 * @author (Kritika)
 * @version (a version number or a date)
 */
public class RegularMember extends GymMember

{  //initializing instance variable
    private final int attendanceLimit;//constant
    private boolean isEligibleForUpgrade;
    private String removalReason;
    private String referralSource;
```

```
    private String plan;
    private double price;

   //Creating Constructor
   public RegularMember(int id,String name,String location,String phone,String email,String gender,String DOB,String membershipStartDate,String referralSource)
    {
        super(id,name,location,phone,email,gender,DOB,membershipStartDate);
        this.referralSource=referralSource;
        this.isEligibleForUpgrade=false;
        this.attendanceLimit=30;
        this.plan= "Basic";
        this.price=6500;
        this.removalReason="";
    }
       //getter method
   public String getReferralSource(){
        return this.referralSource;
    }

   public boolean getIsEligibleForUpgrade(){
        return this.isEligibleForUpgrade;
    }

   public String getRemovalReason(){
        return this.removalReason;
    }

   public int getAttendanceLimit(){
        return this.attendanceLimit;
```

```
}

public String getPlan(){
    return this.plan;
}

public double getPrice(){
    return this.price;
}


//attendance & loyalty points
@Override
public void markAttendance(){
    attendance=attendance+1;
    loyaltyPoints=loyaltyPoints+5;
    System.out.println("the attendance is"+attendance);
    System.out.println("the loyaltyPoints is"+loyaltyPoints);
}

// plan's price
public double getPlanPrice(String plan){
    switch(plan.toLowerCase())
    {
        case "basic":
            return 6500;

        case "standard":
            return 8500;

        case "deluxe":
```

```
            return 10500;

        default:
            System.out.println("invalid plan"+plan);
            return -1;

    }
   }

    //upgrade from the current plan
    public String upgradePlan(String nPlan){
        if(attendance<this.attendanceLimit){
            return "Cannot upgrade the current because of low attendance";
        }

        if(nPlan.equalsIgnoreCase(plan)){
            return "Cannot upgrade cause you are subscribed to the same membership";
        }

        double nPrice=getPlanPrice(nPlan);

        if(nPrice ==-1){
            return "The entered plan not found";
        }

       if (nPrice <= this.price) {
          return "Cannot upgrade to same \n or lower plans.";
       }
        //for the upgrade
        this.plan=nPlan;
```

```
        this.price=nPrice;
        return "Sucessfully updated the membership!!";
       }

    //Remove the membership
    public void revertRegularMember(String removalReason){
        //calling parent class methos
        super.resetValue();

        this.isEligibleForUpgrade=false;
        this.plan="basic";
        this.price=6500;
        this.removalReason=removalReason;
        //printing the reason
        System.out.println("this account has been reverted because"+removalReason);

    }

    //Display all the data
    public void display(){
        super.display();//firstly cally parent class method using super keyword
        System.out.println("The referral source is: "+getReferralSource());
        System.out.println("The plan's type is: "+getPlan());
        System.out.println("The price is: "+getPrice());

        if(!removalReason.isEmpty()){
            System.out.println("The removal reason is: "+getRemovalReason());
        }
    }
}
```

10.4. CODE FOR GYM GUI

```
import javax.swing.JLabel;
import javax.swing.JComboBox;
import javax.swing.JTextField;
import javax.swing.ButtonGroup;
import javax.swing.JRadioButton;
import javax.swing.JButton;
import javax.swing.JFrame;
import java.awt.Font;
import java.util.ArrayList;
import javax.swing.JPanel;
import java.awt.event.ActionListener;
import javax.swing.JOptionPane;
import java.awt.event.ActionEvent;
import java.io.IOException;
import java.io.File;
import java.io.FileWriter;
import java.io.FileReader;
import javax.swing.JTextArea;

/**
 * Write a description of class GymGUI here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */

public class GymGUI implements ActionListener
{//initializing everything outside the constructor
    private JLabel gymMember, memberInfo, gymInfo, id, name, location, phone,
trainer, startC;
```

```
    private JLabel email, dob, gender, plan, price, paidAmt, premiumPlan, discount,
start, referral;
    private JTextField idText, nameText, locationText, premiumPlanText,
phoneText, emailText, trainerText;
    private JTextField priceText, paidText, discountText, startText, referralText;
    private JComboBox<String> yearDob, monthDob, dayDob, planYear,
planMonth, planDay;
    private JComboBox<String> planC;
    private JRadioButton otherButton, maleButton, femaleButton;
    private ButtonGroup button;
    private JButton clear, readFile, saveFile, display, addRegular, addPremium;
    private JFrame frame,rframe,pframe,readFrame;
    private JPanel panel,rpanel,ppanel;
    private JLabel ptitle,pId,pIdText,pName,pAmt, pLocation, pPhone,
pEmail,pStart, pDob, pGender,pStartText,pActive,pPlan,pAttendance,pLoyal;
    private JTextField pNameText, pLocationText ,pPhoneText,
pEmailText,pDobText,
pGenderText,pAttendText,pActiveText,pPlanText,pLoyalText;
    private JLabel title,rId,rIdText,rName, removal, rLocation, rPhone, rEmail,rStart,
rDob, rGender,rStartText,active,rPlan,attendance,removalReason,loyal;
    private JTextField rNameText,pAmtText, rLocationText,removalText
,rPhoneText, rEmailText,rDobText,
rGenderText,attendText,activeText,rPlanText,removalReasonText,loyalText;
    private JButton markAttendance, activateMembership,
deactivateMembership,revertMember,revertRegularMember,revertPremiumMem
ber;
    private JButton pMarkAttendance, pActivateMembership,
pDeactivateMembership;
    private JButton upgradePlan, due, discountBtn;
    private FileReader reading;
    private File file;
```

```
private FileWriter writing;
private JTextArea textArea;
private File Doc;
private ArrayList<GymMember> gym = new ArrayList<>();

public GymGUI()//constructor
{
    //creating new frame
    frame = new JFrame("GYM");
    panel = new JPanel();
    panel.setLayout(null);

    gymMember = new JLabel(" W Gym ");
    gymMember.setFont(new Font("Arial", Font.BOLD, 24));
    gymMember.setBounds(350, 20, 300, 30);
    panel.add(gymMember);

    memberInfo = new JLabel("Member's Info");
    memberInfo.setFont(new Font("Arial", Font.BOLD, 16));
    memberInfo.setBounds(350, 70, 200, 25);
    panel.add(memberInfo);

    // ID label
    id = new JLabel("ID");
    id.setBounds(50, 110, 100, 25);
    panel.add(id);

    idText = new JTextField();
    idText.setBounds(150, 110, 200, 25);
    panel.add(idText);
```

```
// Name label
name = new JLabel("Name");
name.setBounds(50, 145, 100, 25);
panel.add(name);

nameText = new JTextField();
nameText.setBounds(150, 145, 200, 25);
panel.add(nameText);

// Location
location = new JLabel("Location");
location.setBounds(50, 180, 100, 25);
panel.add(location);

locationText = new JTextField();
locationText.setBounds(150, 180, 200, 25);
panel.add(locationText);

// Phone
phone = new JLabel("Phone");
phone.setBounds(50, 215, 100, 25);
panel.add(phone);

phoneText = new JTextField();
phoneText.setBounds(150, 215, 200, 25);
panel.add(phoneText);

// Email
email = new JLabel("Email");
email.setBounds(50, 250, 100, 25);
panel.add(email);
```

```
        emailText = new JTextField();
        emailText.setBounds(150, 250, 200, 25);
        panel.add(emailText);

        // Date of Birth
        dob = new JLabel("DOB");
        dob.setBounds(50, 285, 100, 25);
        panel.add(dob);

        String[] year = new String[100];
        for (int i = 0; i < 100; i++) {//using for loop for ot to show the year until 100 years
            year[i] = String.valueOf(1980 + i);
        }
        yearDob = new JComboBox<>(year);
        yearDob.setBounds(150, 285, 70, 25);
        yearDob.insertItemAt("Year", 0);
        yearDob.setSelectedIndex(0);
        panel.add(yearDob);

        String[] month = new String[]{"Month", "1", "2", "3", "4", "5", "6", "7", "8", "9", "10", "11", "12"};//months list
        monthDob = new JComboBox<>(month);
        monthDob.setBounds(230, 285, 60, 25);
        monthDob.setSelectedIndex(0);
        panel.add(monthDob);

        String[] day = new String[31];//loop for days
        for (int i = 0; i < 31; i++) {
            day[i] = String.valueOf(i + 1);//string
```

```
}
dayDob = new JComboBox<>(day);
dayDob.setBounds(300, 285, 60, 25);
dayDob.insertItemAt("Day", 0);//default in combo box
dayDob.setSelectedIndex(0);
panel.add(dayDob);

//Gender
gender = new JLabel("Gender");
gender.setBounds(50, 320, 100, 25);
panel.add(gender);

button = new ButtonGroup();
maleButton = new JRadioButton("Male");
maleButton.setBounds(150, 320, 60, 25);
button.add(maleButton);
panel.add(maleButton);

femaleButton = new JRadioButton("Female");
femaleButton.setBounds(220, 320, 70, 25);
button.add(femaleButton);
panel.add(femaleButton);

otherButton = new JRadioButton("Others");
otherButton.setBounds(300, 320, 80, 25);
button.add(otherButton);
panel.add(otherButton);

gymInfo = new JLabel("Gym Inquiries");
gymInfo.setFont(new Font("Arial", Font.BOLD, 16));
gymInfo.setBounds(350, 360, 200, 25);
```

```
        panel.add(gymInfo);

        //Membership start date
        start = new JLabel("Membership Start Date");
        start.setBounds(50, 400, 200, 25);
        panel.add(start);

        String[] planYearArray = new String[50];
        for (int i = 0; i < 50; i++) {
            planYearArray[i]= (String.valueOf(2015+i));
        }
        planYear = new JComboBox<>(planYearArray);
        planYear.setBounds(230, 400, 70, 25);
        planYear.insertItemAt("Year", 0);
        planYear.setSelectedIndex(0);
        panel.add(planYear);

        String[] planMonthArray = new String[]{"Month", "1", "2", "3", "4", "5", "6", "7",
"8", "9", "10", "11", "12"};
        planMonth = new JComboBox<>(planMonthArray);
        planMonth.setBounds(310, 400, 60, 25);
        planMonth.setSelectedIndex(0);
        panel.add(planMonth);

        String[] planDayArray = new String[32];
        for (int i = 0; i < 31; i++) {
            planDayArray[i] = String.valueOf(i + 1);
        }
        planDay = new JComboBox<>(planDayArray);
        planDay.setBounds(380, 400, 60, 25);
        planDay.insertItemAt("Day", 0);
```

```
planDay.setSelectedIndex(0);
panel.add(planDay);

plan = new JLabel("Plan");//plan to select
plan.setBounds(50, 470, 100, 25);
panel.add(plan);

String[] planCombo = new String[]{"Basic", "Standard", "Deluxe"};
planC = new JComboBox<>(planCombo);
planC.setBounds(150, 470, 100, 25);
planC.addActionListener(this);
panel.add(planC);

price = new JLabel("Price");//price thet had to be field
price.setBounds(50, 505, 100, 25);
panel.add(price);

priceText = new JTextField();
priceText.setBounds(150, 505, 100, 25);
priceText.setEditable(false);
panel.add(priceText);

premiumPlan = new JLabel("Premium Plan:");
premiumPlan.setBounds(400, 505, 100, 25);
panel.add(premiumPlan);

premiumPlanText = new JTextField("50000");//non editable premium price
premiumPlanText.setBounds(500, 505, 200, 25);
premiumPlanText.setEditable(false);
panel.add(premiumPlanText);
//discount
```

```
referral = new JLabel("Referral Source");
referral.setBounds(50, 540, 100, 25);
panel.add(referral);

referralText = new JTextField();
referralText.setBounds(150, 540, 200, 25);
panel.add(referralText);

//Trainer 's name
trainer = new JLabel("Trainers Name:");
trainer.setBounds(400, 540, 200, 25);
panel.add(trainer);

trainerText = new JTextField();
trainerText.setBounds(500, 540, 150, 25);
panel.add(trainerText);

// Buttons
addRegular = new JButton("ADD REGULAR MEMBER");
addRegular.setBounds(90, 590, 200, 30);
addRegular.addActionListener(this);
panel.add(addRegular);

addPremium = new JButton("ADD PREMIUM MEMBER");
addPremium.setBounds(500, 590, 200, 30);
addPremium.addActionListener(this);
panel.add(addPremium);

clear = new JButton("Clear");
clear.setBounds(80, 650, 100, 25);
```

```
        clear.addActionListener(this);
        panel.add(clear);

        readFile = new JButton("Read from file");
        readFile.setBounds(250, 650, 130, 25);
        readFile.addActionListener(this);
        panel.add(readFile);

        saveFile = new JButton("Save to file");
        saveFile.setBounds(440, 650, 130, 25);
        saveFile.addActionListener(this);
        panel.add(saveFile);

        display = new JButton("Display");
        display.setBounds(600, 650, 100, 25);
        display.addActionListener(this);
        panel.add(display);

        frame.add(panel);
        frame.setSize(800, 800);
        frame.setDefaultCloseOperation(frame.EXIT_ON_CLOSE);//it  totally  shuts
the program running on the background
        frame.setResizable(false);
        frame.setLocationRelativeTo(null);// aligns in the center
        frame.setVisible(true);
    }

    public void RegularMemberAc(RegularMember gym){
        rframe = new JFrame("Member's ACCOUNT");
        rpanel= new JPanel();
        rpanel.setLayout(null);
```

```
title = new JLabel(" Regular Member Account");
title.setFont(new Font("Arial", Font.BOLD, 24));
title.setBounds(320, 20, 300, 30);
rpanel.add(title);

//id
rId = new JLabel("Id:");
rId.setBounds(90, 80, 180, 25);
rpanel.add(rId);

rIdText = new JLabel(String.valueOf(gym.getId()));
rIdText.setBounds(185, 80, 200, 25);
rpanel.add(rIdText);

//membershipStartDate
rStart =new JLabel ("Membership Start Date:");
rStart.setBounds(470, 80, 180, 25);
rpanel.add(rStart);

rStartText= new JLabel(gym.getMembershipStartDate());
rStartText.setBounds(630, 80, 180, 25);
rpanel.add(rStartText);

//Name

rName = new JLabel("Name");
rName.setBounds(90, 115, 100, 25);
rpanel.add(rName);

rNameText= new JTextField(gym.getName());
```

```
rNameText.setBounds(185, 115, 200, 25);
rNameText.setEditable(false);
rpanel.add(rNameText);

//location
rLocation = new JLabel("Location");
rLocation.setBounds(470, 115, 100, 25);
rpanel.add(rLocation);

rLocationText = new JTextField(gym.getLocation());
rLocationText.setBounds(630, 115, 200, 25);
rLocationText.setEditable(false);
rpanel.add(rLocationText);

//Phone
rPhone = new JLabel("Phone");
rPhone.setBounds(90,150, 100, 25);
rpanel.add(rPhone);

rPhoneText = new JTextField(gym.getPhone());
rPhoneText.setBounds(185,150, 200, 25);
rPhoneText.setEditable(false);
rpanel.add(rPhoneText);

//Email

rEmail = new JLabel("Email");
rEmail.setBounds(470, 150, 100, 25);
rpanel.add(rEmail);

rEmailText = new JTextField(gym.getEmail());
```

```
rEmailText.setBounds(630, 150, 200, 25);
rEmailText.setEditable(false);
rpanel.add(rEmailText);

//Date Of Birth
rDob = new JLabel("DOB");
rDob.setBounds(90, 185, 100, 25);
rpanel.add(rDob);

rDobText = new JTextField(gym.getDOB());
rDobText.setBounds(185, 185, 200, 25);
rDobText.setEditable(false);
rpanel.add(rDobText);
//Gender
rGender = new JLabel("Gender");
rGender.setBounds(470, 185, 100, 25);
rpanel.add(rGender);

rGenderText = new JTextField(gym.getGender());
rGenderText.setBounds(630, 185, 200, 25);
rGenderText.setEditable(false);
rpanel.add(rGenderText);

attendance=new JLabel("Attendance:");
attendance.setBounds(90,220,100,25);
rpanel.add(attendance);

attendText=new JTextField("0");
attendText.setBounds(185,220, 200,25);
attendText.setEditable(false);
rpanel.add(attendText);
```

```
loyal=new JLabel("Loyalty Points");
loyal.setBounds(470,220,100,25);
rpanel.add(loyal);

loyalText=new JTextField("0.0");
loyalText.setBounds(630,220,200,25);
loyalText.setEditable(false);
rpanel.add(loyalText);

active=new JLabel("Active Status:");
active.setBounds(90,255,100,25);
rpanel.add(active);

activeText=new JTextField("Inactive");
activeText.setBounds(185,255, 200,25);
activeText.setEditable(false);
rpanel.add(activeText);

rPlan=new JLabel("Plan:");
rPlan.setBounds(470,255,100,25);
rpanel.add(rPlan);

rPlanText=new JTextField("Regular");
rPlanText.setBounds(630,255, 200,25);
rPlanText.setEditable(false);
rpanel.add(rPlanText);

// Adding Buttons
markAttendance = new JButton("Mark Attendance");
markAttendance.setBounds(185, 300, 200, 25);
```

```
markAttendance.addActionListener(this);
rpanel.add(markAttendance);

upgradePlan= new JButton("Upgrade Plan");
upgradePlan.setBounds(470, 300, 200, 25);
upgradePlan.addActionListener(this);
rpanel.add(upgradePlan);

activateMembership = new JButton("Activate Membership");
activateMembership.setBounds(185, 335, 200, 25);
activateMembership.addActionListener(this);
rpanel.add(activateMembership);

deactivateMembership= new JButton("Deactivate Membership");
deactivateMembership.setBounds(470, 335, 200, 25);
deactivateMembership.addActionListener(this);
rpanel.add(deactivateMembership);

revertRegularMember=new JButton("Revert Member");
revertRegularMember.setBounds(185,370, 200,25);
revertRegularMember.addActionListener(this);
rpanel.add(revertRegularMember);

//Removal Reason
removalReason = new JLabel("Removal reason");
removalReason.setBounds(470, 370, 200, 25);
rpanel.add(removalReason);

removalReasonText = new JTextField();
removalReasonText.setBounds(470 , 400 , 200, 25);
rpanel.add(removalReasonText);
```

```
    //frames to display
    rframe.add(rpanel);
    rframe.setSize(900,550);
    rframe.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    rframe.setResizable(false);
    rframe.setLocationRelativeTo(null);
    rframe.setVisible(true);
}

public void PremiumMemberAc(PremiumMember gym){
    pframe = new JFrame("Member's ACCOUNT");
    ppanel= new JPanel();
    ppanel.setLayout(null);

    ptitle = new JLabel(" Premium Member Account");
    ptitle.setFont(new Font("Arial", Font.BOLD, 24));
    ptitle.setBounds(290, 20, 400, 30);
    ppanel.add(ptitle);

    //id
    pId = new JLabel("Id:");
    pId.setBounds(90, 80, 180, 25);
    ppanel.add(pId);

    pIdText = new JLabel(String.valueOf(gym.getId()));
    pIdText.setBounds(185, 80, 200, 25);
    ppanel.add(pIdText);

    //membershipStartDate
    pStart =new JLabel ("Membership Start Date:");
    pStart.setBounds(470, 80, 180, 25);
```

```
ppanel.add(pStart);

pStartText= new JLabel(gym.getMembershipStartDate());
pStartText.setBounds(630, 80, 180, 25);
ppanel.add(pStartText);

//Name

pName = new JLabel("Name");
pName.setBounds(90, 115, 100, 25);
ppanel.add(pName);

pNameText= new JTextField(gym.getName());
pNameText.setBounds(185, 115, 200, 25);
pNameText.setEditable(false);
ppanel.add(pNameText);

//location
pLocation = new JLabel("Location");
pLocation.setBounds(470, 115, 100, 25);
ppanel.add(pLocation);

pLocationText = new JTextField(gym.getLocation());
pLocationText.setBounds(630, 115, 200, 25);
pLocationText.setEditable(false);
ppanel.add(pLocationText);

//Phone
pPhone = new JLabel("Phone");
pPhone.setBounds(90,150, 100, 25);
ppanel.add(pPhone);
```

```
pPhoneText = new JTextField(gym.getPhone());
pPhoneText.setBounds(185,150, 200, 25);
pPhoneText.setEditable(false);
ppanel.add(pPhoneText);

//Email

pEmail = new JLabel("Email");
pEmail.setBounds(470, 150, 100, 25);
ppanel.add(pEmail);

pEmailText = new JTextField(gym.getEmail());
pEmailText.setBounds(630, 150, 200, 25);
pEmailText.setEditable(false);
ppanel.add(pEmailText);

//Date Of Birth
pDob = new JLabel("DOB");
pDob.setBounds(90, 185, 100, 25);
ppanel.add(pDob);

pDobText = new JTextField(gym.getDOB());
pDobText.setBounds(185, 185, 200, 25);
pDobText.setEditable(false);
ppanel.add(pDobText);
//Gender
pGender = new JLabel("Gender");
pGender.setBounds(470, 185, 100, 25);
ppanel.add(pGender);
```

```
pGenderText = new JTextField(gym.getGender());
pGenderText.setBounds(630, 185, 200, 25);
pGenderText.setEditable(false);
ppanel.add(pGenderText);
//attendance
pAttendance=new JLabel("Attendance:");
pAttendance.setBounds(90,220,100,25);
ppanel.add(pAttendance);

pAttendText=new JTextField("0");
pAttendText.setBounds(185,220, 200,25);
pAttendText.setEditable(false);
ppanel.add(pAttendText);
//loyalty points
pLoyal=new JLabel("Loyalty Points");
pLoyal.setBounds(470,220,100,25);
ppanel.add(pLoyal);

pLoyalText=new JTextField("0.0");
pLoyalText.setBounds(630,220,200,25);
pLoyalText.setEditable(false);
ppanel.add(pLoyalText);
//actice status
pActive=new JLabel("Active Status:");
pActive.setBounds(90,255,100,25);
ppanel.add(pActive);

pActiveText=new JTextField("Inactive");
pActiveText.setBounds(185,255, 200,25);
pActiveText.setEditable(false);
ppanel.add(pActiveText);
```

```
pPlan=new JLabel("Plan:");
pPlan.setBounds(470,255,100,25);
ppanel.add(pPlan);

pPlanText=new JTextField("Premium");
pPlanText.setBounds(630,255, 200,25);
pPlanText.setEditable(false);
ppanel.add(pPlanText);

// Adding Buttons
pMarkAttendance = new JButton("Mark Attendance");
pMarkAttendance.setBounds(185, 300, 200, 25);
pMarkAttendance.addActionListener(this);
ppanel.add(pMarkAttendance);

discountBtn= new JButton("Discount");
discountBtn.setBounds(470, 300, 200, 25);
discountBtn.addActionListener(this);
ppanel.add(discountBtn);

pActivateMembership = new JButton("Activate Membership");
pActivateMembership.setBounds(185, 335, 200, 25);
pActivateMembership.addActionListener(this);
ppanel.add(pActivateMembership);

pDeactivateMembership= new JButton("Deactivate Membership");
pDeactivateMembership.setBounds(470, 335, 200, 25);
pDeactivateMembership.addActionListener(this);
ppanel.add(pDeactivateMembership);
```

```
    //pay due amount
    pAmt = new JLabel("Amount");
    pAmt.setBounds(185, 370, 200, 25);
    ppanel.add(pAmt);

    pAmtText = new JTextField();
    pAmtText.setBounds(185 , 405 , 200, 25);
    ppanel.add(pAmtText);
    //revert member
    revertPremiumMember=new JButton("Revert Member");
    revertPremiumMember.setBounds(470,370, 200,25);
    revertPremiumMember.addActionListener(this);
    ppanel.add(revertPremiumMember);

    due=new JButton("Pay Due Amount");
    due.setBounds(470,405, 200,25);
    due.addActionListener(this);
    ppanel.add(due);

    pframe.add(ppanel);
    pframe.setSize(900,550);
    pframe.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    pframe.setResizable(false);
    pframe.setLocationRelativeTo(null);
    pframe.setVisible(true);
}
//methof to clear everything
public void clearEverything(){
    idText.setText("");
    nameText.setText("");
    locationText.setText("");
```

```
        phoneText.setText("");
        emailText.setText("");
        priceText.setText("");
        referralText.setText("");
        trainerText.setText("");
        button.clearSelection();
        yearDob.setSelectedIndex(0);
        monthDob.setSelectedIndex(0);
        dayDob.setSelectedIndex(0);
        planYear.setSelectedIndex(0);
        planMonth.setSelectedIndex(0);
        planDay.setSelectedIndex(0);
        planC.setSelectedIndex(0);
    }
    //helper method
    private GymMember searchMember(int id) {
        for (GymMember exist : gym) {
            if (exist.getId() == id) {
                return exist;
            }
        }
        return null;
    }
    //reader
    public void detailsFromFile() {
        JFrame readFrame = new JFrame("Saved Member Details");
        readFrame.setSize(1000, 500);
        readFrame.setLocationRelativeTo(null);
        readFrame.setLayout(null);

        textArea = new JTextArea();
```

```
        textArea.setFont(new Font("Monospaced", Font.PLAIN, 12));
        textArea.setEditable(false);
        textArea.setBounds(10, 10, 960, 440);

        try {
            file = new File("MemberDetails.txt");

            if (!file.exists() || file.length() == 0) {
                textArea.setText("No saved member details found.");
            } else {
                reading = new FileReader(file);
                int chart;
                String content="";

                while ((chart = reading.read()) != -1) {
                    content+=(char) chart;
                }//read until the last character

                textArea.setText(content);
                reading.close();
            }
        } catch (IOException ex) {
            JOptionPane.showMessageDialog(readFrame, "Error reading file: " +
ex.getMessage());
        }

        readFrame.add(textArea);
        readFrame.setVisible(true);
    }
    //save
    public void saveToFile() {
```

```
        try {
            File file = new File("MemberDetails.txt");
            writing = new FileWriter(file, false); // true for append mode

            // Write header only if the file is empty
            if (file.length() == 0) {
                writing.write(String.format("%-5s %-15s %-15s %-15s %-15s %-15s %-
10s %-10s %-10s %-15s %-10s %-15s %-15s %-15s\n",
                        "ID", "Name", "Location", "Phone", "Email", "Start Date", "Plan",
"Price", "Attendance",
                        "Loyalty", "Active", "FullPayment", "Discount", "Paid"));
            }
            if (gym.size() == 0) {
                JOptionPane.showMessageDialog(frame, "No members to save.");
                writing.close();
                return;
            }
            for (GymMember check : gym) {
                if (check instanceof RegularMember) {
                    RegularMember r = (RegularMember) check;
                    writing.write(String.format("%-5d %-15s %-15s %-15s %-15s %-15s
%-10s %-10s %-10s %-15s %-10s %-15s %-15s %-15s\n",//format of printing
                            r.getId(), r.getName(), r.getLocation(), r.getPhone(),
r.getEmail(), r.getMembershipStartDate(),
                            r.getPlan(), r.getPrice(), r.getAttendance(), r.getLoyaltyPoints(),
//getter method
                            r.getActiveStatus(), "N/A", "N/A", "N/A"));
                    clearEverything();
                } else if (check instanceof PremiumMember) {
                    PremiumMember pre = (PremiumMember) check;
```

```
                writing.write(String.format("%-5d %-15s %-15s %-15s %-15s %-15s
%-10s %-10.2f %-10s %-15s %-10s %-15s %-15.2f %-15.2f\n",
                        pre.getId(), pre.getName(), pre.getLocation(), pre.getPhone(),
pre.getEmail(), pre.getMembershipStartDate(),
                        "Premium", pre.getPremiumCharge(), pre.getAttendance(),
pre.getLoyaltyPoints(),
                        pre.getActiveStatus(), pre.getIsFullPayment(),
pre.getDiscountAmount(), pre.getPaidAmount()));

            }
            clearEverything();
        }
        System.out.println("Number of members to save: " + gym.size());

        //  Flush before closing to ensure data is saved
        writing.flush();
        writing.close();

        JOptionPane.showMessageDialog(frame, "Data saved successfully!");
      } catch (IOException ex) {
        JOptionPane.showMessageDialog(frame, "Error saving data: " +
ex.getMessage());
      }
   }

   public void actionPerformed(ActionEvent event) {
      //selecting the price according to the plan
      String selectedPlan = (String) planC.getSelectedItem();
      switch (selectedPlan) {
        case "Basic":
          priceText.setText("6500");
```

```
            break;
        case "Standard":
            priceText.setText("8500");

            break;
        case "Deluxe":
            priceText.setText("10500");

            break;
        default:
            priceText.setText(" ");

    }

    // To Display
    if (event.getSource() == display) {
        String abc = idText.getText();
        if (abc.isEmpty()) {
            JOptionPane.showMessageDialog(frame, "Empty id field");
            return;
        }
        try {
            int id = Integer.parseInt(abc); // Parse the ID from input
            boolean found=false;//default value false
            for (GymMember exist : gym) {//if the id already exist in arraylist

                if (exist.getId() == id) {//comparing
                    if(exist instanceof RegularMember){
                        RegularMember regM = (RegularMember) exist;
                        RegularMemberAc(regM);
```

```
                    }
                    if(exist instanceof PremiumMember){
                        PremiumMember preM = (PremiumMember) exist;
                        PremiumMemberAc(preM);
                    }
                    //MemberAc(exist); //does exists
                    found=true;//becomes true
                    break;
                }
            }
            if(!found){//if it does not exist
                JOptionPane.showMessageDialog(frame, "This ID doesnot exist please enter the existing ID");
            }
        } catch (NumberFormatException e) {//if anything else added than number then this will run
            JOptionPane.showMessageDialog(frame, "Please enter a valid numeric ID.");
        }
    }
    //Clear button to remove data temporarily
    else if (event.getSource() == clear) {
        clearEverything();
    }
    //Adding regular member
    else if (event.getSource() == addRegular) {
        try {//if any field id null then popup will be displayed
            int id=Integer.parseInt(idText.getText());
            if(idText.getText().isEmpty() ||
            nameText.getText().isEmpty() ||
            locationText.getText().isEmpty() ||
```

```
            phoneText.getText().isEmpty() ||
            emailText.getText().isEmpty() ||
            referralText.getText().isEmpty() ||
            planC.getSelectedItem() == null ||
            yearDob.getSelectedItem() == null ||
            monthDob.getSelectedItem() == null ||
            dayDob.getSelectedItem() == null ||
            planYear.getSelectedItem() == null ||
            planMonth.getSelectedItem() == null ||
            planDay.getSelectedItem() == null ||
            (!maleButton.isSelected() && !femaleButton.isSelected() &&
!otherButton.isSelected()))
            {

                JOptionPane.showMessageDialog(frame, "Please fill in all required
fields.");
                return;
            }
            // Validate phone number
            String phone = phoneText.getText();
            if (!phone.matches("^[0-9]{10}$")) {
                JOptionPane.showMessageDialog(frame, "Please enter a valid
phone number.");
                return;
            }
            // Validate email
            String email = emailText.getText();
            if (!email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$")) {
                JOptionPane.showMessageDialog(frame, "Please enter a valid email
address.");
                return;
```

```
            }

            String gender = maleButton.isSelected() ? "Male" :femaleButton.isSelected() ? "Female" :otherButton.isSelected() ? "Others" : "Unspecified";
            String dob = yearDob.getSelectedItem() + "/ " +monthDob.getSelectedItem() + "/ " + dayDob.getSelectedItem();//selected item is stored
            String start= planYear.getSelectedItem() + "/" +planMonth.getSelectedItem() + "/" + planDay.getSelectedItem();
            //all the imformation is stored in an object of child class
            RegularMember fault = new RegularMember(id, nameText.getText(), locationText.getText(),phoneText.getText(), emailText.getText(), gender,dob,start,referralText.getText());
            for (GymMember exit : gym) {//if the id already exists then compares
                if (exit.getId() == id) {
                    JOptionPane.showMessageDialog(frame, "ID already exists.");
                    return; // Don't add if ID exists
                }
            }
            gym.add(fault);//adding fault in gym arraylist
            //Clear

            JOptionPane.showMessageDialog(frame, "Regular member added.");
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(frame, "Please fill all the details.");
        }
    }
    else if (event.getSource() == addPremium) {
        //if any field is empty then the popup will be shown
        try {
```

```
            int id=Integer.parseInt(idText.getText());
            if(idText.getText().isEmpty() ||
            nameText.getText().isEmpty() ||
            locationText.getText().isEmpty() ||
            phoneText.getText().isEmpty() ||
            emailText.getText().isEmpty() ||
            trainerText.getText().isEmpty()||
            planC.getSelectedItem() == null ||
            yearDob.getSelectedItem() == null ||
            monthDob.getSelectedItem() == null ||
            dayDob.getSelectedItem() == null ||
            planYear.getSelectedItem() == null ||
            planMonth.getSelectedItem() == null ||
            planDay.getSelectedItem() == null ||
            (!maleButton.isSelected() && !femaleButton.isSelected() &&
!otherButton.isSelected()))
            {

               JOptionPane.showMessageDialog(frame, "Please fill in all required
fields.");
               return;
            }
            // Validate phone number
            String phone = phoneText.getText();
            if (!phone.matches("^[0-9]{10}$")) {
               JOptionPane.showMessageDialog(frame, "Please enter a valid
phone number (10 digits).");
               return;
            }

            // Validate email
```

```
            String email = emailText.getText();
            if (!email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$")) {
                JOptionPane.showMessageDialog(frame, "Please enter a valid email
address.");
                return;
            }

            String gender = maleButton.isSelected() ? "Male"
:femaleButton.isSelected() ? "Female" :otherButton.isSelected() ? "Others" :
"Unspecified";
            String dob = yearDob.getSelectedItem() + "/ "
+monthDob.getSelectedItem() + "/ " + dayDob.getSelectedItem();//Selected items
is store
            String start= planYear.getSelectedItem() + "/"
+planMonth.getSelectedItem() + "/" + planDay.getSelectedItem();
            //storing the value of every cpmponent in an object
            PremiumMember fault = new PremiumMember(id, nameText.getText(),
locationText.getText(),phoneText.getText(), emailText.getText(),
gender,dob,start,trainerText.getText());
            //for loop to compare the id if it already exists
            for (GymMember exit : gym) {
                if (exit.getId() == id) {
                    JOptionPane.showMessageDialog(frame, "ID already exists.");
                    return; // Don't add if ID exists
                }
            }

            gym.add(fault);
            //clear everthing

            JOptionPane.showMessageDialog(frame, "Premium member added.");
```

```
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(frame, "Invaild");
            }
        }
        else if(event.getSource()==readFile){
            detailsFromFile();//calling the function
        }
        else if(event.getSource() == saveFile) {
            saveToFile();
        }
        else if(event.getSource()== markAttendance){
            GymMember
attend=searchMember(Integer.parseInt(rIdText.getText()));//comparing id
            if (attend instanceof RegularMember) {
                RegularMember reg = (RegularMember) attend;//downcasting
                if(reg.activeStatus){//mark attendance true
                    reg.markAttendance();
                    attendText.setText(String.valueOf(reg.getAttendance()));
                    loyalText.setText(String.valueOf(reg.getLoyaltyPoints()));
                    JOptionPane.showMessageDialog(rframe, "Attendance marked for
ID: " +attend.getId() );
                    saveToFile();
                }
                else{
                    JOptionPane.showMessageDialog(rframe, "Attendance cannot be
marked.\nAccount is not activated");
                }
            }

        }
```

```
        else if(event.getSource()== pMarkAttendance){//PREMIUM MARK
ATTENDANCE
            GymMember attend=searchMember(Integer.parseInt(pIdText.getText()));
            if (attend instanceof PremiumMember) {
                PremiumMember pre = (PremiumMember) attend;//DOWN CASTING
                if(pre.activeStatus){
                    pre.markAttendance();
                    pAttendText.setText(String.valueOf(pre.getAttendance()));
                    pLoyalText.setText(String.valueOf(pre.getLoyaltyPoints()));
                    JOptionPane.showMessageDialog(pframe, "Attendance marked for "
+attend.getId() );
                    saveToFile();
                }
                else{
                    JOptionPane.showMessageDialog(pframe, "Attendance cannot be
marked.\nAccount is not activated");
                }
            }
        }

        else if(event.getSource()==revertPremiumMember){//REVERT MEMBER OF
PREMIUM
            GymMember revert = searchMember(Integer.parseInt(pIdText.getText()));
            if (revert instanceof PremiumMember) {
                PremiumMember pre = (PremiumMember) revert;
                if(pre.activeStatus){
                    pre.revertPremiumMember();
                    pAttendText.setText(String.valueOf(pre.getAttendance()));
                    pLoyalText.setText(String.valueOf(pre.getLoyaltyPoints()));
                    pActiveText.setText("Inactive");//SET TO INACTIVE
```

```
                JOptionPane.showMessageDialog(pframe,"Membership reverted");
                saveToFile();
            }else{
                JOptionPane.showMessageDialog(pframe,"Membership not
activated");
            }
        }

    }
    else if(event.getSource()==revertRegularMember){//REVERT MEMBER
FOR REGULAR
        GymMember revert = searchMember(Integer.parseInt(rIdText.getText()));
        String fReason = removalReasonText.getText();//REASON MUST BE
THERE

        if (revert instanceof RegularMember) {
            if(fReason==null  || fReason.isEmpty()){
                JOptionPane.showMessageDialog(rframe,"Please fill the reason");

            }else{
                try{
                    Integer.parseInt(fReason);
                    JOptionPane.showMessageDialog(rframe,"The reason must be
valid");
                }

                catch(NumberFormatException e){
                    RegularMember reg = (RegularMember) revert;
                    if(reg.activeStatus){
                        reg.revertRegularMember(fReason);
                        rPlanText.setText(reg.getPlan());
```

```
                    attendText.setText(String.valueOf(reg.getAttendance()));
                    loyalText.setText(String.valueOf(reg.getLoyaltyPoints()));
                    activeText.setText("Inactive");
                    removalReasonText.setText("");

                    JOptionPane.showMessageDialog(rframe, "Membership
reverted. \nReason: " + fReason);
                    saveToFile();//SAVE THE FILE IN THE TEXT FILE
                }else{
                    JOptionPane.showMessageDialog(rframe, "Membership not
activated");
                }
            }
        }
    }

    else if(event.getSource()==activateMembership){//ACTIVATE FOR
REGULAR
        GymMember active=searchMember(Integer.parseInt(rIdText.getText()));
        if (!active.getActiveStatus()){
            active.activateMembership();
            JOptionPane.showMessageDialog(rframe,"Membership Activated.");
            activeText.setText("Active");
            saveToFile();
        }else{
            JOptionPane.showMessageDialog(rframe,"Membership already
Activated.");
        }

    }
```

```
        else if(event.getSource()==pActivateMembership){//ACTIVATE FOR
PREMIUM
            GymMember active=searchMember(Integer.parseInt(pIdText.getText()));
            if (!active.getActiveStatus()){
                active.activateMembership();
                JOptionPane.showMessageDialog(pframe,"Membership Activated.");
                pActiveText.setText("Active");
                saveToFile();
            }else{
                JOptionPane.showMessageDialog(pframe,"Membership already
Activated.");
            }

        }
        else if(event.getSource()==pDeactivateMembership){//DEACTIVATE FOR
PREMIUM
            GymMember
deactive=searchMember(Integer.parseInt(pIdText.getText()));
            if(deactive.getActiveStatus()){
                deactive.deactivateMembership();
                JOptionPane.showMessageDialog(pframe,"Membership
Deactivated.");
                pActiveText.setText("Inactive");
                saveToFile();
            }
            else{
                JOptionPane.showMessageDialog(pframe,"Membership already
Deactivated.");
            }
        }
```

```
        else if(event.getSource()==deactivateMembership){//DEACTIVATE FOR
REGULAR
            GymMember
deactive=searchMember(Integer.parseInt(rIdText.getText()));
            if(deactive.getActiveStatus()){
                deactive.deactivateMembership();
                JOptionPane.showMessageDialog(rframe,"Membership Deactivated.");
                activeText.setText("Inactive");
                saveToFile();
            }

            else{
                JOptionPane.showMessageDialog(rframe,"Membership already
Deactivated.");
            }
        }

        else if(event.getSource()==upgradePlan){//UPGRADE PLAN
            GymMember
upgrade=searchMember(Integer.parseInt(rIdText.getText()));

            if(upgrade instanceof RegularMember){

                RegularMember reg=(RegularMember) upgrade;
                if(reg.activeStatus){
                    String newPlan=JOptionPane.showInputDialog(rframe,"Enter New
Plan(Basic,Standard,Deluxe):");

                    if(newPlan!=null){
                        String finall=reg.upgradePlan(newPlan);
                        if(finall.equals("Sucessfully updated the membership!!")){
```

```
                    JOptionPane.showMessageDialog(rframe,finall+"\nNew
plan:"+reg.getPlan()+"\nNew price"+reg.getPrice());
                    rPlanText.setText(String.valueOf(reg.getPlan()));

                }
                else{JOptionPane.showMessageDialog(rframe,finall);
                    saveToFile();
                }
            }
            else{
                JOptionPane.showMessageDialog(rframe,"Empty input");
            }
        }
        else{
            JOptionPane.showMessageDialog(rframe,"Membership not
activated.");
        }
    }
}
else if(event.getSource()==due){//PAY DUE AMOUNT
    GymMember due = searchMember(Integer.parseInt(pIdText.getText()));
    String amt =pAmtText.getText();
    if(due.activeStatus){
        if(amt.isEmpty()){
            JOptionPane.showMessageDialog(pframe,"Please Enter the
balance.");
            return;
        }
        double left;
        try{
            left=Double.parseDouble(amt);
```

```
            }
            catch(NumberFormatException e){
                JOptionPane.showMessageDialog(pframe,"Please Enter A Valid
Number.");
                return;
            }
            if (due instanceof PremiumMember) {
                PremiumMember pre = (PremiumMember) due;
                String total=pre.payDueAmount(left);
                JOptionPane.showMessageDialog(pframe,total);
                pAmtText.setText("");
                saveToFile();
            }

        }
        else{
            JOptionPane.showMessageDialog(pframe,"Membership not
activated.");
        }
    }
    else if(event.getSource()==discountBtn){//DISCOUNT BUTTON
        GymMember
discount=searchMember(Integer.parseInt(pIdText.getText()));
        if(discount instanceof PremiumMember){
            PremiumMember dis=(PremiumMember) discount;
            String display=dis.calculateDiscount();
            JOptionPane.showMessageDialog(pframe,display);
            saveToFile();
        }
    }
  }
```

```
    public static void main(String args[])
    {
        new GymGUI();//calling the main class
    }
}
```